



Contents

Part I

Foundations

Chapter 1

Why OOP Exists

1.1 The Question Nobody Starts With

Deep Insight

Before learning OOP we must answer:

Why was OOP invented at all?

Every powerful idea appears because an older approach started breaking.

Common Trap

Most students begin with:

```
1 class Student {  
2  
3 }
```

before understanding what problem a class solves.

This is backwards.

Rosetta Rule

Never learn a programming construct before learning the problem it was invented to solve.

1.2 The World Before OOP

Deep Insight

Programming originally revolved around procedures.

Data lived in one place.

Functions lived elsewhere.

The program was viewed as:

Data + Functions

Mental Model

Imagine a city.

All citizens live in one district. All jobs exist in another district.

Whenever something must happen, people travel across the city.

> Small city: Works fine

> Huge city: Chaos

1.3 The C Programmer's View

```
1 struct Student {  
2     char name[100];  
3     int age;  
4 };  
5  
6 void printStudent(...);  
7 void updateStudent(...);  
8 void saveStudent(...);  
9 void deleteStudent(...);
```

Deep Insight

Notice something.

The data **does not** know who is *allowed* to touch it.

The functions do not belong to the data.

They merely operate on it.

1.4 The Hidden Scaling Problem

Mental Model

Suppose we have:

> 100 variables 100 functions

Everything seems manageable.

Now imagine:

> 10000 variables 50000 functions

Questions become difficult:

- * Who owns this data?
- * Who can modify it?
- * Which function should update it?
- * Which function already updated it?
- * Which function broke it?

Rosetta Rule

Programming becomes difficult long before algorithms become difficult.
Complexity is usually organizational.

1.5 The Core Discovery

Deep Insight

State and behaviour should stay together.
This single sentence created OOP.

$$\textit{Object} = \textit{State} + \textit{Behaviour}$$

Everything else is a consequence.

1.6 State

Deep Insight

State answers:
What does this thing know?

Examples:

```
1 name
2 age
3 balance
4 cgpa
5 position
6 velocity
```

Rosetta Rule

State is memory.

1.7 Behaviour

Deep Insight

Behaviour answers: What can this thing do?

Examples:

```
1 deposit()
2 withdraw()
3 study()
4 move()
5 rotate()
```

Rosetta Rule

Behaviour is controlled change of state.

1.8 The Most Important Equation

$$\textit{Object} = \textit{State} + \textit{Behaviour}$$

Mental Model

A bank account is **not just**:

[balance]

A bank account is:

[State + Operations + Rules]

where:

- **State:** balance
- **Operations:** deposit () and withdraw ()
- **Rules:** logic that protects and controls the balance

Mental Model:

[Object → Data + Behavior + Invariants]

1.9 Why Objects Exist

Deep Insight

Objects exist because responsibility must exist.

Example:

Question: Who should ensure? : $balance \geq 0$

Random function?

Random module?

No The account itself.

Rosetta Rule

Objects are responsible entities.
Not data containers.

1.10 The Beginner Mistake

Beginners think:

```
1 Object = Data
```

Experts think:

```
1 Object = Responsibility
```

Huge difference.

1.11 Reality vs OOP

Common Trap

Many books say:

:: Dog Object

:: Car Object

:: Student Object

This creates the wrong impression. OOP is **not about copying** reality.

Examples of useful objects:

```
1 Compiler
2 DatabaseConnection
3 ThreadPool
4 HashMap
5 Lexer
6 Parser
```

These are not physical objects. Yet OOP models them perfectly.

1.12 Complexity Is The Real Enemy

Deep Insight

The enemy of software is not syntax.
The enemy is complexity.

Everything in OOP is an attack against complexity.
Later we will see:

Encapsulation → Protect Complexity

Abstraction → Hide Complexity

Inheritance → Reuse Complexity

Polymorphism → Manage Complexity

Composition → Organize Complexity

1.13 Rosetta Summary

Rosetta Rule

OOP was not invented to create classes. OOP was invented to control complexity.

Rosetta Rule

Object = State + Behaviour

Rosetta Rule

State answers:
‘What do I **know**?’

Rosetta Rule

Behaviour answers:
‘What **can** I do?’

Rosetta Rule

The true purpose of OOP is:
Responsibilities, Boundaries, Ownership.

Part II

Objects and Memory

Chapter 2

Objects, Identity, State and Responsibility

2.1 The Second Wrong Idea

Common Trap

Many students think:

Object = Memory Block

This is false.

A memory block is merely storage.

An object is a much richer concept.

Rosetta Rule

Objects are not created because memory exists.

Objects are created because **responsibilities** exist.

2.2 What Is An Object?

Deep Insight

An object is a self-contained entity that:

- owns state
- performs behaviour
- maintains rules
- has identity

Mental Model

Think of a small kingdom.
The kingdom owns resources.
The kingdom has laws.
The kingdom has responsibilities.
The kingdom protects itself.
That kingdom is an object.

2.3 The Four Pillars Of An Object

Every useful object has:

Identity

State

Behaviour

Responsibility

think about all four.

2.4 Identity

Deep Insight

Identity answers:

Who am I?

Example:

Two students may have:

- same name
- same age
- same CGPA

Yet they are different students.

Why?

Identity.

Mental Model

Two houses can look identical.
Different addresses.
Different houses.
Identity survives even when state is identical.

2.5 State

Deep Insight

State answers:

What do I know?

Example:

```
1 name
2 age
3 cgpa
```

State is memory.
State changes over time.

2.6 Behaviour

Deep Insight

Behaviour answers:

What can I do?

Examples:

```
1 study()
2 graduate()
3 registerCourse()
```

2.7 Responsibility

Deep Insight

Responsibility answers:

What am I accountable for?

This is the most neglected idea in beginner OOP.
Example:
BankAccount
State:

```
1 balance
```

Behaviour:

```
1 deposit ()
2 withdraw ()
```

Responsibility:

$$balance \geq 0$$

must remain true.

2.8 The Expert View

Beginners see:

```
1 class Student
2 {
3     ...
4 }
```

Experts immediately ask:

- What does it know?
- What can it do?
- What rules must remain true?
- What is it responsible for?

2.9 Objects Protect Invariants

Deep Insight

Invariant:

A condition that must always remain true.

Examples:

$$Age \geq 0$$

$$Balance \geq 0$$

$$Marks \leq 100$$

Good objects protect invariants.

2.10 A Hidden Difference Between C and OOP

In C:

```
1 student.age = -50;
```

Nothing stops you.

In OOP:

The object should protect itself.

This is a huge philosophical shift.

2.11 Object Thinking

Whenever you encounter a new system:

Do not ask:

What data structures exist?

Ask:

Who owns the responsibility?

This single question often reveals the design.

2.12 Rosetta Rule

Good OOP is not:

Finding classes

Good OOP is:

Finding responsibilities

Classes come later.

2.13 The Responsibility Game

Suppose we are designing:

```
1 Library System
```

Question:

Who should know:

```
1 Book Availability?
```

Book?

Library?

Shelf?

Database?

Finding the correct owner is OOP design.

2.14 Rosetta Summary

Rosetta Rule

Every object has:
Identity
State
Behaviour
Responsibility

Rosetta Rule

State answers:
What do I know?

Rosetta Rule

Behaviour answers:
What can I do?

Rosetta Rule

Responsibility answers:
What must I guarantee?

Rosetta Rule

Good OOP begins with responsibilities.
Not classes.

Chapter 3

Class vs Object — Static, Instance and Memory

3.1 The Seventeenth Wrong Idea

Common Trap

Many beginners think:
Class = Object
False.
A class is not an object. An object is not a class.

3.2 The Blueprint Analogy

Textbooks often say:

Class is a blueprint.

Helpful.
But incomplete.
A better view:

Class = Type + Rules + Structure + Behavior

Object is:

Concrete Instance

3.3 First Example

```
1 class Student
2 {
3     String name;
4     int age;
5 }
```

Question:

How many students exist?

Answer:

Zero.

Class declaration creates no objects.

3.4 Object Creation

```
1 Student s1 =  
2 new Student();
```

Now:

One object exists.

Class Definition

```
Student  
{  
  name  
  age  
}
```

```
  |  
  |  
  v
```

Object

```
+-----+  
| name=null |  
| age=0     |  
+-----+
```

3.5 Two Students

```
1 Student s1 = new Student();  
2 Student s2 = new Student();
```

Memory:

s1 ---> Object 1

s2 ---> Object 2

Each object has independent state.

Changing:

```

1 s1.age
2 \Enforced{minted}
3
4 does not affect:
5
6 \begin{minted}{java}
7 s2.age

```

3.6 Instance Variables

Observe:

```

1 class Student
2 {
3     String name;
4     int age;
5 }

```

These belong to:

Objects

Not class itself.
Therefore called:

Instance Variables

Each object owns its own copy.

3.7 The Problem

Suppose:

```

1 class Student
2 {
3     String name;
4 }

```

Now:

100000

students created.

Question:

What if we want to count total students?

Should each object store:

```

1 totalStudents

```

?

No.

There should be only one copy.

3.8 Static Variables

Java provides:

```
1 class Student
2 {
3     static int totalStudents;
4 }
```

Now:

totalStudents

belongs to class.

Not individual objects.

Only one copy exists.

3.9 Mental Model

Mental Model

Instance Variable = One copy per object.

Static Variable = One copy per class.

3.10 Memory Diagram

Student Class

```
totalStudents
|
v
42
```

Object 1

```
name = Alice
```

Object 2

```
name = Bob
```

Object 3

```
name = Charlie
```


Only one:

totalStudents

Many:

name

3.11 Static Methods

Suppose:

```
1 class MathUtil
2 {
3     static int square(int x)
4     {
5         return x*x;
6     }
7 }
```

Call:

```
1 MathUtil.square(5);
```

No object required.

Why?

Because method belongs to class.

Not instance.

3.12 Instance Methods

Observe:

```
1 class Student
2 {
3     void study()
4     {
5     }
6 }
```

Now:

```
1 Student s =
2 new Student();
3
4 s.study();
```

Requires object.

Because behavior depends on instance.

3.13 The this Keyword

Question:

Inside an object,
how does Java refer to the current object?

Answer:

```
1 this
```

3.14 Example

```
1 class Student
2 {
3     String name;
4
5     void setName(String name)
6     {
7         this.name = name;
8     }
9 }
```

Without:

```
1 this
```

ambiguity exists.

3.15 Mental Model

Mental Model

this = Current Object

If:

```
1 s1.study ()
```

Then inside study ():

this = s1

3.16 The Hidden Parameter

Many languages internally treat:

```
1 obj.method ()
```

as:

```
1 method(obj)
```

Conceptually:

this

is that hidden object.

3.17 Static Cannot Use this

Observe:

```
1 static void test ()
2 {
3     this.x = 5;
4 }
```

Illegal.

Why?

Static method belongs to class.

No object exists.

No current object.

Therefore:

this

has no meaning.

3.18 The Famous Error

```
1 non-static variable cannot
2 be referenced from a static context
```

This error appears everywhere.

Reason:

Static code does not know which object you mean.

3.19 main () Is Static

Observe:

```
1 public static void main(String[] args)
2 {
3 }
```

Question:

Why static?

Because JVM must start somewhere.

Before any objects exist.

Therefore:

main()

belongs to class.

3.20 The super Keyword

Previously:

```
1 this
```

means:
Current Object.
Now:

```
1 super
```

means:
Parent View.
Example:

```
1 super.speak();
```

Calls parent implementation.

3.21 Class Loading

Before object creation:

JVM loads class metadata.
Think:

Load Class

↓

Initialize Static Members

↓

Create Objects

This order explains many strange bugs.

3.22 Static Initialization Block

```
1 class Test
2 {
3     static
4     {
```

```

5      System.out.println("Loaded");
6  }
7  }

```

Runs once.
 When class loads.
 Not when objects are created.

3.23 Rosetta Rules

Rosetta Rule

Class defines structure.

Rosetta Rule

Objects contain state.

Rosetta Rule

Instance variables belong to objects.

Rosetta Rule

Static variables belong to classes.

Rosetta Rule

Instance methods require objects.

Rosetta Rule

Static methods do not.

Rosetta Rule

this means current object.

Rosetta Rule

super means parent view.

Rosetta Rule

main() is static because JVM starts before any objects exist.

3.24 Mental Compression

Object:

State + Behavior

Instance:

Chapter 4

final — Preventing Change

4.1 The Eighteenth Wrong Idea

Common Trap

Many beginners think:

final = constant

Not quite.
final means:

Cannot Change This Particular Thing

The important question becomes:

What thing?

4.2 Three Different Meanings

Depending on context:

Usage	Prevents
final variable	Reassignment
final method	Overriding
final class	Inheritance

Same keyword.

Different targets.

4.3 Final Variables

```
1 final int x = 10;
```

Valid.

Later:

```
1 x = 20;
```

Compiler Error.

Reason:

Reference cannot be reassigned.

4.4 Mental Model

Mental Model

final variable

=

One Assignment Only

Not:

One Value Forever

The distinction matters.

4.5 The Reference Trap

Observe:

```
1 final Student s =  
2 new Student();
```

Question:

Can we do:

```
1 s = new Student();
```

?

No.

Question:

Can we do:

```
1 s.name = "Alice";
```

?

Yes.

Many beginners find this surprising.

4.6 Why?

Because:

final

protects:

Reference

Not necessarily:

Object

4.7 Memory Picture

```
final s
  |
  |
  v
```

```
+-----+
| Student |
| name=Alice |
+-----+
```

Reference fixed.
Object mutable.
These are different concepts.

4.8 True Immutability

Suppose:

```
1 class Point
2 {
3     final int x;
4     final int y;
5 }
```

Now state cannot change after construction.
This is closer to:

Immutable Object

4.9 Blank Final Variables

Observe:

```
1 class Student
2 {
3     final int id;
4
5     Student(int id)
```

```

6      {
7          this.id = id;
8      }
9  }

```

No initialization at declaration.
Still valid.
Because assigned exactly once.

4.10 The Rule

A final variable must be assigned:
Exactly once.
Before constructor completes.

4.11 Final Parameters

```

1  void print(final int x)
2  {
3  }

```

Inside method:

```

1  x = 5;

```

Illegal.
Useful for preventing accidental modification.

4.12 Final Methods

Suppose:

```

1  class Animal
2  {
3      final void breathe()
4      {
5      }
6  }

```

Question:
Can Dog override breathe ()?
No.
Compiler rejects it.

4.13 Mental Model

Mental Model

final method
=
Behavior is locked.

Subclasses may inherit.
Cannot replace.

4.14 Why Use Final Methods?

Suppose:

```
1 class BankAccount
2 {
3     final void withdraw()
4     {
5     }
6 }
```

Designer may want:
Security logic preserved.
No subclass may change it.

4.15 Final Classes

Observe:

```
1 final class Animal
2 {
3 }
```

Now:

```
1 class Dog extends Animal
2 {
3 }
```

Illegal.
Reason:
Inheritance forbidden.

4.16 The Famous Example

```
1 String
```

is:

final

Question:
Why?
Security.
Correctness.
Immutability.
Predictability.
Java designers did not want:

```
1 class EvilString
2 extends String
```

changing fundamental behavior.

4.17 Final and Polymorphism

Inheritance enables:

Polymorphism

Final class disables inheritance.
Therefore:

final

trades flexibility
for
stability.

4.18 The Design Tradeoff

More inheritance:

- More flexibility.
- More complexity.
- More risk.
- More final:
 - Less flexibility.
 - More predictability.
 - More safety.

4.19 The Immutable Object Pattern

One of the most important patterns in Java.

```
1 final class Point
2 {
3     private final int x;
4     private final int y;
5
6     Point(int x,int y)
7     {
8         this.x = x;
9         this.y = y;
10    }
11
12    int getX()
13    {
14        return x;
15    }
16
```

```
17     int getY()  
18     {  
19         return y;  
20     }  
21 }
```

Characteristics:

- final class
- final fields
- no setters
- state fixed after construction

Object becomes immutable.

4.20 Why Immutability Matters

Immutable objects are:

Easier to reason about.

Thread-safe.

Safer for APIs.

Less bug-prone.

This idea becomes critical later.

4.21 The Constant Pattern

```
1 public static final double PI  
2 =  
3 3.1415926535;
```

Observe:

static

One copy per class.

final

Cannot change.

Combined:

Class Constant

4.22 Rosetta Rules

Rosetta Rule

final variable
prevents reassignment.

Rosetta Rule

final reference
does not imply immutable object.

Rosetta Rule

final method
prevents overriding.

Rosetta Rule

final class
prevents inheritance.

Rosetta Rule

Immutability is stronger than final.

Rosetta Rule

String is final.

Rosetta Rule

public static final
is the standard constant pattern.

4.23 Mental Compression

final variable

→

No reassignment
final method

→

No overriding
final class

→

No inheritance

Chapter 5

Strings, String Pool and Wrapper Classes

5.1 The Nineteenth Wrong Idea

Common Trap

Many beginners think:
String is a primitive type.
False.
String is a class.

5.2 The First Surprise

Observe:

```
1 String s = "Hello";
```

Looks like:

```
1 int x = 5;
```

But internally:

```
1 String
```

is an object.

5.3 What Is A String?

Conceptually:

String = Sequence of Characters

Examples:

```

1 "Hello"
2
3 "Java"
4
5 "JEE"

```

Unlike C:

```

1 char str[] = "Hello";

```

Java String is a full object.

5.4 Two Ways To Create Strings

Method 1:

```

1 String s1 = "Hello";

```

Method 2:

```

1 String s2 =
2 new String("Hello");

```

Both create strings.

Not identical internally.

5.5 The String Pool

One of Java's smartest optimizations.

Suppose:

```

1 String a = "Hello";
2 String b = "Hello";

```

Question:

How many String objects exist?

Answer:

Usually one.

Because Java stores literals in:

String Pool

5.6 Memory Picture

String Pool

```

+-----+
| "Hello" |
+-----+
  ^

```

```
    |  
a   |  
b   |
```

Both references point to same object.
Memory saved.

5.7 The Identity Experiment

```
1 String a = "Hello";  
2 String b = "Hello";  
3  
4 System.out.println(a == b);
```

Output:

```
1 true
```

Because both references point to same pooled object.

5.8 Now Using new

```
1 String a =  
2 new String("Hello");  
3  
4 String b =  
5 new String("Hello");
```

Memory:

Object A

"Hello"

Object B

"Hello"

Different objects.

5.9 Now Compare

```
1 System.out.println(a == b);
```

Output:

```
1 false
```

Because:

==

checks identity.

Not contents.

5.10 The equals () Rule

```
1 System.out.println(  
2 a.equals(b)  
3 );
```

Output:

```
1 true
```

Because contents match.

5.11 The Golden String Rule

Rosetta Rule

For Strings:

Use equals ()

not ==

for content comparison.

5.12 Why String Is Immutable

Observe:

```
1 String s = "Hello";  
2  
3 s.concat(" World");
```

Question:

Does s change?

No.

A new String is created.

5.13 Memory Picture

s

|

v

```
"Hello"

concat()

|
v

"Hello World"
```

Original remains unchanged.

5.14 Why Make String Immutable?

Several reasons.

- Security.
- Caching.
- Thread Safety.
- String Pooling.
- Predictability.

5.15 The Concatenation Trap

Observe:

```
1 String s = "";
2
3 for(int i=0;i<10000;i++)
4 {
5     s += i;
6 }
```

Very inefficient.
Why?
Each concatenation creates a new String.

5.16 StringBuilder

Java provides:

```
1 StringBuilder sb =
2 new StringBuilder();
```

Efficient mutable string construction.
Example:

```
1 sb.append("Hello");
2 sb.append("World");
```

Only final result converted:

```

1 String s =
2 sb.toString();

```

5.17 Primitive Types

Java has:

```

1 byte
2 short
3 int
4 long
5
6 float
7 double
8
9 char
10 boolean

```

These are not objects.

5.18 The Problem

Suppose:

```

1 ArrayList<int>

```

Illegal.

Why?

Generics require objects.

int

is not an object.

5.19 Wrapper Classes

Java creates object versions.

Primitive	Wrapper
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

5.20 Example

```
1 Integer x =  
2 Integer.valueOf(5);
```

Object.
Not primitive.

5.21 Autoboxing

Java automatically converts:

```
1 Integer x = 5;
```

into something conceptually like:

```
1 Integer x =  
2 Integer.valueOf(5);
```

This is:

Autoboxing

5.22 Unboxing

Observe:

```
1 Integer x = 5;  
2  
3 int y = x;
```

Automatic conversion back.
Called:

Unboxing

5.23 The Null Trap

Observe:

```
1 Integer x = null;  
2  
3 int y = x;
```

Runtime Error.
Why?
Java tries to unbox:

null

into:

int

Impossible.
Produces:

```
1 NullPointerException
```

5.24 Wrapper Interning

Observe:

```
1 Integer a = 100;  
2 Integer b = 100;
```

Often:

```
1 a == b
```

returns:

```
1 true
```

Because Java caches small integers.
Usually:

−128

to

127

5.25 The Famous Trap

```
1 Integer a = 1000;  
2 Integer b = 1000;  
3  
4 System.out.println(a == b);
```

Often:

```
1 false
```

Different objects.
Use:

1 equals()

5.26 Rosetta Rules

Rosetta Rule

String is a class.

Rosetta Rule

String literals use the String Pool.

Rosetta Rule

String is immutable.

Rosetta Rule

Use equals() for String comparison.

Rosetta Rule

StringBuilder is efficient for repeated concatenation.

Rosetta Rule

Wrapper classes are object versions of primitives.

Rosetta Rule

Autoboxing converts primitive to wrapper.

Rosetta Rule

Unboxing converts wrapper to primitive.

Rosetta Rule

ArrayList<int>
is illegal.

Rosetta Rule

ArrayList<Integer>
is valid.

5.27 Mental Compression

String

→

Immutable Object
String Pool

→

Memory Optimization
Wrapper Class

→

Object Form Of Primitive
Autoboxing

→

Primitive → Object
Unboxing

→

Object → Primitive

Chapter 6

Generics - Parameterized Types

6.1 The Twentieth Wrong Idea

Common Trap

Many beginners think:
Generics are only for collections.
False.
Generics are a type-system feature.
Collections merely use them.

6.2 The Real Problem

Suppose we build:

```
1 class IntBox
2 {
3     int value;
4
5     void set(int x)
6     {
7         value = x;
8     }
9
10    int get()
11    {
12        return value;
13    }
14 }
```

Works.

Now we need:

```
1 StringBox
```

Then:

```
1 StudentBox
```

Then:

```
1 DoubleBox
```

Problem:
Same code repeated.

6.3 The Ancient Solution

Use:

```
1 Object
```

Because every class derives from:

Object

```
1 class Box
2 {
3     Object value;
4
5     void set(Object x)
6     {
7         value = x;
8     }
9
10    Object get()
11    {
12        return value;
13    }
14 }
```

Now Box can store anything.

6.4 The New Problem

Observe:

```
1 Box b = new Box();
2
3 b.set("Hello");
4
5 String s =
6 (String)b.get();
```

Required:

Cast

Everywhere.

6.5 The Catastrophe

```
1 Box b = new Box();
2
3 b.set(5);
4
5 String s =
6 (String)b.get();
```

Compiler accepts.
Runtime explodes.

```
1 ClassCastException
```

6.6 The Goal

We want:

- No duplicated classes.
- No unsafe casts.
- Compile-time checking.
- Generics achieve exactly this.

6.7 The First Generic Class

```
1 class Box<T>
2 {
3     private T value;
4
5     void set(T x)
6     {
7         value = x;
8     }
9
10    T get()
11    {
12        return value;
13    }
14 }
```

Question:

What is T?

Not a class.

Not an object.

It is:

Type Parameter

6.8 Mental Model

Mental Model

T is a placeholder.
A hole in the type system.
The actual type will be supplied later.

6.9 Instantiation

```
1 Box<String> box =  
2 new Box<>();
```

Compiler mentally sees:

```
1 String substituted for T
```

Conceptually:

```
1 class Box  
2 {  
3     String value;  
4 }
```

Not actual implementation.
Useful mental model.

6.10 Type Safety

Observe:

```
1 Box<String> box =  
2 new Box<>();  
3  
4 box.set("Hello");
```

Valid.
Now:

```
1 box.set(5);
```

Compiler Error.
Problem detected before execution.

6.11 Why This Is Huge

Without Generics:

Error discovered at runtime.

With Generics:

Error discovered during compilation.

Much safer.

6.12 Multiple Type Parameters

Observe:

```
1 class Pair<K,V>
2 {
3     K key;
4     V value;
5 }
```

Used by:

```
1 Pair<String,Integer>
```

Examples:

$K = \textit{String}$

$V = \textit{Integer}$

6.13 HashMap Suddenly Makes Sense

Observe:

```
1 HashMap<String,Integer>
```

Meaning:

Key:

$\textit{String}$

Value:

$\textit{Integer}$

Compiler enforces correctness.

6.14 Generic Methods

Generics are not limited to classes.

```
1 class Util
2 {
3     static <T> void print(T x)
4     {
5         System.out.println(x);
6     }
7 }
```

Call:

```
1 Util.print(5);
2
3 Util.print("Hello");
```

Compiler infers T.

6.15 Naming Convention

Common names:

Name	Meaning
T	Type
E	Element
K	Key
V	Value
R	Return Type

Convention only.

Not keywords.

6.16 Bounded Type Parameters

Suppose:

```
1 class Box<T extends Number>
2 {
3 }
```

Meaning:

T must be:

Number

or subclass of Number.

Allowed:

```
1 Integer
2 Double
3 Float
4 Long
```

Not Allowed:

```
1 String
```

6.17 Why Bounds Exist

Suppose:

```
1 class Calculator<T extends Number>
2 {
3 }
```


Now compiler knows:
Every T is numeric.
Can safely use Number methods.

6.18 The Primitive Problem

Question:

Can we write:

```
1 Box<int>
    ?
    No.
    Generics work only with reference types.
    Use:
```

```
1 Box<Integer>
```

This is why wrapper classes matter.

6.19 The Erasure Mystery

Now the strange part.

Question:

Does JVM really create:

```
1 Box<Integer>
2 Box<String>
3 Box<Double>
```

as separate classes?
No.
Java uses:

Type Erasure

6.20 Mental Model Of Erasure

Compiler checks generic correctness.

After checking:

Generic type information is largely removed.

Conceptually:

```
1 Box<T>
```

becomes something close to:

```
1 Box<Object>
```

Not exactly.
Close enough initially.

6.21 Why Erasure Exists

Historical compatibility.

Old Java code:

```
1 Java 1.4
```

Needed to continue working after:

Java 5

introduced generics.

6.22 The Important Consequence

Observe:

```
1 Box<String>
```

and

```
1 Box<Integer>
```

Runtime does not fully distinguish them.

Many surprising rules come from this.

6.23 Rosetta Rules

Rosetta Rule

Generics provide compile-time type safety.

Rosetta Rule

T is a type parameter.

Rosetta Rule

Generic classes eliminate unsafe casts.

Rosetta Rule

Bounds restrict allowed types.

Rosetta Rule

Generics require reference types.

Rosetta Rule

Use wrapper classes instead of primitives.

Rosetta Rule

Java generics use type erasure.

6.24 Mental Compression

Without Generics

Object + Casting + Runtime Errors

With Generics

Type Parameters + Compile Time Checking + Safety

T

→

Placeholder For Type
Type Erasure

→

Compiler Uses Generic Information
JVM Mostly Does Not

6.25 Lower Bound Wildcards

Observe:

```
1 List<? super Dog>
```

Meaning:
A list of some unknown type
that is a superclass of Dog.
Possible actual types:

List < Dog >

List < Animal >

List < Object >

All are valid.

6.26 The Important Difference

Compare:

?extends Animal

vs

?super Dog

First says:

Some subtype of Animal.

Second says:

Some supertype of Dog.

Opposite directions.

6.27 What Can We Insert?

Suppose:

```
1 List<? super Dog> dogs;
```

Can we insert:

```
1 dogs.add(new Dog());
```

Yes.

Always safe.

Because every possible target list can store Dogs.

6.28 Can We Insert Subclasses?

Suppose:

```
1 class Puppy extends Dog
2 {
3 }
```

Can we do:

```
1 dogs.add(new Puppy());
```

Yes.

Inheritance rules still apply.

6.29 What Can We Read?

Observe:

```
1 Object x =
2 dogs.get(0);
```

Allowed.

Question:

Can we write:

```
1 Dog d =  
2 dogs.get(0);
```

No.

Compiler Error.

6.30 Why?

Actual list may be:

```
1 List<Object>
```

Then retrieved value could be any Object.

Compiler cannot guarantee:

Dog

Therefore safest type is:

Object

6.31 Mental Model

Mental Model

? super

Write Safely

Read Poorly

6.32 Comparison Table

Type	Read	Write
? extends T	Good	Bad
? super T	Bad	Good

6.33 The Deep Insight

Observe:

List <?extendsAnimal >

Compiler thinks:

“I know every element is at least an Animal.”

Hence reading is safe.

Writing is unsafe.
Now:

List <?super Dog >

Compiler thinks:
“I know every element can accept Dogs.”
Hence writing is safe.
Reading becomes uncertain.

6.34 PECS

The famous rule.

Rosetta Rule

Producer Extends
Consumer Super

Known everywhere as:

PECS

6.35 What Is A Producer?

Suppose:

```
1 List<Dog>
```

and we only read from it.
The list produces values.
Therefore:

```
1 List<? extends Animal>
```

Producer

→

Extends

6.36 Example

```
1 void printAnimals(  
2     List<? extends Animal> animals)  
3 {  
4     for (Animal a : animals)  
5     {  
6         System.out.println(a);  
7     }  
8 }
```

Accepts:

List < *Dog* >

List < *Cat* >

List < *Bird* >

All safely.

6.37 What Is A Consumer?

Suppose:

```
1 void addDogs(  
2     List<? super Dog> dogs)  
3 {  
4     dogs.add(new Dog());  
5 }
```

This list consumes Dogs.

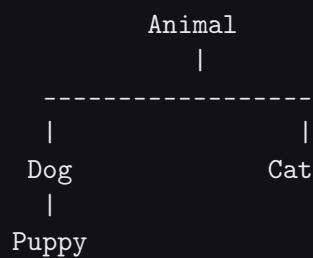
Therefore:

Consumer

→

Super

6.38 The Most Important Generics Diagram



Now:

```
1 ? extends Animal
```

Moves downward.

Accepts:

Animal

Dog

Cat

Puppy

```
1 ? super Dog
```

Moves upward.
Accepts:
Dog
Animal
Object

6.39 Unbounded Wildcards

Observe:

```
1 List<?>
```

Meaning:
List of unknown type.
Could be anything.

6.40 What Can We Read?

```
1 Object x =  
2 list.get(0);
```

Only Object guaranteed.

6.41 What Can We Insert?

Almost nothing.
Compiler cannot know actual type.

6.42 Wildcards vs Type Parameters

Observe:

```
1 <T>
```

introduces a type variable.
Now:

```
1 ?
```

means:
“Some type already exists, but I do not know which one.”
Huge conceptual difference.

6.43 Rosetta Rules

Rosetta Rule

Generics are invariant.

Rosetta Rule

List<Dog>
is not
List<Animal>

Rosetta Rule

? extends T
reads safely.

Rosetta Rule

? super T
writes safely.

Rosetta Rule

Producer Extends.

Rosetta Rule

Consumer Super.

Rosetta Rule

PECS explains most wildcard usage.

6.44 Mental Compression

?extendsT

Read
Read
Read

?superT

Write
Write
Write
PECS
Producer

→

Extends
Consumer

→

Super

Chapter 7

Collections Framework – Why Arrays Are Not Enough

7.1 The Twenty Second Wrong Idea

Common Trap

Many beginners think:
Arrays
=
Collections.
False.
Arrays are merely one data structure.
Collections are an entire framework.

7.2 The Fundamental Problem

Suppose:

```
1 int[] marks =  
2 new int[100];
```

Question:

How many students?
Exactly:

100

slots reserved.
What if only:

12

students exist?
88 slots wasted.
What if:

101

67

students exist?
Array fails.

7.3 Arrays Have Fixed Size

This is the first limitation.

Rosetta Rule

Array size is fixed after creation.

Example:

```
1 int[] arr =  
2 new int[10];
```

Later:

```
1 size = 20
```

Impossible.
Need new array.

7.4 The Second Problem

Suppose:

```
1 String[] names =  
2 new String[100];
```

Need to insert:

Alice

Bob

Charlie

Question:

Where is next free slot?

You must track it manually.

Arrays provide storage.

Not management.

7.5 The Third Problem

Suppose:

```
1 remove("Bob");
```

Array cannot automatically shift.

You write loops yourself.

Tedious.

Error-prone.

7.6 The Fourth Problem

Suppose:

```
1 find("Charlie");
```

Must manually scan.

$O(n)$

every time.

7.7 The Bigger Realization

Arrays provide:

Storage.

Nothing more.

Modern software needs:

Insertion.

Deletion.

Searching.

Sorting.

Iteration.

Mapping.

Priority management.

Uniqueness.

Arrays alone become painful.

7.8 The Collection Idea

Think:

Not

Array

But

Collection = Group of Objects

plus

Useful operations.

7.9 Historical View

Before Collections Framework:

Many custom containers.

Different APIs.

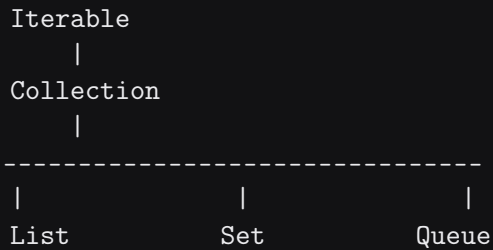
Inconsistent design.

Java introduced:

to unify everything.

7.10 The Framework View

Think:



Later:

Map

joins as a parallel hierarchy.

7.11 The Root: Iterable

Most beginners start from `ArrayList`.

Wrong starting point.

The real root is:

```
1 Iterable
```

7.12 Why Iterable Exists

Suppose:

```
1 for(String s : names)
2 {
3     System.out.println(s);
4 }
```

This enhanced for-loop works because:

Object implements:

Iterable

7.13 Mental Model

Mental Model

Iterable means:
"I know how to provide items one by one."

7.14 Iterator

Iterable provides:

```
1 iterator()
```

which returns:

Iterator

Iterator is a traversal machine.

7.15 Iterator Example

```
1 Iterator<String> it =  
2 names.iterator();  
3  
4 while(it.hasNext())  
5 {  
6     System.out.println(  
7         it.next()  
8     );  
9 }
```

This is what foreach eventually uses.

7.16 The Collection Interface

Above specific structures exists:

```
1 Collection<E>
```

Think:
Generic group of elements.
Provides operations such as:
add()
remove()
contains()
size()
isEmpty()

7.17 The Three Major Families

Family	Purpose
List	Ordered Storage
Set	Unique Elements
Queue	Processing Order

7.18 Lists

Question:

Can duplicates exist?

Yes.

Order preserved?

Yes.

Examples:

```
1 ArrayList
2 LinkedList
3 Vector
```

7.19 Example

```
1 List<String> names =
2 new ArrayList<>();
3
4 names.add("Alice");
5 names.add("Alice");
6 names.add("Bob");
```

Valid.

Duplicates allowed.

7.20 Sets

Question:

Can duplicates exist?

No.

Primary purpose:

Uniqueness.

Examples:

```
1 HashSet
2 TreeSet
3 LinkedHashSet
```

7.21 Example


```

1 Set<String> names =
2 new HashSet<>();
3
4 names.add("Alice");
5 names.add("Alice");

```

Only one Alice remains.

7.22 Queues

Model:

Line at ticket counter.

First In

First Out

FIFO

Examples:

```

1 LinkedList
2 PriorityQueue
3 ArrayDeque

```

7.23 Map Is Different

Map does not extend Collection.

This surprises many programmers.

Map stores:

Key

→

Value

pairs.

Example:

```

1 Map<String,Integer>
2 marks;

```

Student

→

Marks

7.24 Rosetta Rules

Rosetta Rule

Arrays are fixed-size containers.

Rosetta Rule

Collections provide storage plus operations.

Rosetta Rule

Iterable enables traversal.

Rosetta Rule

Iterator powers foreach loops.

Rosetta Rule

List preserves order.

Rosetta Rule

Set enforces uniqueness.

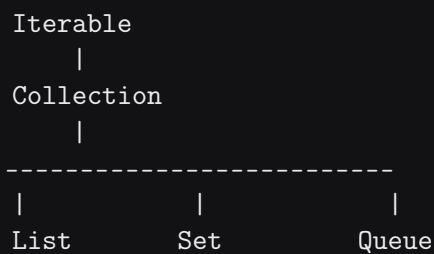
Rosetta Rule

Queue models processing order.

Rosetta Rule

Map stores key-value associations.

7.25 Mental Compression



Map (parallel hierarchy)

Array = Storage

Collection = Storage + Operations + Abstractions

Chapter 8

Identity, Equality and the Object Class

8.1 The Twenty Third Wrong Idea

Common Trap

Many beginners think:

`==`

and

`equals()`

mean the same thing.

False.

One of the most important distinctions in Java.

8.2 The Hidden Ancestor

Observe:

```
1 class Dog
2 {
3 }
```

Looks innocent.

Compiler mentally sees:

```
1 class Dog extends Object
2 {
3 }
```

Every class ultimately derives from:

Object

8.3 The Root Of The Hierarchy

```
Object
|
+----- Dog
|
+----- Cat
|
+----- Student
|
+----- String
```

Everything is an Object.

8.4 What Does Object Provide?

Important methods:

```
1 equals()
2 hashCode()
3 toString()
4 getClass()
```

Nearly every Java program uses them.
Often indirectly.

8.5 Identity

Suppose:

```
1 Student s1 =
2 new Student();
3
4 Student s2 =
5 new Student();
```

Question:

Same object?

No.

Two different objects.

Memory picture:

s1 ----> Student A

s2 ----> Student B

8.6 The Meaning Of ==

Observe:

```
1 s1 == s2
```

Question asked:

"Do these references point to the same object?"

Not:

"Do these objects contain the same data?"

8.7 Example

```
1 Student s1 =  
2 new Student();  
3  
4 Student s2 =  
5 new Student();  
6  
7 System.out.println(  
8 s1 == s2  
9 );
```

Output:

```
1 false
```

Different objects.

8.8 Now Observe

```
1 Student s1 =  
2 new Student();  
3  
4 Student s2 = s1;
```

Memory:

```
s1 ----+  
      |  
      v  
  
      Student  
  
      ^  
      |  
s2 ----+
```

One object.

Two references.

8.9 Now Compare

```
1 s1 == s2
```

Output:

```
1 true
```

Same object.

8.10 Identity vs State

Consider:

```
1 Student s1 =  
2 new Student("Alice");  
3  
4 Student s2 =  
5 new Student("Alice");
```

Question:

Same state?

Yes.

Question:

Same object?

No.

This distinction drives everything.

8.11 The equals() Idea

Humans often care about:

Contents.

Not identity.

Example:

Two books with same ISBN.

Two students with same roll number.

Two strings with same characters.

We need equality of meaning.

8.12 Default equals()

Object already provides:

```
1 equals()
```

Default implementation behaves roughly like:

```
1 return this == other;
```

Identity comparison.

8.13 Example

```
1 Student s1 =  
2 new Student("Alice");  
3  
4 Student s2 =  
5 new Student("Alice");  
6  
7 System.out.println(  
8 s1.equals(s2)  
9 );
```

Output:

```
1 false
```

Because Object's version is used.

8.14 Overriding equals()

We may define our own notion of equality.

```
1 class Student  
2 {  
3     String roll;  
4  
5     @Override  
6     public boolean equals(  
7         Object obj)  
8     {  
9         Student other =  
10            (Student)obj;  
11  
12            return roll.equals(  
13                other.roll  
14            );  
15        }  
16    }
```

Now equality means:

Same roll number.

Not same memory location.

8.15 The Fundamental Distinction

Concept	Question
Identity	Same object?
Equality	Same meaning?

8.16 The String Example

Observe:

```

1 String a =
2 new String("Java");
3
4 String b =
5 new String("Java");

```

Identity:

```

1 a == b

```

False.

Equality:

```

1 a.equals(b)

```

True.

Because String overrides equals().

8.17 The toString() Method

Observe:

```

1 Student s =
2 new Student();

```

Now:

```

1 System.out.println(s);

```

Actually calls:

```

1 s.toString()

```

automatically.

8.18 Default toString()

Produces something like:

```

1 Student@5acf9800

```

Usually not useful.

8.19 Overriding toString()

```

1 @Override
2 public String toString()
3 {
4     return roll;
5 }

```

Now:


```
1 System.out.println(s);
```

prints meaningful information.

8.20 The getClass() Method

Observe:

```
1 obj.getClass()
```

Returns runtime type information.

Useful for:

Reflection

Debugging

Frameworks

Serialization

We revisit later.

8.21 Why This Chapter Matters

Everything below depends on it:

HashSet

HashMap

TreeSet

TreeMap

Caching

ORM Frameworks

Serialization

Testing

Without understanding identity and equality, those structures appear magical.

8.22 Rosetta Rules

Rosetta Rule

Every class ultimately extends Object.

Rosetta Rule

== checks identity.

Rosetta Rule

equals() checks logical equality.

Rosetta Rule

Identity and equality are different concepts.

Rosetta Rule

String overrides equals().

Rosetta Rule

`System.out.println(obj)`
calls `toString()`.

Rosetta Rule

`Object` is the root of Java's class hierarchy.

8.23 Mental Compression

Identity

→

Same Object
Equality

→

Same Meaning

==

→

Identity

equals()

→

Logical Equality
Object

→

Ancestor Of Everything

Chapter 9

hashCode(), Hashing and the Logic of HashMap

9.1 The Twenty Fourth Wrong Idea

Common Trap

Many beginners think:
hashCode()
=
Unique Identifier
False.
Two different objects may have the same hash code.
Hash codes are not identities.

9.2 The Real Problem

Suppose:

```
1 List<Student> students;
```

Question:

Does a student exist?

Need search.

Typical approach:

```
1 for(Student s : students)
2 {
3     ...
4 }
```

Time complexity:

$$O(n)$$

Must potentially inspect every element.

9.3 The Dream

Imagine:

Instead of checking:

Student 1

Student 2

Student 3

Student 4

...

we could jump directly to the correct location.

That is the idea behind hashing.

9.4 The Library Analogy

Suppose a library contains:

1,000,000

books.

Question:

How do we find one book?

Method 1:

Look through all books.

$O(n)$

Terrible.

Method 2:

Use catalogue number.

Jump immediately.

Much faster.

9.5 Hash Function

Conceptually:

$Object \rightarrow Integer$

Example:

$Alice \rightarrow 34152$

$Bob \rightarrow 77291$

$Charlie \rightarrow 10843$

This mapping is called:

Hashing

9.6 The hashCode() Method

Every object provides:

```
1 hashCode()
```

Returns an integer.

Used heavily by:

HashMap

HashSet

Hashtable

ConcurrentHashMap

9.7 The Naive View

Imagine:

```
1 String s = "Alice";
```

Hash function computes:

"Alice" → 153248

Then:

153248

determines storage location.

9.8 Buckets

Hash structures contain buckets.

Think:

Bucket 0

Bucket 1

Bucket 2

Bucket 3

Bucket 4

...

Hash value chooses bucket.

9.9 Example

Suppose:

$$\text{hashCode()} = 17$$

Number of buckets:

$$5$$

Location:

$$17 \bmod 5 = 2$$

Store inside:

Bucket 2.

9.10 The Problem

Question:

Can two objects produce same bucket?

Of course.

9.11 Collision

Observe:

$$17 \bmod 5 = 2$$

and

$$22 \bmod 5 = 2$$

Different hashes.

Same bucket.

This is called:

Collision

9.12 Fundamental Truth

Rosetta Rule

Collisions are unavoidable.

Reason:

Infinite possible objects.

Finite integer range.

Finite bucket count.

Pigeonhole principle.

9.13 HashMap Does Not Require

Unique hash codes.

This surprises many students.

HashMap only requires:

Reasonably distributed hashes.

9.14 The equals() Contract

Now the critical rule.

Suppose:

```
1 a.equals(b)
```

returns:

true

Then:

```
1 a.hashCode()
```

and

```
1 b.hashCode()
```

must be equal.

9.15 The Most Important Hash Rule

Rosetta Rule

Equal objects must have equal hash codes.

This rule is mandatory.

9.16 The Reverse Is False

Suppose:

$$a.hashCode() = b.hashCode()$$

Question:

Must objects be equal?

No.

Collision possible.

9.17 Memory Trick

Mental Model

equals()

$\Rightarrow$

same hash
same hash

$\nRightarrow$

equals()

9.18 A Broken Class

Observe:

```
1  class Student
2  {
3      String roll;
4
5      @Override
6      public boolean equals(
7          Object obj)
8      {
9          ...
10     }
11 }
```

equals () overridden.

hashCode () forgotten.

Danger.

9.19 Why Dangerous?

HashSet relies on both.

HashMap relies on both.

If only equals () changes:
Collection behavior becomes incorrect.

9.20 Golden Rule

Rosetta Rule

Whenever equals () is overridden,
hashCode () should also be overridden.

9.21 Simple Example

```
1 @Override
2 public int hashCode()
3 {
4     return roll.hashCode();
5 }
```

Now:
equal roll numbers
produce equal hashes.
Contract preserved.

9.22 How HashMap Searches

Suppose:

```
1 map.get("Alice");
```

Step 1:
Compute hash.
Step 2:
Choose bucket.
Step 3:
Check equals () among objects in bucket.
Both methods participate.

9.23 The Deep Insight

Hashing does not replace equality. Hashing narrows search. Equality confirms answer. Think:

Hash → Candidate Bucket
Equals → Final Verification

9.24 Rosetta Rules

Rosetta Rule

hashCode () returns an integer.

Rosetta Rule

Hashing accelerates searching.

Rosetta Rule

Collisions are normal.

Rosetta Rule

Equal objects must have equal hash codes.

Rosetta Rule

Equal hash codes do not imply equal objects.

Rosetta Rule

HashMap uses both hashCode () and equals ().

Rosetta Rule

Override hashCode () when overriding equals ().

9.25 Mental Compression

Object $\rightarrow$ hashCode ()

hashCode () $\rightarrow$ Bucket

Bucket $\rightarrow$ equals ()

equals () $\rightarrow$ Final Match

Equal Objects $\rightarrow$ Equal Hashes

Equal Hashes $\nrightarrow$ Equal Objects

Part III

Java Fundamentals

Chapter 10

Class vs Object

10.1 The Third Wrong Idea

Common Trap

Most beginners think:

Class = Object

False.

They are related.

But they are not the same thing.

10.2 The Fundamental Distinction

Deep Insight

A class describes.

An object exists.

Class = Description

Object = Actual Entity

10.3 Blueprint vs House

Mental Model

Architect creates:

Blueprint

Builder creates:

House

People live in:

House

Not blueprint.

Therefore:

Class = Blueprint

Object = House

10.4 Student Example

```
1 class Student
2 {
3     String name;
4     int age;
5 }
```

Question:

How many students exist?

Answer:

Zero.

Common Trap

Many beginners think:

Class written

therefore object exists.

False.

A class merely describes.

Nothing has been created yet.

10.5 Creating Reality

```
1 Student s1;
```

Question:

Did we create a student?

Answer:

No.

We only created a reference.

More on this later.

```
1 Student s1 = new Student();
```

Now an object exists.

For the first time.

10.6 The Manufacturing Mental Model

Mental Model

Factory:
Produces cars.
Class:
Produces objects.

Class $\rightarrow$ *Object*

Just as:

Mould $\rightarrow$ *PlasticToy*

10.7 One Class Many Objects

```
1 Student a = new Student();  
2 Student b = new Student();  
3 Student c = new Student();
```

Deep Insight

One class.
Three objects.

Rosetta Rule

Classes are reused.
Objects are instantiated.

10.8 Identity Appears Here

Consider:

```
1 Student a = new Student();  
2 Student b = new Student();
```

Suppose:

```
1 a.name = "PS";  
2 b.name = "PS";
```

Same name.
Same age.
Same CGPA.
Still:

$a \neq b$

Why?
Identity.

10.9 Type vs Object

This distinction becomes important later.

`Student`
is a type.

`a`
is an object reference.

`new Student ()`
creates an object.

10.10 Reality Check

Think about:

```
1 int
```

and

```
1 int x
```

Not same.
Likewise:

```
1 Student
```

and

```
1 Student s
```

Not same.

10.11 The Hidden Power Of Classes

A class does not merely describe data.

A class describes:

- state
- behaviour
- rules

- responsibilities
- interactions

Therefore a class is really:

A specification for objects.

10.12 A Better Definition

Many books define:

Class is a blueprint.

Useful.

But incomplete.

Better:

A class is a formal specification that defines the state, behaviour, and responsibilities of a family of objects.

10.13 Why Classes Exist

Question:

Why not create objects directly?

Why introduce classes?

Deep Insight

Because software needs consistency.

Suppose university has:

50000 students.

You don't invent every student structure from scratch.

You define one specification.

Then create many instances.

10.14 Class Thinking vs Object Thinking

Beginner:

What fields should this class have?

Intermediate:

What methods should this class have?

Advanced:

What responsibilities should this object own?

10.15 Rosetta Rule

Objects are the reality.

Classes are the description.

Never confuse the two.

10.16 Rosetta Rule

One class can create many objects.

One object belongs to exactly one class type.

(ignoring advanced inheritance discussions for now)

10.17 Rosetta Rule

A class is not memory.

A class is not an object.

A class is a specification.

10.18 Rosetta Summary

Class $\neq$ Object

Class = Specification

Object = ActualEntity

Class $\rightarrow$ Many Objects

Objects contain:

Identity + State + Behaviour + Responsibility

Classes describe those things.

Chapter 11

Object Creation, References, Stack and Heap

11.1 The Most Important Line In Beginner OOP

Consider:

```
1 Student s = new Student();
```

Most beginners see:

Create Student

and move on.

Experts immediately ask:

- What memory was allocated?
- Where was it allocated?
- What is stored in s?
- What exactly did new do?
- What exists after execution?

11.2 The Fourth Wrong Idea

Common Trap

Many beginners think:

```
1 Student s;
```

creates a Student object.

False.

No object exists.

11.3 Reality Check

When Java sees:

```
1 Student s;
```

only a variable named `s` is created.
Nothing more.
No object.
No student.
No memory for fields.
No constructor call.

11.4 Mental Model

Mental Model

Suppose you write:

```
1 House h;
```

You have created:
An address card.
Not a house.
The actual house still does not exist.

11.5 Reference

Deep Insight

A reference is a way to locate an object.
It is not the object itself.

Rosetta Rule

Reference
 $\neq$
Object

11.6 The House Analogy

Think:

Object = House

Reference = Address

You can hold:

```
1 221B Baker Street
```

without owning the actual house.
Similarly:

```
1 Student s;
```

can exist without any Student object.

11.7 What Does new Actually Do?

Consider:

```
1 Student s = new Student();
```

The keyword:

```
1 new
```

performs object creation.
Conceptually:

1. Allocate memory
2. Create object
3. Initialize fields
4. Run constructor
5. Return reference

11.8 Memory Appears

After:

```
1 Student s = new Student();
```

two things now exist.

1. Reference variable s
2. Student object

11.9 The Memory Diagram

Stack

```
+-----+
|  s  |
+-----+
|
|
```

v

Heap

```
+-----+
| Student Object |
+-----+
```

11.10 The Two Worlds

Java programs live in two major worlds.

1. Stack
2. Heap

Everything becomes easier once this is understood.

11.11 Stack World

Deep Insight

The stack stores:

- local variables
- references
- method information

Think:
Fast.
Small.
Temporary.

11.12 Heap World

Deep Insight

The heap stores:
Objects.

Think:
Large.
Shared.
Long-lived.

11.13 Rosetta Memory Rule

Rosetta Rule

Variables usually live on stack.
Objects live on heap.

For now this rule is good enough.
Later we will refine it.

11.14 Two References One Object

Consider:

```
1 Student a = new Student();
2
3 Student b = a;
```

What exists?
Beginners often answer:
Two students.
Wrong.
Memory:

Stack

```
+-----+
|  a  |
+-----+
|
|
v
```

Heap

```
+-----+
| Student |
+-----+
```

Stack

```
+-----+
|  b  |
+-----+
|
|
|
+-----> same object
```

11.15 The Alias Concept

Deep Insight

Multiple references can point to the same object.

This is called aliasing.

11.16 Why Beginners Get Confused

Observe:

```
1 Student a = new Student();
2
3 Student b = a;
4
5 b.name = "PS";
```

Then:

```
1 System.out.println(a.name);
```

prints:

```
1 PS
```

Why?

Because there is only one object.

11.17 Object Identity Revisited

Consider:

```
1 Student a = new Student();
2
3 Student b = new Student();
```

Now we have:

Two objects.

Different identities.

Even if every field value is identical.

11.18 The Null Reference

Java provides:

```
1 Student s = null;
```

Meaning:

Points nowhere.

Stack

```
+-----+  
|  s  |  
+-----+
```

null

Heap

(empty)

11.19 Null Is Not An Object

Common Trap

Many beginners think:
null is a special object.
False.
null means:
No object reference exists.

11.20 The Famous Crash

```
1 Student s = null;  
2  
3 s.name = "PS";
```

Result:

```
1 NullPointerException
```

Reason:
You tried to access an object that does not exist.

11.21 Rosetta Rules

Rosetta Rule

Reference
is not
Object

Rosetta Rule

new creates objects.

Rosetta Rule

Objects live on heap.

Rosetta Rule

References typically live on stack.

Rosetta Rule

Multiple references may point to one object.

Rosetta Rule

null means:
No object.

11.22 The One Sentence Summary

Reference = Way To Reach Object

Object = Actual Entity

Confusing these two causes a huge fraction of beginner OOP bugs.

Chapter 12

Constructors and Object Birth

12.1 The Fifth Wrong Idea

Common Trap

Many beginners think:
Constructor
=
Special Function
This definition is technically correct.
But it misses the entire point.

12.2 The Better Question

Do not ask:

What is a constructor?

Ask:

Why does an object need a constructor?

12.3 Object Birth

Deep Insight

Every object enters existence at some moment.
That moment is called construction.

Mental Model

A newborn child does not appear half-created.
Certain things must happen immediately.
Name may be assigned.
Identity exists.
Basic properties exist.
The object must begin life in a valid state.

12.4 The Real Purpose

The purpose of a constructor is:

Guarantee valid birth.

Not merely initialize variables.

12.5 Example

```
1 class Student
2 {
3     String name;
4     int age;
5 }
```

Now:

```
1 Student s = new Student();
```

Java creates the object.

But what should:

```
1 name
2 age
```

contain?

Someone must decide.

12.6 Default Constructor

If you write:

```
1 class Student
2 {
3     String name;
4     int age;
5 }
```

Java secretly creates:

```
1 Student()
2 {
3 }
```

This is called the default constructor.

12.7 Important Distinction

Common Trap

Many books mix up:
Default Constructor
and
No-Argument Constructor
They are not exactly the same.

Default constructor:
Generated by compiler.
No-argument constructor:
Written by programmer.

12.8 Default Values

Consider:

```
1 class Student
2 {
3     String name;
4     int age;
5 }
```

Then:

```
1 Student s = new Student();
```

Fields receive defaults.

Type	Default Value
byte	0
short	0
int	0
long	0L
float	0.0f
double	0.0
char	'\u0000'
boolean	false
reference	null

12.9 Why Default Values Exist

Deep Insight

Java prefers safety.
Uninitialized memory is dangerous.

C allows:

```
1 int x;  
2 printf("%d",x);
```

which may produce garbage.
Java prevents this.

12.10 Custom Constructor

```
1 class Student  
2 {  
3     String name;  
4  
5     Student(String n)  
6     {  
7         name = n;  
8     }  
9 }
```

Now:

```
1 Student s =  
2 new Student("PS");
```

12.11 Mental Model

Mental Model

Think:
Factory receives specifications.
Produces object.
Constructor defines:
What information must be supplied at creation time.

12.12 The Incomplete Object Problem

Bad design:

```
1 Student s =  
2 new Student();  
3  
4 s.name = "PS";  
5  
6 s.age = 18;
```

Notice:
Object exists.
Yet important information is missing.
Between creation and assignment:

Object is incomplete.

12.13 A Better Design

```
1 Student s =  
2 new Student("PS",18);
```

Now object begins life correctly.

12.14 Rosetta Rule

Good constructors create valid objects.

Not partially valid objects.

12.15 Constructor Name Rule

```
1 class Student  
2 {  
3     Student()  
4     {  
5     }  
6 }
```

Constructor name must match class name.
Exactly.

12.16 No Return Type

```
1 Student()  
2 {  
3 }
```

Correct.

```
1 void Student()  
2 {  
3 }
```

Wrong.

Common Trap

If a return type appears,
it is no longer a constructor.
It becomes an ordinary method.

12.17 Constructor Overloading

Multiple constructors may exist.

```

1 Student()
2 {
3 }
4
5 Student(String name)
6 {
7 }
8
9 Student(String name,int age)
10 {
11 }

```

Java selects the appropriate version.

12.18 The this Keyword Appears

Consider:

```

1 class Student
2 {
3     String name;
4
5     Student(String name)
6     {
7         name = name;
8     }
9 }

```

Looks correct.

Actually wrong.

12.19 The Shadowing Problem

Inside constructor:

```

1 name

```

refers to parameter.

Not field.

Therefore:

```

1 name = name;

```

changes nothing.

12.20 The Solution

```

1 class Student
2 {
3     String name;
4
5     Student(String name)

```



```

6      {
7          this.name = name;
8      }
9  }

```

12.21 Mental Model Of this

Mental Model

Imagine every object secretly says:
 “I”
 this means:
 “the current object”

12.22 Object Construction Timeline

When:

```

1  new Student("PS");

```

executes:

1. Memory allocated
2. Default values assigned
3. Constructor executes
4. Reference returned

12.23 Construction Diagram

new Student("PS")

|
v

Allocate Memory

|
v

Default Values

|
v

Run Constructor

|
v

12.24 Rosetta Rules

Rosetta Rule

Constructor = Object Birth

Rosetta Rule

Good objects begin life valid.

Rosetta Rule

Constructors establish invariants.

Rosetta Rule

Default constructor and no-argument constructor are not identical concepts.

Rosetta Rule

this = Current Object

12.25 Rosetta Summary

Object $\rightarrow$ *Birth* $\rightarrow$ *Initialization* $\rightarrow$ *ValidState*

Good constructors guarantee that path.

Bad constructors create objects that can exist in broken or incomplete states.

Part IV

Core OOP

Chapter 13

Encapsulation — Ownership, Protection and Control

13.1 The Sixth Wrong Idea

Common Trap

Most beginners think:
Encapsulation
=
Making variables private.
This is not wrong.
But it misses almost everything important.

13.2 The Real Question

Suppose we have:

```
1 class BankAccount
2 {
3     int balance;
4 }
```

Question:

Who may modify balance?
Everyone?
Some people?
Only the object itself?
This is the problem encapsulation solves.

13.3 The Ownership Principle

Deep Insight

Every piece of data should have an owner.

Without ownership:

- anyone can change data

- bugs become difficult to locate
- invariants break
- responsibilities become unclear

13.4 Mental Model

Mental Model

Imagine a kingdom.
The treasury contains gold.
Can random citizens walk in and change numbers?
No.
Only authorized officials.
Why?
Because ownership exists.

13.5 The Public Variable Disaster

```
1 class BankAccount
2 {
3     public int balance;
4 }
```

Now:

```
1 account.balance = -100000;
```

Compiler says:

Okay.

Business logic says:

Disaster.

13.6 The Hidden Problem

The problem is not:

```
1 -100000
```

The problem is:

Anyone can modify state.

13.7 The Encapsulation Solution

```
1 class BankAccount
2 {
3     private int balance;
4 }
```

Now:

```
1 account.balance = 100;
```

becomes illegal.

13.8 Why This Matters

Deep Insight

The object now controls its own state.

This is the key idea.

Not privacy.

Control.

13.9 Object Sovereignty

Mental Model

A country controls its borders.

Not because walls are important.

Because sovereignty is important.

Likewise:

```
1 private
```

is not about hiding.

It is about maintaining authority.

13.10 The Real Definition

Deep Insight

Encapsulation is the bundling of:

- state
- behaviour
- protection rules

inside one controlled unit.

13.11 Behaviour Becomes Gatekeeper

Instead of:

```
1 account.balance += 500;
```

we write:

```
1 account.deposit(500);
```

Now:

State changes only through behaviour.

13.12 A Deep Shift

Procedural thinking:

Manipulate data.

Object thinking:

Ask the object to do something.

Bad:

```
1 account.balance += 500;
```

Better:

```
1 account.deposit(500);
```

13.13 Information Hiding

Deep Insight

Users should know:
What the object does.
Not how it does it.

Example:

```
1 account.deposit(500);
```

User need not know:

- database updates
- logging
- auditing
- transaction handling

13.14 Black Box Thinking

Mental Model

ATM Machine.
You insert card.
You press buttons.
You receive money.
You do not know the internal circuitry.

That is abstraction.
Encapsulation makes abstraction possible.

13.15 The Four Access Modifiers

Modifier	Visibility
private	Same Class
default	Same Package
protected	Package + Subclasses
public	Everywhere

For now:
Learn only:

```
1 private
2 public
```

The others become meaningful after inheritance.

13.16 Getter and Setter Trap

Many books teach:

```
1 getBalance()
2
3 setBalance()
```

and call it encapsulation.
Dangerous oversimplification.
Consider:

```
1 public void setBalance(int b)
2 {
3     balance = b;
4 }
```

This allows:

```
1 setBalance(-10000);
```

The invariant is still broken.

Rosetta Rule

Private fields alone do not create good design.
Good behaviour creates good design.

13.17 Protecting Invariants

Suppose:

$$balance \geq 0$$

must always remain true.
Then:

```
1 void withdraw(int amount)
2 {
3     if(balance >= amount)
4     {
5         balance -= amount;
6     }
7 }
```

Now the object protects itself.

13.18 Expert View

Beginners ask:

What variables should be private?

Experts ask:

What invariants must survive?

Huge difference.

13.19 Encapsulation And Complexity

Imagine:

100000 lines of code.

If every variable is public:

Every function becomes a suspect.

If state is protected:

Only a few methods can modify it.

Debugging becomes dramatically easier.

13.20 Rosetta Rules

Rosetta Rule

Encapsulation is ownership.

Rosetta Rule

Encapsulation is control.

Rosetta Rule

Encapsulation is not merely privacy.

Rosetta Rule

State should usually be private.

Rosetta Rule

Behaviour should protect invariants.

Rosetta Rule

Good objects defend themselves.

13.21 Mental Compression

Procedural World:

$$Data \rightarrow Manipulate$$

Object World:

$$Object \rightarrow Request \rightarrow Object \text{ Decides}$$

This is one of the deepest shifts in OOP.

Chapter 14

Abstraction – Seeing What Matters

14.1 The Seventh Wrong Idea

Common Trap

Many beginners believe:
Encapsulation
=
Abstraction
False.
They often appear together.
But they solve different problems.

14.2 A Quick Comparison

Encapsulation	Abstraction
Protection	Simplification
Who may access?	What should be visible?
Control	Focus

14.3 The Core Problem

Reality is complicated.
Software is complicated.
Brains are limited.
Therefore:

We cannot think about everything at once.

Abstraction exists because human thinking has limits.

14.4 The Map Analogy

Mental Model

Imagine a map of India.
Should the map contain:

- every grain of sand?
- every ant?
- every leaf?
- every stone?

No.
Such a map would be useless.

Useful maps deliberately remove information.

Rosetta Rule

Good abstractions hide details.
Not because details are unimportant.
Because details are irrelevant right now.

14.5 The Car Example

Most books use:

Car.

But now let's examine it properly.

When driving:

You care about:

- steering wheel
- brake
- accelerator

You do not care about:

- fuel injector timing
- piston movement
- combustion chamber geometry
- transmission algorithms

Deep Insight

Abstraction means:
Show what matters.
Hide what does not.

14.6 The ATM Example

Suppose:

```
1 Withdraw Money
```

is visible.

User presses button.

Money arrives.

What user does not see:

- authentication logic
- network communication
- database transactions
- logging
- encryption
- audit systems

Yet the machine works.

Why?

Abstraction.

14.7 Programming Example

Suppose Java provides:

```
1 System.out.println("Hello");
```

Looks simple.

Reality:

- JVM internals
- operating system calls
- memory management
- device drivers
- terminal communication

are involved.

Yet we think:

```
1 Print Text
```

because abstraction exists.

14.8 Abstraction Is Compression

Deep Insight

Abstraction compresses complexity.

Example:

Without abstraction:

10000 *details*

After abstraction:

1 *concept*

14.9 The Engineer's Superpower

A good engineer can operate at multiple levels.

Level 1:

1 **Send Message**

Level 2:

1 **TCP Protocol**

Level 3:

1 **Packets**

Level 4:

1 **Bits**

Abstraction allows movement between levels.

14.10 The Hidden Question

Whenever you design software ask:

What information is essential?

Everything else may become implementation detail.

14.11 Essential vs Accidental Complexity

One of the most important ideas in software engineering.

Essential Complexity:

Cannot be removed.

Example:

Bank must handle accounts.
Accidental Complexity:
Created by implementation.
Example:
Database details.
Logging details.
Network details.
Good abstractions remove accidental complexity.

14.12 Object Abstraction

Consider:

```
1 account.deposit(500);
```

User knows:
Money deposited.
User does not know:

- SQL queries
- transaction logs
- database updates
- synchronization logic

This is abstraction.

14.13 A Dangerous Beginner Habit

Beginners often expose internal details.

Example:

```
1 public ArrayList<Course> courses;
```

Now everybody knows implementation details.
Suppose later:

```
1 ArrayList
```

becomes:

```
1 LinkedList
```

Many parts of code may break.
Good abstraction hides implementation choices.

14.14 The Interface Before Interfaces

Even before learning Java interfaces:
understand the idea.
Every object has:

Public Face

and

Private Machinery

Users should interact with:

Public Face

not

Private Machinery

14.15 The Restaurant Mental Model

Mental Model

Restaurant Menu

Customer sees:

- Pizza
- Burger
- Pasta

Customer does not see:

- supplier contracts
- inventory systems
- kitchen workflows
- accounting software

Menu

=

Abstraction

Kitchen

=

Implementation

14.16 Why Abstraction Matters

Without abstraction:

Every programmer must understand everything.

Impossible.

With abstraction:

Programmers understand only the layer they need.

This is how million-line codebases become possible.

14.17 Rosetta Rules

Rosetta Rule

Abstraction is simplification.

Rosetta Rule

Abstraction hides irrelevant details.

Rosetta Rule

Abstraction compresses complexity.

Rosetta Rule

Abstraction exposes essentials.

Rosetta Rule

Good abstractions survive implementation changes.

14.18 Mental Compression

Encapsulation:

Protect Data

Abstraction:

Hide Complexity

Together:

Reliable + Understandable

software becomes possible.

14.19 Rosetta Summary

Abstraction answers:

What should people see?

Encapsulation answers:

Who may touch it?

Both are necessary.
Neither replaces the other.

Chapter 15

Generalization, Specialization and IS-A Thinking

15.1 The Eighth Wrong Idea

Common Trap

Most beginners think:
Inheritance exists to reuse code.
Partially true.
But dangerous.
Many terrible designs arise from this belief.

15.2 The Real Question

Suppose we have:

```
1 Dog
2 Cat
3 Cow
4 Horse
```

Question: Should these be completely unrelated? Or is there a common concept?

15.3 Pattern Recognition

Humans naturally classify.

Observe:

```
1 Dog
2 Cat
3 Cow
4 Horse
```

Immediately brain notices:

Animal

Observe:

- 1 Circle
- 2 Triangle
- 3 Rectangle

Brain notices:

Shape

Observe:

- 1 Car
- 2 Bike
- 3 Truck

Brain notices:

Vehicle

Deep Insight

Finding common concepts is called generalization.

15.4 Generalization

Deep Insight

Generalization means
-Move upward.
-Find common truth.

Example:

Dog

Cat

Horse

become:

Animal

Mental Model

Imagine zooming out on a map.
Fine details disappear.
Common structure remains.

15.5 Specialization

The opposite process.

Deep Insight

Specialization means

- Move downward.
- Add details.

Example:

Animal

becomes:

Dog

Cat

Horse

15.6 The Taxonomy Tree

```
Animal
|
+----- Dog
|
+----- Cat
|
+----- Horse
```

Rosetta Rule

Generalization moves upward.
Specialization moves downward.

15.7 The IS-A Test

One of the most useful OOP tests.

Question: Dog IS-A Animal?

Yes.

Question: Car IS-A Vehicle?

Yes.

Question: Student IS-A Person?

Usually yes.

Question: Engine IS-A Car?

No.

Question: Wheel IS-A Car?

No.

Rosetta Rule

If the sentence sounds wrong in English,
inheritance is usually wrong in code.

15.8 The First Big Trap

Suppose:

```
1 Dog has a Tail
```

Many beginners create:

```
1 class Dog extends Tail
```

Disaster.

Reason: Dog

IS NOT A

Tail

The correct relationship is:

Dog

HAS A

Tail

This distinction will later become:

Inheritance vs Composition

15.9 The Geometry Example

Suppose:

```
1 Circle
2 Rectangle
3 Triangle
```

All have:

- area

- perimeter

Instead of repeating:

```
1 area()
2 perimeter()
```

we notice:

Shape

This is not code reuse.

This is concept discovery.

15.10 Expert Thinking

Beginners see:

Dog

Experts ask:

What larger concept exists?

Beginners see:

Circle

Experts ask:

What family does it belong to?

15.11 A Deep Observation

Generalization compresses knowledge.

Without generalization:

```
1 Dog eats
2 Cat eats
3 Horse eats
4 Cow eats
```

With generalization:

Animal Eats

One statement replaces many.

15.12 Abstraction Connection

Notice:

Generalization is actually a form of abstraction.

We hide differences.

We keep similarities.

```
Dog      \
Cat      \
Horse    > Animal
Cow      /
Goat     /
```

Many things become one idea.

15.13 The Wrong Reason To Inherit

Bad reason:

The code looks similar.

Good reason:

The concepts have an IS-A relationship.

15.14 Example Of Disaster

Suppose:

```
1 Stack
2 Queue
```

share some code.

A beginner may force inheritance.

Question:

Queue IS-A Stack?

No.

Stack IS-A Queue?

No.

Similarity alone is not enough.

15.15 Classification Before Coding

Before writing:

```
1 extends
```

always ask:

1. What family does this belong to?
2. Is the relationship truly IS-A?
3. Will this remain true in the future?

15.16 Rosetta Rules

Rosetta Rule

Generalization finds common truth.

Rosetta Rule

Specialization adds detail.

Rosetta Rule

Inheritance models IS-A.

Rosetta Rule

Composition models HAS-A.

Rosetta Rule

Code similarity alone is not a reason to inherit.

15.17 Mental Compression

Dog $\rightarrow$ *Animal*

Circle $\rightarrow$ *Shape*

Car $\rightarrow$ *Vehicle*

Finding these relationships is the real skill.
Inheritance merely implements them.

15.18 Rosetta Summary

Generalization:

Many $\rightarrow$ *One*

Specialization:

One $\rightarrow$ *Many*

Inheritance should model concepts.
Not convenience.
Not laziness.
Not code reuse alone.

Chapter 16

Types, Contracts and Classification

16.1 The Ninth Wrong Idea

Common Trap

Many beginners think:

Type = **Data Type**

Examples:

```
1 int
2 double
3 char
```

This is far too narrow.

16.2 The Bigger Meaning Of Type

Deep Insight

A type answers:

→ *What kind of thing is this?*

and

→ *What operations are valid on it?*

Examples:

```
1 int
2 String
3 Student
4 Animal
5 Shape
```

are all types.

16.3 The Compiler's View

Suppose:

```
1 int x = 5;
```

Compiler knows:

- x stores integers
- addition is legal
- multiplication is legal
- string operations are illegal

Type tells compiler: **What is allowed**

16.4 Type As Capability

Mental Model

Imagine obtaining a driver's license.
The license tells society:

→ This person may perform certain operations

Similarly:

A type grants capabilities.

16.5 Student Type

```
1 class Student {  
2  
3     void study() {  
4  
5     }  
6  
7     void registerCourse() {  
8  
9     }  
10 }
```

The type:

```
1 Student
```

implies:

```
1 study()  
2 registerCourse()
```

are available.

16.6 Type Is A Promise

Deep Insight

A type is a contract.

When code sees:

```
1 Student
```

it expects certain *behaviour*.

A type promises: **These operations exist**

16.7 The Shape Example

Suppose:

```
1 Circle
2 Rectangle
3 Triangle
```

All support:

```
1 area()
```

Then: **Shape** becomes a useful type.

Why? Because clients only need: *area()*

16.8 The Hidden Power

Suppose:

```
1 Shape s
```

Question:

$\rightarrow$ *Circle?*

$\rightarrow$ *Rectangle?*

$\rightarrow$ *Triangle?*

– Maybe.

Compiler only cares:

Can it behave as a Shape?

This idea eventually becomes: **Polymorphism**

16.9 Classification Revisited

Earlier:

⇒ Dog

⇒ Cat

⇒ Horse

became: Animal

Now observe:

Animal is not merely a category.

Animal is also a type.

Meaning:

Code can reason about Animals.

16.10 Reality vs Type

Important distinction

Reality: Dog Type View: Animal

- A veterinarian may think: Animal

- A breeder may think: Dog

Both are correct

Different levels of abstraction.

16.11 The Substitutability Question

Suppose a function needs:

```
1 Animal
```

Can we provide:

```
1 Dog ??
```

Usually yes.

Can we provide:

```
1 Cat ??
```

Usually yes.

Why?

Because they satisfy the Animal contract.

This observation eventually leads to: Inheritance and Polymorphism

16.12 The Type Ladder

```
Animal
|
+--- Dog
|
+--- Cat
|
+--- Horse
```

Higher:
–More general.
Lower:
–More specific.

16.13 The Rosetta View

- **Beginners** think:

Type = Storage

- **Experts** think:

Type = Contract

Storage is merely one consequence.

16.14 A Deep Mental Model

Mental Model

Imagine hiring engineers.

You do not care:

Who specifically?

You care:

Can they perform engineering work?

The role matters more than identity.

Type behaves similarly.

16.15 Rosetta Rules

Rosetta Rule

A type classifies.

Rosetta Rule

A type defines allowed operations.

Rosetta Rule

A type is a contract.

Rosetta Rule

Types enable abstraction.

Rosetta Rule

Types enable polymorphism.

16.16 Mental Compression

Type answers:

What kind of thing is this?

and

What may I do with it?

Contracts create trust.

Types express contracts.

OOP is built upon that trust.

Chapter 17

Inheritance — Reusing Concepts, Behaviour and Types

17.1 The Tenth Wrong Idea

Common Trap

Most beginners think:
Inheritance = Copy Paste Without Copy Paste.
This is dangerous.
Inheritance is much more than code reuse.

17.2 The Real Question

Suppose:

```
1 Dog
2 Cat
3 Horse
```

all have:

```
1 age
2 eat()
3 sleep()
```

Question:

- Should we rewrite these repeatedly?
 - Or Should we recognize a deeper truth?
 - Dog **IS-A** Animal
 - Cat **IS-A** Animal
 - Horse **IS-A** Animal
- Inheritance** is Java's mechanism for expressing this.

17.3 The First Example

```
1 class Animal{
2
3     int age;
4
5     void eat(){
6
7         System.out.println("Eating");
8
9     }
10
11    void sleep(){
12
13        System.out.println("Sleeping");
14
15    }
16 }
```

Now:

```
1 class Dog extends Animal{
2
3 }
```

Question:

Can Dog access ?:

```
1 age
2 eat()
3 sleep()
```

— Yes.

17.4 Mental Model

Mental Model

Inheritance is **not** cloning.

It is specialization.

Think: Animal already knows some truths.

Dog **inherits** those truths and adds **new** ones.

17.5 General To Specific

```
Animal
|
|
+----- Dog
|
+----- Cat
|
+----- Horse
```

Upper levels:

— General.

Lower levels:

— Specific.

17.6 Parent and Child

Java terminology:

Old Name	Modern Name
Parent	Superclass
Child	Subclass
Base Class	Superclass
Derived Class	Subclass

All mean roughly the same thing.

17.7 What Actually Gets Inherited?

Many beginners think: **Everything**. **False**.

Conceptually inherited:

- fields
- methods
- contracts
- type relationships

Not inherited:

- constructors
- private members

17.8 A Huge Insight

Most people focus on: **Methods**

Experts focus on: **Types**

Because:

```
1 Dog
```

becomes an

```
1 Animal
```

type.

That is more important than inherited code.

17.9 Adding Special Behaviour

```
1 class Dog extends Animal {  
2  
3     void bark() {  
4  
5         System.out.println("Woof");  
6  
7     }  
8 }
```

Now Dog has:

- age
- eat ()
- sleep ()
- bark ()

Inheritance extends capabilities.

17.10 The Type Relationship

Observe:

```
1 Dog d = new Dog();
```

Clearly valid.

Now:

```
1 Animal a = new Dog();
```

Also valid.

Many beginners find this strange.

It is actually the most important consequence of inheritance.

Because:

Dog IS-A Animal

17.11 The Box Mental Model

Mental Model

Animal Box
accepts:

- Animal
- Dog
- Cat
- Horse

because all satisfy the Animal contract.

17.12 The Reverse Is Not True

```
1 Dog d = new Animal();
```

Illegal.

Reason:

Not every Animal is a Dog.

17.13 The Substitution Principle Begins

If:

Dog IS – A Animal

Then:

Anywhere expecting Animal
should accept Dog.

This eventually becomes:

Liskov Substitution Principle

One of the most important ideas in OOP.

17.14 The Constructor Problem

Consider:

```
1 class Animal {  
2  
3     Animal() {  
4  
5         System.out.println("Animal");  
6     }
```

```
7     }
8 }
```

```
1 class Dog extends Animal {
2
3     Dog() {
4
5         System.out.println("Dog");
6
7     }
8 }
```

Now:

```
1 new Dog();
```

prints:

```
1 Animal
2 Dog
```

Why?

Because parent construction happens first.

The foundation must exist before specialization.

17.15 The super Keyword

Sometimes child wants to explicitly invoke parent constructor.

```
1 class Dog extends Animal {
2
3     Dog() {
4
5         super();
6
7     }
8 }
```

Think:

super = Parent Object View

17.16 The First Design Smell

Common Trap

If you frequently think:

"I only inherited because code looked similar."

Be careful.

Similarity is weaker than: **IS — A**

17.17 The Ostrich Problem

Suppose:

Bird has:

```
1 fly()
```

Then:

Ostrich **extends**: Bird

Problem:

Ostrich **cannot** fly.

This is an example of poor inheritance design.

We will revisit it later.

17.18 Rosetta Rules

Rosetta Rule

Inheritance models IS-A.

Rosetta Rule

Inheritance is not merely code reuse.

Rosetta Rule

Subclasses specialize.

Rosetta Rule

Superclass generalizes.

Rosetta Rule

Parent constructors execute *before* child constructors.

Rosetta Rule

Type reuse is more important than code reuse.

17.19 Mental Compression

Inheritance gives:

- Code Reuse
- Behaviour Reuse
- Type Reuse

The third is usually the most important.

Chapter 18

Method Overriding and Dynamic Dispatch

18.1 The Eleventh Wrong Idea

Common Trap

Many beginners think:
Inheritance exists so child classes can use parent methods.
This is only half the story.
The real power begins when child classes *replace* parent behaviour.

18.2 The Problem

Suppose:

```
1 class Animal {  
2  
3     void speak() {  
4  
5         System.out.println("Animal Sound");  
6  
7     }  
8 }
```

Now:

```
1 class Dog extends Animal {  
2  
3 }
```

Question:

What sound should a Dog make?

Certainly not: **Animal Sound**

The child must *specialize* behaviour.

18.3 Method Overriding

```
1 class Dog extends Animal {  
2  
3     @Override  
4     void speak() {  
5  
6         System.out.println("Woof");  
7  
8     }  
9 }
```

Deep Insight

Overriding means:

A subclass provides its own implementation of an inherited method.

18.4 Mental Model

Mental Model

Animal says:

“I know how to speak.”

Dog replies:

“I know a better version.”

18.5 Before Overriding

```
Animal  
|  
+---- speak()  
  
Dog  
|  
+---- inherits speak()
```

18.6 After Overriding

```
Animal  
|  
+---- speak()  
  
Dog  
|  
+---- speak() [new version]
```

Same method name.

Different implementation.

18.7 The @Override Annotation

```
1 @Override
2 void speak() {
3
4 }
```

— *Not required.*
— **Strongly recommended.**
Why?

Compiler checks:

- Are you actually **overriding**?

Without it:

- **Typos** become dangerous.

Example:

```
1 void speak() {
2
3 }
```

Not overriding.
Just creating a new method.

18.8 The Strange Line

Consider:

```
1 Animal a = new Dog();
```

This line is one of the most important lines in all of OOP.

18.9 Two Different Questions

Observe:

```
1 Animal a = new Dog();
```

Question 1: Reference Type?
Answer:

```
1 Animal
```

Question 2: Actual Object Type?
Answer:

```
1 Dog
```

Rosetta Rule

Reference Type
and
Object Type
are not always identical.

18.10 The Critical Experiment

```
1 Animal a = new Dog();  
2  
3 a.speak();
```

What prints?

Possibility A:

```
1 Animal Sound
```

Possibility B:

```
1 Woof
```

Correct answer:

```
1 Woof
```

18.11 Why?

Because Java chooses method **implementation** based on: Actual **object** type **not** **Reference** type
This process is called:

Dynamic Dispatch

18.12 Dynamic Dispatch

Deep Insight

Method selection happens at runtime.

Compiler sees:

```
1 Animal
```

Runtime sees:

```
1 Dog
```

Runtime wins.

18.13 Mental Model

Mental Model

Reference:

“This is an Animal.”

Object:

“Actually I am a Dog.”

When `speak()` is called, Java asks the object itself.

18.14 The Dispatch Diagram

```
Animal a = new Dog();
```

|
v

Reference Type

Animal

|
v

Actual Object

Dog

|
v

`Dog.speak()`

18.15 Another Example

```
1 class Animal {  
2  
3     void speak() {  
4  
5         System.out.println("Animal");  
6     }  
}
```

```
7     }  
8 }
```

```
1 class Cat extends Animal {  
2  
3     @Override  
4     void speak() {  
5  
6         System.out.println("Meow");  
7  
8     }  
9 }
```

```
1 Animal a = new Cat();  
2  
3 a.speak();
```

Output:

```
1 Meow
```

Again:
Runtime type decides.

18.16 The Hidden Goal

Why does Java do this?
Because it enables:

One Interface

Many Behaviours

Without this feature:
Polymorphism would not exist.

18.17 The Animal Zoo

Suppose:

```
1 Animal[] zoo = {  
2  
3     new Dog(),  
4     new Cat(),  
5     new Dog()  
6  
7 };
```

Now:


```

1  for(Animal a : zoo) {
2
3      a.speak();
4
5  }

```

Outputs:

```

1  Woof
2  Meow
3  Woof

```

Same method call.
 Different behaviours.
 This is the beginning of polymorphism.

18.18 Method Overloading vs Overriding

Massive source of confusion.

Overloading:

- Same **method name**.
- Different **parameter list**.

```

1  void print(int x) {
2
3  }
4
5  void print(String s) {
6
7  }

```

Overriding:

- Same **signature**.
- Different **implementation**.

```

1  Animal.speak()
2
3  Dog.speak()

```

18.19 Rosetta Rules

Rosetta Rule

Overriding **replaces** inherited behaviour.

Rosetta Rule

Runtime **object type** determines which overridden method executes.

Rosetta Rule

Reference type does not determine overridden method selection.

Rosetta Rule

Use @Override whenever possible.

Rosetta Rule

Overloading and Overriding are **completely different** ideas.

18.20 Mental Compression

- Inheritance gives: **IS — A**
- Overriding gives: **Specialized Behaviour**
- Dynamic Dispatch gives: **Runtime Selection**

Together they create: **Polymorphism**
which is the next major chapter.

Chapter 19

Polymorphism — One Interface, Many Forms

19.1 The Twelfth Wrong Idea

Common Trap

Many beginners define polymorphism as:
“One thing taking many forms.”
This is memorization.
Not understanding.

19.2 The Real Problem

Suppose a zoo contains:

```
1 Dog
2 Cat
3 Horse
4 Cow
```

Question:

How many separate functions should we write?

```
1 makeDogSpeak()
2 makeCatSpeak()
3 makeHorseSpeak()
4 makeCowSpeak()
```

Or:

```
1 makeSpeak(Animal a)
```

Polymorphism allows the second.

19.3 The Core Idea

Deep Insight

Polymorphism means:
Treat many specific things as one general thing.

Dog Cat Horse
Became: Animal

19.4 The Meaning Of The Word

- Poly = *Many*
- Morph = *Form*
- Polymorphism = *Many Forms*

But the engineering meaning is:

One interface.
Many implementations.

19.5 The First Example

```
1 class Animal {  
2  
3     void speak() {  
4  
5     }  
6  
7 }
```

```
1 class Dog extends Animal {  
2  
3     @Override  
4     void speak() {  
5  
6         System.out.println("Woof");  
7  
8     }  
9  
10 }
```

```
1 class Cat extends Animal {  
2  
3     @Override  
4     void speak() {  
5  
6         System.out.println("Meow");  
7  
8     }  
9  
10 }
```

Now:

```
1 Animal a;
```

may represent:

- Dog
- Cat
- Horse
- Cow

This flexibility is [Polymorphism](#).

19.6 The Universal Remote Control

Mental Model

Imagine a TV remote.

Same buttons:

- Power
- Volume
- Channel

Different TV brands.

Button interface remains.

Implementation changes.

That is polymorphism.

19.7 The Zoo Example

```
1 Animal[] zoo =  
2 {  
3     new Dog(),  
4     new Cat(),  
5     new Horse()  
6 };
```

Now:

```
1 for(Animal a : zoo)  
2 {  
3     a.speak();  
4 }
```

No type checking.

No if statements.

No switch.

Each object knows what to do.

19.8 The Beginner Solution

Without polymorphism:

```
1  if(animal instanceof Dog) {
2
3  }
4  else if(animal instanceof Cat) {
5
6  }
7  else if(animal instanceof Horse) {
8
9  }
```

This grows forever.
Every new animal changes old code.

19.9 The Polymorphic Solution

```
1  animal.speak();
```

No conditions.
No type inspection.
Object decides.

19.10 A Deep Design Principle

- **Bad Design:** *Ask Type* then decide.
- **Good Design:** *Ask Object* to perform behaviour.

19.11 The Open/Closed Principle Appears

Suppose tomorrow:

```
1  Elephant
```

is added.
Without polymorphism: Existing code changes.
With polymorphism:

```
1  class Elephant extends Animal {
2
3      @Override
4      void speak() {
5
6          System.out.println("Trumpet");
7
8      }
9
10 }
```

Existing code remains untouched.

This is the beginning of:

Open/Closed Principle

Open for extension.

Closed for modification.

19.12 Why Large Systems Need This

Suppose: 1000 animal types exist.

Without polymorphism:

— Every addition modifies existing code.

— Eventually system becomes fragile.

— **Polymorphism** localizes change.

19.13 The Liskov Substitution Principle

One of the deepest ideas in OOP.

Informal statement:

If Dog is an Animal,

then Dog should be usable anywhere Animal is expected.

This sounds *obvious*.

Actually extremely powerful.

19.14 The Bank Account Example

Suppose:

```
1 Animal a
```

expects:

```
1 eat()
2 sleep()
3 speak()
```

Any subclass must preserve those expectations.

Otherwise substitution fails.

19.15 The Ostrich Problem Revisited

Suppose:

```
1 class Bird {
2
3     void fly() {
4
5     }
6
7 }
```

Then:

```
1 class Ostrich extends Bird {  
2  
3 }
```

Problem:

Ostrich cannot fly.

Now:

```
1 Bird b = new Ostrich();  
2  
3 b.fly();
```

The abstraction is broken.

Inheritance hierarchy was wrong.

Rosetta Rule

Good inheritance preserves substitution.

19.16 Compile-Time vs Runtime

Earlier:

```
1 Animal a
```

Compile time knows:

- Animal

Runtime knows:

- Dog Or
- Cat Or
- Horse

Dynamic dispatch bridges the gap.

19.17 Polymorphism Is Decoupling

Deep Insight

Polymorphism separates:

What is done from How it is done

Client says:

```
1 speak()
```

Object decides implementation.

19.18 Rosetta Rules

Rosetta Rule

Polymorphism means:

- One interface.
- Many implementations.

Rosetta Rule

Polymorphism removes condition-heavy code.

Rosetta Rule

Polymorphism enables extensibility.

Rosetta Rule

Polymorphism depends on:
Inheritance + Overriding + Dynamic Dispatch

Rosetta Rule

Good subclasses preserve substitutability.

19.19 Mental Compression

- Inheritance creates: **IS — A**
- Overriding creates: **Specialized Behaviour**
- Dynamic Dispatch creates: **Runtime Selection**
- Together: **Polymorphism**

This is one of the central pillars of OOP.

Chapter 20

Comparable — Teaching Objects How To Compare Themselves

20.1 The Thirtieth Wrong Idea

Common Trap

Many beginners think:

Sorting = A property of numbers.

False.

Sorting is fundamentally about *comparison*.

If two objects can be **compared**, they can usually be sorted.

20.2 The Real Question

Observe:

```
1 5
2 10
3 2
4 30
5 20
```

How do we sort?

We repeatedly ask:

$5 < 10?$

$10 < 2?$

and so on.

Sorting algorithms do not understand numbers.

They only understand comparison.

20.3 A Bigger Problem

Suppose:

```
1 class Student {  
2  
3     String name;  
4  
5     int marks;  
6  
7 }
```

Question:

How should students be sorted?

By name?

By marks?

By roll number?

Compiler cannot guess.

20.4 The Comparable Interface

Java solution:

```
1 Comparable<T>
```

Meaning:

‘This object knows how to compare itself.’

20.5 The Contract

Conceptually:

```
1 interface Comparable<T>  
2 {  
3     int compareTo(T other);  
4 }
```

20.6 The Meaning Of compareTo()

Observe:

```
1 a.compareTo(b)
```

Returns:

- Negative if $a < b$
- Zero if $a = b$
- Positive if $a > b$

20.7 Example With Integers

Observe:

```
1 Integer a = 5;
2 Integer b = 10;
3 a.compareTo(b);
```

Result: **-5** (Negative) Meaning: **5 < 10**

20.8 Student Example

Suppose:

```
1 class Student implements Comparable<Student> {
2     int marks;
3
4     @Override
5     public int compareTo(Student other){
6         return this.marks - other.marks;
7     }
8 }
```

Now students know how to compare themselves.

20.9 The Hidden Conversation

When Java sorts: Student A and Student B Java internally asks:

```
1 A.compareTo(B) again and again.
```

20.10 Sorting A List

Observe:

```
1 Collections.sort(students);
```

Question: How does Java know what ordering means?

Answer: *compareTo ()*

20.11 Natural Ordering

Comparable defines: Natural Ordering

- Integers: Ascending
- Strings: Alphabetical
- Dates: Chronological

20.12 The Mathematics Behind It

A good comparison should satisfy:

- **Reflexive:** $a = a$
- **Consistent:** If $a > b$ then $b < a$
- **Transitive** If $a > b$ and $b > c$ then $b > c$

20.13 Bad Comparison Example

```
1 public int compareTo(Student other) {  
2     return 1;  
3 }
```

Always positive.
Broken ordering.
Sorting may behave strangely.

20.14 Comparable And Trees

Soon we will study:

- TreeSet
- TreeMap

They constantly ask:

- Left?
- Right?
- Greater?
- Smaller?

Without comparison, trees cannot function.

20.15 Comparable And Equality

IMPORTANT -> Observe:

```
1 a.compareTo(b) == 0
```

Usually implies:

```
1 a.equals(b)
```

Good design keeps them consistent.

20.16 Mental Model

- Hashing asks: Same object?
- Comparison asks: Which comes first?

Different goals.

20.17 Rosetta Rules

Rosetta Rule

Comparable defines natural ordering.

Rosetta Rule

`compareTo ()` returns an `int`.

Rosetta Rule

Negative means *smaller*.

Rosetta Rule

Zero means equal.

Rosetta Rule

Positive means greater.

Rosetta Rule

Sorting relies on comparison.

Rosetta Rule

`TreeSet` relies on comparison.

Rosetta Rule

`TreeMap` relies on comparison.

20.18 Mental Compression

- `HashSet` $\rightarrow$ `hashCode ()`
- `HashMap` $\rightarrow$ `hashCode ()`
- `TreeSet` $\rightarrow$ `compareTo ()`
- `TreeMap` $\rightarrow$ `compareTo ()`
- `Comparable` $\rightarrow$ Object compares itself
- `compareTo ()` $\rightarrow$ Negative / Zero / Positive

Chapter 21

Comparator — Multiple Ways To Compare The Same Object

21.1 The Thirty First Wrong Idea

Common Trap

Many beginners think: An object has exactly one ordering. **False**. A student may be sorted by:

- Marks
- Name
- Roll Number
- age

Different situations require different orderings.

21.2 The Comparable Limitation

Recall:

```
1 class Student
2 implements Comparable<Student> {
3     public int compareTo(Student other) {
4         return marks - other.marks;
5     }
6 }
```

Question:

What if tomorrow we need:

Alphabetical ordering?

Problem.

compareTo () already defines the natural ordering.

Only one can exist.

21.3 The Real Requirement

We want:

Today:

— Sort by *marks*.

Tomorrow:

— Sort by *name*.

Next week:

— Sort by *roll number*.

Without modifying *Student*.

21.4 The Comparator Idea

Instead of asking:

Student
*How do **you** compare yourself?*



Instead, ask:

Comparator
*How should **these students** be compared?*

Comparison logic belongs outside the Student class.

21.5 The Interface

Conceptually:

```
1 interface Comparator<T> {  
2     int compare(T a,T b);  
3 }
```

Notice:

- *Comparable* —> *compareTo* ()
- *Comparator* —> *compare* ()

21.6 A Marks Comparator

```
1 class MarksComparator implements Comparator<Student> {  
2     @Override  
3     public int compare(Student a,Student b) {  
4         return a.marks - b.marks;  
5     }  
6 }
```

Now comparison logic lives outside Student.

21.7 A Name Comparator

```
1 class NameComparator implements Comparator<Student> {  
2     @Override  
3     public int compare(Student a, Student b) {  
4         return a.name.compareTo(b.name);  
5     }  
6 }
```

Same Student class. Different ordering.

21.8 The Deep Insight

Observe:

Comparable changes the object.

Comparator changes the viewpoint.

21.9 Sorting With Comparator

Observe:

```
1 Collections.sort(students, new MarksComparator());
```

Java asks **Comparator**: Which comes first? again and again.

21.10 Mental Picture

Think:

Student:

Alice 95

Bob 88

Carol 99

Comparator acts like:

A judge.

The students do not decide.

The judge decides.

21.11 Comparable vs Comparator

Feature	Comparable	Comparator
Location	Inside Class	Outside Class
Methods	compareTo ()	compare ()
Count	One Natural Order	Many Orders
Flexibility	Lower	Higher

21.12 The OOP Lesson

`Student` class belongs to a library.
You cannot modify it.

Can you still sort? $\Rightarrow$ **Yes.**

Comparator $\Rightarrow$ **Behavior attached externally**

21.13 Anonymous Comparator

Observe:

```
1 Collections.sort(students,new Comparator<Student>() {
2     @Override
3     public int compare(Student a,Student b) {
4         return a.marks - b.marks;
5     }
6 });
```

Comparator object created on the spot.

21.14 The Lambda Revolution

Modern Java:

```
1 students.sort((a,b) -> a.marks - b.marks);
```

Much shorter.

We will derive this later when studying Lambdas.

21.15 Comparator Chaining

Suppose:

- Sort by `marks`.
- If marks equal: Sort by `name`.

$\Rightarrow$ *Possible because comparison logic is external.*

Very powerful.

21.16 Where Comparator Appears

- `Collections.sort ()`
- `Arrays.sort ()`
- `TreeSet`
- `TreeMap`
- `PriorityQueue`
- `Streams`

$\Rightarrow$ **Practically everywhere.**

21.17 The Bigger Design Principle

Comparable says:

- Object owns behavior.

Comparator says:

- Behavior can be injected.

This idea later appears in:

- Strategy Pattern
- Dependency Injection
- Functional Programming
- Modern Frameworks

21.18 Rosetta Rules

Rosetta Rule

Comparable defines natural ordering.

Rosetta Rule

Comparator defines custom ordering.

Rosetta Rule

Comparable uses `compareTo ()`.

Rosetta Rule

Comparator uses `compare ()`.

Rosetta Rule

Comparable lives inside class.

Rosetta Rule

Comparator lives outside class.

Rosetta Rule

A class may have one natural order.

Rosetta Rule

A class may have many comparators.

21.19 Mental Compression

Comparable ↓ Object compares
itself

`compareTo ()` ↓ Natural Order

Comparator ↓ External
comparison logic

`compare ()` ↓ Custom Order

One Object ↓ Many Possible Comparators

Chapter 22

TreeMap — Ordered Key-Value Storage

22.1 The Thirty Third Wrong Idea

Common Trap

Many beginners think:

TreeMap = HashMap that prints sorted output.

False.

TreeMap is **not** HashMap with extra sorting. It is built on an entirely different foundation.

- * HashMap uses hashing.

- * TreeMap uses ordering.

22.2 Recall HashMap

Observe:

```
1 Map<String,Integer> marks = new HashMap<>();
```

Storage:

Alice -> 95

Bob -> 88

Charlie -> 91

Question: Order??

No guarantee.

22.3 The New Requirement

Suppose we want:

Alice -> 95 Charlie -> 91

Bob -> 88 David -> 97

always alphabetically ordered. without calling sort (). without copying data elsewhere.

22.4 The TreeMap Idea

Observe:

```
1 Map<String,Integer> marks = new TreeMap<>();
```

Now keys remain sorted.
Automatically.

22.5 The Key Insight

> HashMap stores:

Key → *Value*

> TreeMap stores:

Key → *Value*

Difference:
Keys are *ordered*.

22.6 Visualization

Insert:

```
1 put("Bob",88)
2
3 put("Alice",95)
4
5 put("David",97)
6
7 put("Charlie",91)
```

TreeMap internally thinks:

```
      Bob
     /  \
    /    \
  Alice  David
     /
   Charlie
```

Values travel with keys.

22.7 The Hidden Structure

Internally:

- Red-Black Tree same family as *TreeSet*.
- Self-balancing.
- O (log n) guaranteed.

22.8 The Fundamental Question

Question: How does TreeMap know?

- Left?
- Right?

Answer:

`compareTo ()` or `Comparator`

22.9 Example With Strings

Observe:

```
1 "Alice".compareTo("Bob")
```

Negative. (alphabetically)

Therefore:

Alice goes **left** of Bob.

22.10 Custom Objects As Keys

Suppose:

```
1 class Student {  
2     int roll;  
3 }
```

Question: Can TreeMap store the following?

```
1 TreeMap<Student,Integer>
```

Only if Student can be compared.

Meaning:

`Comparable` or `Comparator` must exist.

22.11 The Common Error

Observe:

```
1 TreeMap<Student,Integer>map = new TreeMap<>();
```

Without comparison logic.

Runtime error.

Tree cannot decide where to place nodes.

22.12 Iteration Order

Observe:

```
1 for(String key : map.keySet()) {  
2     ...  
3 }
```

Output always sorted by key order.

22.13 Important Methods

TreeMap provides:

```
1 firstKey()
2
3 lastKey()
4
5 higherKey()
6
7 lowerKey()
8
9 ceilingKey()
10
11 floorKey()
```

These are impossible in ordinary HashMap.

22.14 Meaning Of These Methods

Suppose the `TreeMap` contains the following keys.

10
20
▶ 30
40
50

Query 1

```
1 higherKey(30)
```

▶ 40

Query 2

```
1 lowerKey(30)
```

▶ 20

Query 3

```
1 ceilingKey(31)
```

▶ 40

Query 4

```
1 floorKey(31)
```

► 30

22.15 Range Queries

TreeMap can answer:

Give all keys between 100 and 500, because ordering exists.

HashMap **cannot** do this efficiently.

22.16 Complexity Table

Operation	Complexity
put ()	$O(\log n)$
get ()	$O(\log n)$
remove ()	$O(\log n)$
containsKey ()	$O(\log n)$
firstKey ()	$O(\log n)$
lastKey ()	$O(\log n)$

22.17 HashMap vs TreeMap

Property	HashMap	TreeMap
Structure	Hash Table	Red-Black Tree
Ordering	No	Yes
Lookup	$O(1)$	$O(\log n)$
Insertion	$O(1)$	$O(\log n)$
Range Queries	Poor	Excellent

22.18 The Deep Connection

Observe:

HashSet ► stores keys only.

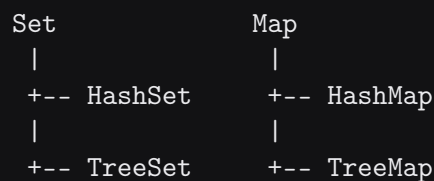
HashMap ► stores key-value pairs.

TreeSet ► stores ordered keys.

TreeMap ► stores ordered key-value pairs.

Everything is becoming symmetrical.

22.19 Collection Family Tree



22.20 The OOP View

TreeMap depends on:

- Abstraction (Map)
- Polymorphism
- Interfaces
- Comparable
- Comparator

Many previous chapters converge here.

22.21 Rosetta Rules

Rosetta Rule

TreeMap stores key-value pairs.

Rosetta Rule

Keys remain sorted.

Rosetta Rule

TreeMap uses comparison.

Rosetta Rule

TreeMap uses a Red-Black Tree.

Rosetta Rule

TreeMap operations are $O(\log n)$.

Rosetta Rule

TreeMap supports range queries.

Rosetta Rule

Keys must be comparable.

Rosetta Rule

Ordering comes from Comparable or Comparator.

22.22 Mental Compression

- HashMap ► Key + Value + **Hashing**.
- TreeMap ► Key + Value + **Ordering**.
- HashMap $O(1)$

- TreeMap **O** ($\log n$)
- HashMap **Fast**
- TreeMap **Sorted**

Chapter 23

Queue and PriorityQueue — The Collection of Importance

23.1 The Thirty Fourth Wrong Idea

Common Trap

Many beginners think:

`Queue = ArrayList` with restrictions.

False.

Queue represents a completely different philosophy.

Lists focus on *storage*.

Queues focus on *processing order*.

23.2 The Real World Inspiration

Imagine:

People waiting at a ticket counter.

The first person arrives.

The first person served.

FIFO

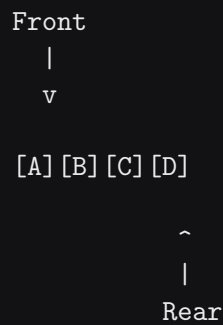
First In First Out

23.3 Queue Interface

Java provides:

```
1 Queue<E>
```

Conceptually:



Insert at rear.

Remove from front.

23.4 Basic Operations

```
1 offer();
2 poll();
3 peek();
```

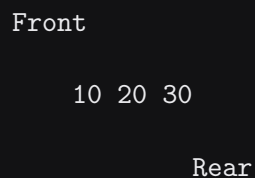
Think of them as:

```
offer()  → enqueue
poll()   → dequeue
peek()   → look at the front without removing
```

23.5 Example

```
1 Queue<Integer> q = new LinkedList<>();
2 q.offer(10);
3 q.offer(20);
4 q.offer(30);
```

Queue becomes:



23.6 Polling

Observe:

```
1 q.poll();
```

Removes:

10

Queue becomes:

```
1 20 30
```

FIFO behavior.

23.7 Peek

Observe:

```
1 q.peek();
```

Returns:

20

Does not remove it.

23.8 Why LinkedList Works Well

Queue requires:

- Insert at rear.
- Remove at front.

Observation:

Linked lists naturally support this.

23.9 Applications

Queues appear naturally in many real-world systems.

- Printer jobs
- Customer service systems
- Breadth First Search (BFS)
- Operating system scheduling
- Network packet processing

23.10 The Limitation

Suppose

Task A	Priority 1
Task B	Priority 100
Task C	Priority 5

FIFO says

- Process oldest first.

But maybe

- Highest priority should go first.

Observation

Queue cannot do this.

23.11 The New Question

- **Not:**
 - Who arrived first?
- **Instead:**
 - Who is most important?
- This creates:
 - `PriorityQueue`

23.12 PriorityQueue Creation

```
1 PriorityQueue<Integer> pq = new PriorityQueue<>();
```

Insert:

```
1 pq.offer(50);
2 pq.offer(10);
3 pq.offer(30);
4 pq.offer(20);
```

Question: What is at the front?

Answer: `10`

Smallest element.

23.13 The Surprise

Insertion Order

50
10
30
20

$\Rightarrow$

Removal Order

10
20
30
50

Key Insight

Priority determines processing.

Not arrival time.

23.14 The Hidden Structure

PriorityQueue is usually implemented using

Binary Heap

Not

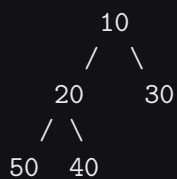
- a tree set
- a hash table

Implementation

A heap.

23.15 Heap Mental Model

Min Heap



Rule

$\text{Parent} \leq \text{Children}$

Observation

Smallest element always at root.

23.16 Why Heap Matters

Question

How quickly can we find the smallest element?

Answer

$$O(1)$$

Look at the root.

23.17 Insertion Complexity

Insertion

1. Place at the bottom.
2. Bubble upward.

Time Complexity

$$O(\log n)$$

23.18 Removal Complexity

Removal Steps

1. Remove the root.
2. Move the last element upward.
3. Heapify.

Time Complexity

$$O(\log n)$$

23.19 PriorityQueue And Comparable

Observe

```
1 PriorityQueue<Student> pq = new PriorityQueue<>();
```

Question: How should students be ordered?

Need

Comparable

or

Comparator

Exactly like `TreeSet`.

23.20 Custom Comparator

```
1 PriorityQueue<Student> pq = new PriorityQueue<>(new MarksComparator());
```

Result

Highest marks may now appear first.

23.21 PriorityQueue vs TreeSet

<code>TreeSet</code>	Entire collection remains sorted.
<code>PriorityQueue</code>	Only the highest-priority element is guaranteed at the top. The remaining elements need not appear sorted internally.

23.22 Complexity Table

Operation	Complexity
<code>peek ()</code>	$O(1)$
<code>offer ()</code>	$O(\log n)$
<code>poll ()</code>	$O(\log n)$
<code>contains ()</code>	$O(n)$

23.23 Queue Family Tree



23.24 Rosetta Rules

Rosetta Rule

Queue follows FIFO.

Rosetta Rule

offer () inserts.

Rosetta Rule

poll () removes front.

Rosetta Rule

peek () views front.

Rosetta Rule

PriorityQueue processes most important element first.

Rosetta Rule

PriorityQueue uses a heap.

Rosetta Rule

PriorityQueue relies on comparison logic.

Rosetta Rule

PriorityQueue is not fully sorted.

23.25 Mental Compression

Queue $\mapsto$ FIFO

PriorityQueue $\mapsto$ Importance First

LinkedList $\mapsto$ Queue Implementation

Heap $\mapsto$ PriorityQueue Implementation

Comparable $\mapsto$ Ordering

Comparator $\mapsto$ Custom Priority

Chapter 24

Iterator — The Universal Collection Navigator

24.1 The Thirty Fifth Wrong Idea

Common Trap

Many beginners think:
for-loop = the way to traverse collections. **False.**
for-loop only works naturally for indexed structures.
HashSet has no index.
TreeSet has no index.
Queue has no index.
Something more general is needed.

24.2 The Fundamental Problem

Observe:

```
1 ArrayList<Integer> list;
```

Traversal:

```
1 for(int i=0;i<list.size();i++) {  
2     System.out.println(list.get(i));  
3 }
```

Works.

Now:

```
1 HashSet<Integer> set;
```

Question: What is?

```
1 set.get (i)
```

Doesn't exist.

No indexing.

24.3 The OOP Requirement

Java wants:

One traversal mechanism for all collections.

That mechanism is:

Iterator

24.4 The Core Idea

Think: Iterator = Collection Cursor

A movable pointer.

24.5 Mental Picture

Suppose:



The iterator points at the (*next*) element to visit.

24.6 The Interface

Conceptually:

```
1 interface Iterator<E> {  
2     boolean hasNext();  
3  
4     E next();  
5 }
```

Two methods.

Huge power.

24.7 hasNext ()

Question: Is there another element?

Returns

```
1 true
```

or

```
1 false
```


24.8 next ()

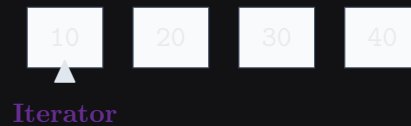
Returns:

Current element.

Then advances `cursor`.

24.9 Visualization

Initial State



Call

```
1 next()
```

Returns

10

Cursor Moves



24.10 Obtaining Iterator

Observe:

```
1 Iterator<Integer> it = list.iterator();
```

Every collection can create an appropriate iterator.

24.11 Canonical Traversal

```
1 Iterator<Integer> it = list.iterator();
2
3 while(it.hasNext()) {
4     System.out.println(it.next());
5 }
```

Works for:

- ArrayList
- LinkedList

- HashSet
- TreeSet
- PriorityQueue
- Many others.

24.12 The Deep OOP Insight

Observe:

`ArrayList` stores data differently from `HashSet`
Yet both provide:

```
1 iterator()
```

Polymorphism again.

24.13 Iterator Hides Structure

User sees:

```
1 hasNext()
2
3 next()
```

User does not see:

Arrays

Linked Lists

Trees

Hash Tables

Classic *Abstraction*.

24.14 The Hidden Relationship

Observe:

```
Iterable
|
Collection
|
ArrayList
```

Important:
Collections implement

Iterable

24.15 Iterable

Conceptually:

```
1 interface Iterable<E> {  
2     Iterator<E> iterator();  
3 }
```

Meaning:

‘I can produce an iterator.’

24.16 The Enhanced For Loop Mystery

Observe:

```
1 for(Integer x : list) {  
2     System.out.println(x);  
3 }
```

Looks magical.

Actually becomes approximately:

```
1 Iterator<Integer> it = list.iterator();  
2  
3 while(it.hasNext()) {  
4     Integer x = it.next();  
5  
6     ...  
7 }
```

The enhanced loop is *syntax sugar*.

24.17 Removing Elements

Iterator also provides:

```
1 remove()
```

for safe deletion during traversal.

24.18 The Dangerous Code

Observe:

```
1 for(Integer x : list) {  
2     list.remove(x);  
3 }
```

Often causes:

`ConcurrentModificationException`

24.19 Safe Removal

Observe:

```
1 Iterator<Integer> it = list.iterator();
2
3 while(it.hasNext()) {
4     Integer x = it.next();
5
6     if(x % 2 == 0) {
7         it.remove();
8     }
9 }
```

Safe.

Iterator understands traversal state.

24.20 Iterator And Maps

Map is special.

Usually traverse:

```
1 map.entrySet()
```

then obtain iterator.

24.21 Mental Model

Think



The Bookmark Remembers

- Current position.

24.22 Rosetta Rules

Rosetta Rule

Iterator provides universal traversal.

Rosetta Rule

hasNext () checks for remaining elements.

Rosetta Rule

next () returns current element.

Rosetta Rule

Iterator hides internal structure.

Rosetta Rule

Enhanced for-loop uses Iterator internally.

Rosetta Rule

Iterable produces Iterators.

Rosetta Rule

Iterator is an abstraction.

Rosetta Rule

Safe removal should use `iterator.remove ()`.

24.23 Mental Compression

Collection	→ Stores data
Iterator	→ Traverses data
Iterable	→ Creates an iterator
<code>hasNext ()</code>	→ Checks for another element
<code>next ()</code>	→ Returns the next element
Enhanced for	→ Iterator syntax sugar

Chapter 25

Generics — Teaching Classes About Types

25.1 The Thirty Sixth Wrong Idea

Common Trap

Many beginners think:

Generics = Angle Brackets.

False.

Generics are Java's way of writing algorithms and data structures independent of type. The angle brackets are merely the syntax.

25.2 The Historical Problem

Before Java 5:

```
1 ArrayList list = new ArrayList();
```

Everything became:

```
1 Object
```

25.3 Example

Observe:

```
1 ArrayList list = new ArrayList();
2
3 list.add("Alice");
4
5 list.add(42);
```

Perfectly legal.

Question: What does list contain?

Strings.

Integers.

Anything.

Compiler cannot help.

25.4 The Disaster

Observe:

```
1 String name = (String)list.get(1);
```

Runtime:

```
1 ClassCastException
```

Program compiles.

Fails later. *Bad design.*

25.5 The Goal

Instead of:

Checking types at runtime.

Check types during compilation.

This creates: *Generics.*

25.6 The First Generic Example

Observe:

```
1 ArrayList<String> names = new ArrayList<>();
```

Meaning: This list stores Strings.

Nothing else.

25.7 Compiler Protection

Observe:

```
1 names.add("Alice");
```

Valid.

Observe:

```
1 names.add(42);
```

Compiler Error.

Problem caught early.

25.8 The Deep Insight

Without Generics: Programmer promises type safety.

With Generics: Compiler enforces type safety.

25.9 Creating Our Own Generic Class

Observe:

```
1 class Box<T>{  
2     private T value;  
3  
4     public void set(T value){  
5         this.value = value;  
6     }  
7  
8     public T get(){  
9         return value;  
10    }  
11 }
```

Question: (? What is T) A type parameter.

25.10 Mental Model

Think:

T = Placeholder

Like:

x in mathematics.

25.11 Using Box

Observe:

```
1 Box<Integer> b = new Box<>();
```

Compiler mentally thinks:

```
1 class Box {  
2     Integer value;  
3 }
```

Conceptually.

25.12 Another Example

Observe:

```
1 Box<String> b = new Box<>();
```

Now:

```
1 String value;
```

Same class.

Different type.

25.13 Why This Is Powerful

Without Generics:

BoxForInteger

BoxForString

BoxForDouble

Many classes.

With Generics:

One class.

Many types.

25.14 Multiple Type Parameters

Observe:

```
1 class Pair<K,V> {  
2     K key;  
3     V value;  
4 }
```

Exactly the idea used by: [Map](#)

25.15 Example

```
1 Pair<String,Integer> student;
```

Meaning:

[String](#) key

[Integer](#) value

25.16 Generic Methods

Observe:

```
1 public static <T> void print(T value) {  
2     System.out.println(value);  
3 }
```

Method itself becomes generic.

25.17 Invocation

Observe:

```

1 print("Hello");
2
3 print(42);
4
5 print(3.14);

```

Same method.
Different types.

25.18 The Collection Revolution

Generics transformed collections.
Observe:

```

1 List<String>

```

Compiler knows:
Elements are Strings.
Observe:

```

1 Iterator<String>

```

Compiler knows:
next() returns String.
No cast required.

25.19 Bounded Generics

Sometimes:
Any type
is too broad.
Observe:

```

1 class Box<T extends Number>
2 {
3 }

```

Allowed:
Integer
Double
Float
Not:
String

25.20 The Meaning Of extends

Important.
Here:

```

1 T extends Number

```

means:
T must be Number or a subclass.

25.21 The Big Surprise

Generics exist only during compilation.

This idea is called:

Type Erasure

25.22 Type Erasure

Compiler checks types.

Then removes most generic information.

At runtime:

```
1 ArrayList<String>
```

and

```
1 ArrayList<Integer>
```

become roughly:

```
1 ArrayList
```

25.23 Why Type Erasure Exists

Historical reason.

Backward compatibility.

Old Java code continues working.

25.24 The Limitation

Observe:

```
1 new T();
```

Illegal.

Because runtime no longer knows what T really is.

25.25 Generics And OOP

Generics add:

Type Abstraction

OOP already provided:

Behavior Abstraction

Together they create reusable APIs.

25.26 Rosetta Rules

Rosetta Rule

Generics provide compile-time type safety.

Rosetta Rule

T is a type parameter.

Rosetta Rule

Generic classes work with many types.

Rosetta Rule

Generic methods are possible.

Rosetta Rule

Bounds use extends.

Rosetta Rule

Generics reduce casting.

Rosetta Rule

Generics improve API design.

Rosetta Rule

Java implements generics using type erasure.

25.27 Mental Compression

Object

⇒

Any Type
Generic

⇒

Specific Type
T

⇒

Type Variable
Compiler

⇒

Type Checking
Type Erasure

⇒

Runtime Simplification
Generics

⇒

Reusable + Safe

Chapter 26

Wildcards – ?, extends, super and PECS

26.1 The Thirty Seventh Wrong Idea

Common Trap

Many beginners think:
List<Integer>
is a subtype of
List<Number>
False.
This is one of the most important surprises in Java.

26.2 The Natural Expectation

Observe:

```
1 Integer
2 extends
3 Number
```

Therefore many people expect:

```
1 List<Integer>
```

to be a subtype of:

```
1 List<Number>
```

But Java says:
No.

26.3 Why Java Refuses

Suppose Java allowed:

```
1 List<Integer> ints =
2 new ArrayList<>();
3
4 List<Number> nums =
5 ints;
```

Then:

```
1 nums.add(3.14);
```

Question:

Where does the Double go?

Into:

```
1 List<Integer>
```

Disaster.

Type safety destroyed.

26.4 The Core Rule

Rosetta Rule

Generics are invariant.

Meaning:

```
1 List<Integer>
```

is NOT a subtype of:

```
1 List<Number>
```

Even though:

```
1 Integer
2 extends Number
```

26.5 The Question Wildcards Solve

Suppose we want:

A method that accepts

List<Integer>

List<Double>

List<Float>

List<Long>

How?

Wildcards.

26.6 The Simplest Wildcard

Observe:

```
1 List<?>
```

Read as:

List of something.

Unknown type.

26.7 Example

```
1 List<Integer> a;  
2 List<String> b;  
3 List<Double> c;
```

All may be assigned to:

```
1 List<?>
```

Because:

We don't care about the exact type.

26.8 The Limitation

Observe:

```
1 List<?> list;
```

Can we do:

```
1 list.add(10);
```

?

No.

Compiler Error.

26.9 Why?

Because:

What if actual type is:

```
1 List<String>
```

?

Adding Integer would be unsafe.

26.10 The Read-Only View

Think:

```
1 List<?>
```

as:

Safe for reading.

Unsafe for writing.

26.11 Upper Bounded Wildcards

Observe:

```
1 List<? extends Number>
```

Read as:
List of some type
that extends Number.

26.12 What Can Match?

Valid:

```
1 List<Integer>
2
3 List<Double>
4
5 List<Float>
6
7 List<Long>
8
9 List<Number>
```

All accepted.

26.13 The Big Benefit

Observe:

```
1 double sum(
2 List<? extends Number>
3 list)
```

Now method works for:
Integers
Doubles
Floats
Anything numeric.

26.14 Reading Is Safe

Question:

Can we read?
Yes.
Every element is at least:

```
1 Number
```

Therefore:

```
1 Number n =
2 list.get(0);
```

Safe.

26.15 Writing Is Dangerous

Can we do:

```
1 list.add(10);
```

?

No.

Question:

What if actual type is:

```
1 List<Double>
```

?

Integer insertion breaks safety.

26.16 The Producer Idea

Think:

The list produces values.

We read from it.

Hence:

Rosetta Rule

Producer Extends.

26.17 Lower Bounded Wildcards

Observe:

```
1 List<? super Integer>
```

Read as:

List of Integer

or any superclass.

26.18 Possible Matches

Valid:

```
1 List<Integer>
```

```
2
```

```
3 List<Number>
```

```
4
```

```
5 List<Object>
```

26.19 The Big Difference

Now:

```
1 list.add(10);
```

Safe.

Because every valid target can store Integer.

26.20 Reading Is Hard

Observe:

```
1 Object x =  
2 list.get(0);
```

Only Object guaranteed.

Compiler cannot promise more.

26.21 The Consumer Idea

Think:

List consumes Integers.

We insert into it.

Hence:

Rosetta Rule

Consumer Super.

26.22 PECS

The most important wildcard rule.

PECS

Producer

Extends

Consumer

Super

26.23 Visualization

Read From

|
v

? extends Number

Producer

Write To

|
v

? super Integer

Consumer

26.24 The Framework Connection

Many professional APIs use:

? extends
? super
everywhere.
Streams.
Collections.
Frameworks.
Libraries.

26.25 Common Interview Question

Why:

```
1 List<Integer>
```

cannot be assigned to:

```
1 List<Number>
```

?
Answer:
Generics are invariant.
Type safety.

26.26 Rosetta Rules

Rosetta Rule

Generics are invariant.

Rosetta Rule

? means unknown type.

Rosetta Rule

? extends T means some subtype of T.

Rosetta Rule

? super T means some supertype of T.

Rosetta Rule

Read from extends.

Rosetta Rule

Write into super.

Rosetta Rule

PECS: Producer Extends Consumer Super.

26.27 Mental Compression

Invariant

⇒

List<Integer>
≠
List<Number>
? extends T

⇒

Read
Producer
? super T

⇒

Write
Consumer
PECS

⇒

Producer Extends
Consumer Super

Chapter 27

Exceptions – Java’s Philosophy of Failure

27.1 The Thirty Eighth Wrong Idea

Common Trap

Many beginners think:
Exceptions
=
Errors.
False.
An exception is not the failure.
An exception is information about the failure.
Java is saying:
Something unexpected happened.
What would you like to do?

27.2 The World Before Exceptions

Consider C.

```
1 FILE* f =  
2 fopen("data.txt", "r");
```

Question:

What if file does not exist?

Programmer manually checks:

```
1 if(f == NULL)  
2 {  
3     ...  
4 }
```

Everywhere.
Repeatedly.
Easy to forget.

27.3 Java's Approach

Instead of:

- Returning special values.
- Java creates an object.
- That object describes the problem.
- That object is:
Exception

27.4 The Fundamental Event

Imagine:

```
1 int x = 10 / 0;
```

- Java immediately says:
I cannot continue normally.
An exceptional situation has occurred.

27.5 What Actually Happens

Java creates:

```
1 ArithmeticException
```

- object.
- Then:
Throws it.

27.6 Throwing

Think:

- Return
sends normal result.
- Throw
sends failure information.

27.7 Mental Model

Imagine:

Method A

calls

Method B

calls

Method C

Problem occurs in C.
Exception travels upward.
Until someone handles it.

27.8 The Exception Highway

```
main()
    |
    v
A()
    |
    v
B()
    |
    v
C()
    X
Exception
    ^
    |
travels upward
```

This process is called:
Stack Unwinding

27.9 Handling Exceptions

Observe:

```
1 try
2 {
3     int x = 10 / 0;
4 }
5 catch(ArithmeticException e)
6 {
7     System.out.println(
8         "Cannot divide by zero"
```

```

9      );
10     }

```

Java says:
 Try this code.
 If failure occurs,
 jump to catch.

27.10 Meaning Of catch

Observe:

```

1  catch(
2  ArithmeticException e
3  )

```

e is just an object.
 Like:

```

1  Student s;

```

except the object describes a failure.

27.11 Exception Hierarchy

Everything derives from:

```

1  Throwable

```

Tree:

```

Throwable
|
+-- Error
|
+-- Exception
    |
    +-- RuntimeException
    |
    +-- IOException
    |
    +-- SQLException

```

27.12 The First Deep Distinction

Java separates:
 Errors
 and
 Exceptions

27.13 Errors

Examples:

- OutOfMemoryError
- StackOverflowError
- VirtualMachineError
- Usually indicate JVM problems.
- Not intended for normal recovery.

27.14 Exceptions

Examples:

- File missing.
- Invalid input.
- Database unavailable.
- Network disconnected.
- Programs often recover.

27.15 Checked vs Unchecked

One of Java's most important ideas.

- Checked Exceptions
- Unchecked Exceptions

27.16 Unchecked Exceptions

Derived from:

```
1 RuntimeException
```

Examples:

```
1 NullPointerException
2
3 ArithmeticException
4
5 IndexOutOfBoundsException
```

Compiler does not force handling.

27.17 Why?

Because these usually indicate:

- Programming mistakes.
- Not environmental failures.

27.18 Example

```
1 String s = null;
2
3 s.length();
```

Produces:

```
1 NullPointerException
```

Bug in program logic.

27.19 Checked Exceptions

Examples:

```
1 IOException
2
3 SQLException
4
5 ClassNotFoundException
```

Compiler forces handling.

27.20 Example

```
1 FileReader f =
2 new FileReader(
3 "data.txt"
4 );
```

Compiler says:
You must handle possibility that file does not exist.

27.21 Two Choices

Option 1:

```
1 try-catch
```

Option 2:

```
1 throws
```

27.22 throws

Observe:

```
1 public void read()
2 throws IOException
3 {
4     ...
5 }
```

Meaning:
I am not handling it.
My caller must decide.

27.23 Important Insight

throws
does NOT throw.
throws
declares possibility.
throw
actually creates and launches an exception.

27.24 throw

Observe:

```
1 throw new IllegalArgumentException(  
2 "Age cannot be negative"  
3 );
```

Programmer manually creates an exception.

27.25 finally

Observe:

```
1 try  
2 {  
3     ...  
4 }  
5 finally  
6 {  
7     ...  
8 }
```

finally executes almost always.
Whether exception occurs or not.

27.26 Why finally Exists

Suppose:

File opened.
Exception occurs.
File still must be closed.
finally guarantees cleanup.

27.27 Exception Propagation

Observe:

```
1 A()
2
3 calls
4
5 B()
6
7 calls
8
9 C()
```

C throws.
If B ignores,
goes to A.
If A ignores,
goes to main.
If nobody handles:
Program terminates.

27.28 Custom Exceptions

Observe:

```
1 class InvalidAgeException
2 extends Exception
3 {
4 }
```

Now domain language becomes part of the program.

27.29 OOP Interpretation

Exception is simply:

An object.
Polymorphism works.
Inheritance works.
Encapsulation works.
Everything learned earlier reappears.

27.30 Rosetta Rules

Rosetta Rule

Exception is an object describing failure.

Rosetta Rule

throw launches an exception.

Rosetta Rule

throws declares possibility.

Rosetta Rule

try contains risky code.

Rosetta Rule

catch handles failure.

Rosetta Rule

finally performs cleanup.

Rosetta Rule

Checked exceptions are enforced by compiler.

Rosetta Rule

Unchecked exceptions are usually programming mistakes.

27.31 Mental Compression

Exception

⇒

Failure Information
throw

⇒

Launch Failure
throws

⇒

Declare Failure
try

⇒

Risky Region
catch

⇒

Recovery
finally

⇒

Part V

Advanced Java

Chapter 28

ArrayList — The Dynamic Array

28.1 The Twenty Fifth Wrong Idea

Common Trap

Many beginners think:
ArrayList
=
Linked List
False.
ArrayList is fundamentally an array.
A very clever array.

28.2 The Problem Revisited

Suppose:

```
1 int[] arr =  
2 new int[10];
```

Question:

Can size become 20?

No.

Array size is fixed.

This is the fundamental limitation.

28.3 The Dream

We want:

```
1 ArrayList<Integer>  
2 numbers =  
3 new ArrayList<>();
```

Then:

```
1 numbers.add(1);  
2 numbers.add(2);
```

```
3 numbers.add(3);  
4 ...
```

without worrying about size.

28.4 The Hidden Truth

Internally:

ArrayList stores elements inside an array.

Conceptually:

```
1 class ArrayList<E>  
2 {  
3     E[] data;  
4  
5     int size;  
6 }
```

Actual implementation is more complex.

But this captures the essence.

28.5 The Two Numbers

ArrayList tracks:

Capacity

and

Size

Different concepts.

28.6 Example

Suppose:

```
1 ArrayList<String>  
2 names =  
3 new ArrayList<>();
```

Internally:

Capacity = 10

Size = 0

No elements yet.

28.7 After Insertions

```
1 names.add("Alice");  
2 names.add("Bob");  
3 names.add("Charlie");
```

Now:

Capacity = 10

Size = 3

Seven unused slots exist.

28.8 The Critical Moment

Suppose:

Capacity = 10

Size = 10

Array is full.

Question:

What happens during next add ()?

28.9 The Magic

Java creates a larger array.

Copies elements.

Switches reference.

Old array discarded.

28.10 Visualization

Before:

[1] [2] [3] [4] [5]

capacity = 5

Need one more element.

Create:

[][][][][]

[][][][][]

capacity = 10

Copy all data.

Then insert.

28.11 Important Observation

Resizing is expensive.

Must copy every element.

Cost:

$$O(n)$$

28.12 Then Why Is add() Fast?

Excellent question.

Most insertions do NOT resize.

Example:

Capacity = 1000

Size = 217

Adding one element merely places it into the next slot.

Cost:

$$O(1)$$

28.13 Amortized Analysis

Occasionally:

Very expensive resize.

Most of the time:

Very cheap insertion.

Average cost:

$$O(1)$$

called

Amortized $O(1)$

28.14 Accessing Elements

Observe:

```
1 numbers.get(500);
```

How many operations?

Array lookup.

Cost:

$$O(1)$$

Very fast.

28.15 Why Fast?

Arrays support:

$$address + index \times size$$

Direct computation.
No traversal.

28.16 Insertion At End

Observe:

```
1 numbers.add(42);
```

Usually:

$$O(1)$$

Unless resizing occurs.

28.17 Insertion In Middle

Observe:

```
1 numbers.add(0,99);
```

Question:

Where should existing elements go?

Need shifting.

Before

[1] [2] [3] [4]

Insert at front

After

[99] [1] [2] [3] [4]

Everything moves.
Cost:

$$O(n)$$

28.18 Deletion

Observe:

```
1 numbers.remove(0);
```

Again:

All remaining elements shift.

Cost:

$$O(n)$$

28.19 Summary Table

Operation	Complexity
get(index)	$O(1)$
set(index)	$O(1)$
add(end)	Amortized $O(1)$
insert(middle)	$O(n)$
delete(middle)	$O(n)$
search	$O(n)$

28.20 The Mental Model

ArrayList optimizes:

Random Access

Not

Middle Insertions

28.21 When To Use ArrayList

Use when:

Many reads.

Frequent indexing.

Appending at end.

Rare middle modifications.

28.22 Rosetta Rules

Rosetta Rule

ArrayList is built on arrays.

Rosetta Rule

ArrayList has capacity and size.

Rosetta Rule

ArrayList grows by resizing.

Rosetta Rule

Random access is $O(1)$.

Rosetta Rule

Appending is amortized $O(1)$.

Rosetta Rule

Middle insertion is $O(n)$.

Rosetta Rule

Middle deletion is $O(n)$.

28.23 Mental Compression

ArrayList

=

Array

+

Automatic Resizing
Strength:

$O(1)$

Random Access
Weakness:

$O(n)$

Middle Modifications

Chapter 29

LinkedList – Escaping The Array

29.1 The Twenty Sixth Wrong Idea

Common Trap

Many beginners think:
LinkedList
=
ArrayList
with different syntax.
False.
Their internal structures are completely different.
Nearly opposite design philosophies.

29.2 The Fundamental Array Problem

Observe:

[10] [20] [30] [40] [50]

Insert:

25

between:

20

and

30

Need:

[10] [20] [25] [30] [40] [50]

Question:

What happens to:

30

40

50

Must shift.

Cost:

$$O(n)$$

29.3 The Radical Idea

Instead of storing data together:

Store pieces separately.

Connect them using references.

29.4 Node

The fundamental building block.

Conceptually:

```
1 class Node
2 {
3     int data;
4
5     Node next;
6 }
```

Contains:

Data

and

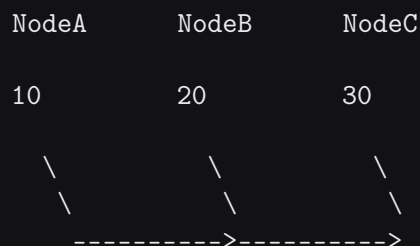
Reference to next node.

29.5 Visualization

Suppose:

10 -> 20 -> 30 -> 40

Actual memory:



Nodes need not be adjacent.

Huge difference from arrays.

29.6 Head Reference

How do we find list?

Need first node.

Called:

Head

Head

|

v

10 -> 20 -> 30

Everything begins from head.

29.7 Traversing

Question:

How to reach:

30

Must walk.

10 -> 20 -> 30

Step by step.

Cannot jump.

29.8 The First Shock

Observe:

```
1 list.get(500);
```

ArrayList:

$O(1)$

LinkedList:

$O(n)$

Must walk through nodes.

29.9 Insertion

Now the advantage.

Before:

10 -> 20 -> 40

Insert:

30

Create node.

Adjust references.

After:

10 -> 20 -> 30 -> 40

No shifting.

29.10 Mental Picture

Array insertion:

Move many elements.

Linked insertion:

Redirect arrows.

29.11 Insertion At Front

Observe:

Head

|

10 -> 20 -> 30

Insert:

5

Simply:

5 -> 10 -> 20 -> 30

Change head.

Cost:

$O(1)$

29.12 Deletion

Before:

10 -> 20 -> 30 -> 40

Delete:

30

Redirect:

20 -> 40

Node removed.

No shifting.

29.13 The Hidden Cost

Question: How do we find node?? :

30

Must traverse first.

Cost:

$O(n)$

Therefore:

Deletion itself:

$O(1)$

Finding node:

$O(n)$

Total often behaves like:

$O(n)$

29.14 Singly Linked List

Each node stores:

```
1 next
```

only.
Movement:
Forward.
Not backward.

29.15 Doubly Linked List

Store:

```
1 next  
2 prev
```

Conceptually:

10 <-> 20 <-> 30

Can move both directions.
More memory.
More flexibility.

29.16 Java's LinkedList

Java implements:

Doubly Linked List.
Internally similar to:

```
1 class Node<E>  
2 {  
3     E item;  
4  
5     Node<E> next;  
6  
7     Node<E> prev;  
8 }
```

Actual code is more complex.

29.17 Operations

Observe:

```
1 addFirst()  
2 addLast()  
3  
4 removeFirst()  
5 removeLast()
```

Very natural operations.
Designed around node manipulation.

29.18 Complexity Table

Operation	LinkedList
Access	$O(n)$
Search	$O(n)$
Insert Front	$O(1)$
Delete Front	$O(1)$
Insert End	$O(1)$
Delete End	$O(1)$

29.19 ArrayList vs LinkedList

Operation	ArrayList	LinkedList
Random Access	$O(1)$	$O(n)$
Insert Front	$O(n)$	$O(1)$
Delete Front	$O(n)$	$O(1)$
Memory	Less	More
Cache Friendly	Excellent	Poor

29.20 The Cache Story

Modern CPUs love arrays.

Reason:

Adjacent memory.

Cache loads nearby elements.

Very efficient.

Linked lists scatter nodes.

Cache performance worsens.

29.21 The Great Surprise

Many beginners expect:

LinkedList

to always beat

ArrayList

for insertion.

Often false in real programs.

Cache effects matter.

Traversal costs matter.

Modern hardware changes theory.

29.22 Rosetta Rules

Rosetta Rule

LinkedList stores nodes.

Rosetta Rule

Nodes are connected by references.

Rosetta Rule

Random access is $O(n)$.

Rosetta Rule

Insertion at known position is $O(1)$.

Rosetta Rule

Traversal is $O(n)$.

Rosetta Rule

Java LinkedList is doubly linked.

Rosetta Rule

ArrayList optimizes access.

Rosetta Rule

LinkedList optimizes structural modification.

29.23 Mental Compression

Array

$\Rightarrow$

Contiguous Memory

$\Rightarrow$

Fast Access
Linked List

$\Rightarrow$

Connected Nodes

$\Rightarrow$

Flexible Structure
ArrayList

$\Rightarrow$

Read Heavy
LinkedList

Chapter 30

The List Interface – Programming To Abstractions

30.1 The Twenty Seventh Wrong Idea

Common Trap

Many beginners think:
ArrayList
is the type.
False.
ArrayList is one implementation.
List is the abstraction.

30.2 A Strange Declaration

Observe:

```
1 List<Integer> list =  
2 new ArrayList<>();
```

Question:

Why not simply write:

```
1 ArrayList<Integer> list =  
2 new ArrayList<>();
```

Both compile.
The difference is philosophical.

30.3 The Vehicle Analogy

Suppose:

You need transportation.
Do you ask for:
Toyota Corolla
or
Vehicle?

Usually:
Vehicle.
You care about capability.
Not implementation.

30.4 The OOP Principle

Recall:

Interface
defines:
What can be done.
Not:
How it is done.

30.5 List As A Contract

Conceptually:

```
1 interface List<E>
2 {
3     add(E e);
4
5     remove(E e);
6
7     get(int index);
8
9     size();
10 }
```

Actual interface is larger.
But this is the essence.

30.6 ArrayList As An Implementation

```
1 class ArrayList<E>
2 implements List<E>
3 {
4     ...
5 }
```

ArrayList promises:
"I satisfy the List contract."

30.7 LinkedList As Another Implementation

```
1 class LinkedList<E>
2 implements List<E>
3 {
4     ...
5 }
```

Same contract.
Different implementation.

30.8 The Key Insight

Observe:

```
1 List<Integer> list;
```

Compiler knows:
What operations exist.
It does not care how.

30.9 Polymorphism Returns

Observe:

```
1 List<Integer> list =  
2 new ArrayList<>();
```

Later:

```
1 list =  
2 new LinkedList<>();
```

Still valid.
Variable type unchanged.
Implementation changed.

30.10 The Deep Meaning

The variable says:

What I need.

The object says:

How it is achieved.

30.11 A Real Example

Suppose:

```
1 public void process(  
2     List<String> names)  
3 {  
4     ...  
5 }
```

Accepts:
ArrayList
LinkedList
Vector
Any List implementation.
Huge flexibility.

30.12 Bad Design

Observe:

```
1 public void process(  
2     ArrayList<String> names)  
3 {  
4     ...  
5 }
```

Now only ArrayList accepted.
Unnecessary restriction.

30.13 Programming To Interfaces

One of the most important Java principles.

Rosetta Rule

Program to abstractions.
Not implementations.

30.14 Why This Matters

Suppose today:

```
1 ArrayList
```

works best.
Tomorrow:

```
1 LinkedList
```

becomes better.
Minimal code changes required.

30.15 The Collection Hierarchy

```
Iterable  
|  
Collection  
|  
List  
|  
-----  
|           |  
ArrayList   LinkedList
```

Notice:
Variable type usually lives higher.
Object type usually lives lower.

30.16 A Familiar Pattern

We already saw:

```
1 Animal a =  
2 new Dog();
```

Now:

```
1 List<Integer> list =  
2 new ArrayList<>();
```

Exactly same idea.
Interface reference.
Concrete object.

30.17 The Interface Advantage

Suppose Java invents:

```
1 SuperFastList
```

that also implements List.
Code written against:

```
1 List
```

continues working.
Future-proof design.

30.18 Mental Model

Mental Model

List
= What
ArrayList
= How

30.19 Common Collection Interfaces

Collection



Implementations:

```
List
|
+-- ArrayList
|
+-- LinkedList
```

```
Set
|
+-- HashSet
|
+-- TreeSet
```

```
Queue
|
+-- PriorityQueue
|
+-- ArrayDeque
```

30.20 Rosetta Rules

Rosetta Rule

Interfaces define contracts.

Rosetta Rule

ArrayList implements List.

Rosetta Rule

LinkedList implements List.

Rosetta Rule

Program to interfaces.

Rosetta Rule

Depend on abstractions.

Rosetta Rule

Polymorphism applies to collections.

Rosetta Rule

List reference can store many implementations.

30.21 Mental Compression

List

Chapter 31

HashSet – The Collection of Uniqueness

31.1 The Twenty Eighth Wrong Idea

Common Trap

Many beginners think:
HashSet
=
ArrayList
without duplicates.
False.
Internally they are completely different structures.
Different goals.
Different complexity guarantees.

31.2 The Philosophy Shift

Recall:

List
cares about:
Position
Order
Index
Now:
Set
cares about:
Uniqueness
Membership

31.3 The Question

Suppose:

```
1 List<String> names =  
2 new ArrayList<>();
```

Need to know:
Does "Alice" exist?

Typical solution:

```
1 names.contains("Alice");
```

Must potentially scan list.

Cost:

$$O(n)$$

31.4 The Dream

Can we answer:

Exists?

without scanning everything?

That is exactly what HashSet tries to achieve.

31.5 Set Interface

Conceptually:

```
1 Set<E>
```

extends:

```
1 Collection<E>
```

but adds a promise:

No duplicates.

31.6 Example

```
1 Set<String> names =  
2 new HashSet<>();  
3  
4 names.add("Alice");  
5 names.add("Bob");  
6 names.add("Alice");
```

Question:

How many elements?

Answer:

2

Duplicate rejected.

31.7 The Internal Idea

HashSet is essentially:

Hashing

+

Buckets

+

Equality

The same concepts from the hashCode() chapter.

31.8 Mental Picture

Imagine:

Bucket 0

Bucket 1

Bucket 2

Bucket 3

Bucket 4

Bucket 5

Each object hashes into some bucket.

31.9 Adding An Element

Suppose:

```
1 names.add("Alice");
```

Step 1:

Compute hash.

Step 2:

Find bucket.

Step 3:

Check if equal element already exists.

Step 4:

If not found, insert.

31.10 Why equals() Matters

Suppose:

```
1 Student s1 =  
2 new Student("23A01");  
3
```

```

4 Student s2 =
5 new Student("23A01");

```

Logically same student.

Question:

Will HashSet detect duplicate?

Only if:

equals()

and

hashCode()

are implemented correctly.

31.11 The Classic Bug

```

1 Set<Student> set =
2 new HashSet<>();
3
4 set.add(
5 new Student("101")
6 );
7
8 set.add(
9 new Student("101")
10 );

```

Expected:

1

Actual:

2

Possible if equals/hashCode are broken.

31.12 Contains Operation

Observe:

```

1 set.contains("Alice");

```

HashSet:

Compute hash.

Jump to bucket.

Check equals().

Average complexity:

$O(1)$

Huge improvement over:

$O(n)$

31.13 No Ordering

Observe:

```
1 Set<Integer> set =  
2 new HashSet<>();  
3  
4 set.add(30);  
5 set.add(10);  
6 set.add(20);
```

Question:

Output order?

No guarantee.

May appear as:

```
1 20 30 10
```

or something else.

31.14 The Important Trade

HashSet gains speed by giving up:

Ordering

Indexing

Position

31.15 No get(index)

Observe:

```
1 set.get(0);
```

Compiler Error.

Sets have no index concept.

31.16 Iteration

Still possible:

```
1 for(String s : set)  
2 {  
3     System.out.println(s);  
4 }
```

Because:

HashSet implements Iterable.

31.17 Complexity Table

Operation	Average Cost
add()	$O(1)$
contains()	$O(1)$
remove()	$O(1)$
iteration	$O(n)$

31.18 How HashSet Is Actually Built

Surprising fact.

Internally:

HashSet is largely implemented using HashMap.

Conceptually:

```
1 HashMap<E, Object>
```

Element stored as key.

Dummy value stored as value.

We will understand this fully after HashMap.

31.19 When To Use HashSet

Use when:

Duplicates forbidden.

Fast membership testing needed.

Ordering unimportant.

31.20 Examples

Unique usernames.

Visited graph vertices.

Unique email addresses.

Dictionary words.

Authentication caches.

31.21 List vs Set

Property	List	Set
Duplicates	Allowed	Forbidden
Order	Preserved	Not Guaranteed
Indexing	Yes	No
contains()	$O(n)$	$O(1)$

31.22 Rosetta Rules

Rosetta Rule

HashSet stores unique elements.

Rosetta Rule

HashSet uses hashing.

Rosetta Rule

HashSet depends on equals() and hashCode().

Rosetta Rule

Duplicates are rejected.

Rosetta Rule

contains() is typically $O(1)$.

Rosetta Rule

HashSet provides no indexing.

Rosetta Rule

HashSet provides no ordering guarantee.

31.23 Mental Compression

List

⇒

Order
HashSet

⇒

Uniqueness
Hash

⇒

Bucket
Bucket

⇒

equals()
equals()

⇒

Chapter 32

HashMap – The Kingdom of Key-Value Thinking

32.1 The Twenty Ninth Wrong Idea

Common Trap

Many beginners think:
HashMap
=
ArrayList with names.
False.
HashMap introduces a completely different way of organizing data.
Not by position.
Not by order.
But by keys.

32.2 The Problem

Suppose:

```
1 String[] names;  
2 int[] marks;
```

Need:

Alice -> 95

Bob -> 88

Charlie -> 91

Question:

How do we find Bob's marks?

Search names array.

Then access marks array.

Inefficient.

32.3 The Real Requirement

We want:

Alice -> 95

Bob -> 88

Charlie -> 91

and later:
Give me Bob's marks.
Directly.

32.4 The Key-Value Idea

HashMap stores:

Key → Value

Examples:

Name → Marks

Username → PasswordHash

RollNumber → Student

Country → Capital

32.5 Creating A HashMap

```
1 Map<String,Integer> marks =  
2 new HashMap<>();
```

Notice:
Interface

Map

Implementation

HashMap

Same philosophy as List.

32.6 Adding Data

```
1 marks.put("Alice",95);
2
3 marks.put("Bob",88);
4
5 marks.put("Charlie",91);
```

Internally:

Alice -> 95

Bob -> 88

Charlie -> 91

32.7 Retrieving Data

Observe:

```
1 marks.get("Bob");
```

Output:

88

Question:

How fast?

Average:

$O(1)$

32.8 The Magic

HashMap uses:

hashCode()

to locate bucket.

Then:

equals()

to verify key.

Exactly like HashSet.

32.9 The Internal Picture

Conceptually:

Bucket 0

Bucket 1

Bucket 2

Bucket 3

Bucket 4

Keys determine location.
Values travel with keys.

32.10 A Better Mental Model

Think:

HashSet stores:

Key

HashMap stores:

Key $\rightarrow$ *Value*

HashMap is the more general structure.

32.11 The Surprising Fact

Conceptually:

HashSet

=

HashMap

without meaningful values

This is why HashSet can be implemented using HashMap.

32.12 Duplicate Keys

Observe:

```
1 marks.put("Bob",88);
2
3 marks.put("Bob",95);
```


Question:

How many Bobs?

One.

Second value replaces first.

Result:

$Bob \rightarrow 95$

32.13 Important Rule

Rosetta Rule

Keys must be unique.

Values may repeat.

32.14 Contains Operations

Observe:

```
1 marks.containsKey("Bob");
```

Checks:

Key existence.

Average:

$O(1)$

```
1 marks.containsValue(88);
```

Checks values.

Must scan.

Cost:

$O(n)$

32.15 Null Values

Observe:

```
1 marks.put("Alice", null);
```

Valid.

HashMap permits null values.

32.16 Null Keys

Observe:

```
1 marks.put(null,100);
```

Also valid.

Only one null key allowed.

32.17 Iteration

Suppose:

```
1 for(String key :  
2 marks.keySet())  
3 {  
4     ...  
5 }
```

Visits all keys.

32.18 Entry Objects

More powerful:

```
1 for(  
2 Map.Entry<String,Integer> e  
3 :  
4 marks.entrySet()  
5 )  
6 {  
7     ...  
8 }
```

Provides:

Key

and

Value

simultaneously.

32.19 Visualizing Entry

Think:

Entry

```
+-----+  
| Key    |  
| Value  |  
+-----+
```

HashMap stores many entries.

32.20 Complexity Table

Operation	Average Cost
put()	O(1)
get()	O(1)
containsKey()	O(1)
remove()	O(1)
Iteration	O(n)

32.21 The equals()/hashCode() Disaster

Suppose:

```
1 Student s =  
2 new Student("101");
```

Used as key.

If equals/hashCode are broken:

Map retrieval may fail.

This is why earlier chapters were critical.

32.22 Real World Examples

DNS

Domain $\rightarrow$ *IP*

Compiler Symbol Tables

Identifier $\rightarrow$ *Information*

Caches

Request $\rightarrow$ *Response*

Databases

PrimaryKey $\rightarrow$ *Record*

Operating Systems

PID $\rightarrow$ *Process*

32.23 HashMap vs ArrayList

Property	ArrayList	HashMap
Lookup By Index	$O(1)$	N/A
Lookup By Key	$O(n)$	$O(1)$
Ordering	Yes	No
Duplicates	Yes	Keys No

32.24 Rosetta Rules

Rosetta Rule

HashMap stores key-value pairs.

Rosetta Rule

Keys must be unique.

Rosetta Rule

Values may repeat.

Rosetta Rule

HashMap uses hashCode().

Rosetta Rule

HashMap uses equals().

Rosetta Rule

get() is typically $O(1)$.

Rosetta Rule

put() is typically $O(1)$.

Rosetta Rule

HashSet is conceptually a HashMap with only keys.

32.25 Mental Compression

HashSet

$\Rightarrow$

Key
HashMap

$\Rightarrow$

Chapter 33

Exceptions – Java’s Philosophy of Failure

33.1 The Thirty Eighth Wrong Idea

Common Trap

Many beginners think:
Exceptions
=
Errors.
False.
An exception is not the failure.
An exception is information about the failure.
Java is saying:
Something unexpected happened.
What would you like to do?

33.2 The World Before Exceptions

Consider C.

```
1 FILE* f =  
2 fopen("data.txt", "r");
```

Question:

What if file does not exist?

Programmer manually checks:

```
1 if(f == NULL)  
2 {  
3     ...  
4 }
```

Everywhere.
Repeatedly.
Easy to forget.

33.3 Java's Approach

Instead of:

- Returning special values.
- Java creates an object.
- That object describes the problem.
- That object is:
Exception

33.4 The Fundamental Event

Imagine:

```
1 int x = 10 / 0;
```

- Java immediately says:
I cannot continue normally.
An exceptional situation has occurred.

33.5 What Actually Happens

Java creates:

```
1 ArithmeticException
```

- object.
- Then:
Throws it.

33.6 Throwing

Think:

- Return
sends normal result.
- Throw
sends failure information.

33.7 Mental Model

Imagine:

Method A

calls

Method B

calls

Method C

Problem occurs in C.
Exception travels upward.
Until someone handles it.

33.8 The Exception Highway

```
main()
|
v
A()
|
v
B()
|
v
C()
X
Exception
^
|
travels upward
```

This process is called:
Stack Unwinding

33.9 Handling Exceptions

Observe:

```
1 try
2 {
3     int x = 10 / 0;
4 }
5 catch(ArithmeticException e)
6 {
7     System.out.println(
8         "Cannot divide by zero"
```

```

9      );
10     }

```

Java says:
 Try this code.
 If failure occurs,
 jump to catch.

33.10 Meaning Of catch

Observe:

```

1  catch(
2  ArithmeticException e
3  )

```

e is just an object.
 Like:

```

1  Student s;

```

except the object describes a failure.

33.11 Exception Hierarchy

Everything derives from:

```

1  Throwable

```

Tree:

```

Throwable
|
+-- Error
|
+-- Exception
    |
    +-- RuntimeException
    |
    +-- IOException
    |
    +-- SQLException

```

33.12 The First Deep Distinction

Java separates:
 Errors
 and
 Exceptions

33.13 Errors

Examples:

- OutOfMemoryError
- StackOverflowError
- VirtualMachineError
- Usually indicate JVM problems.
- Not intended for normal recovery.

33.14 Exceptions

Examples:

- File missing.
- Invalid input.
- Database unavailable.
- Network disconnected.
- Programs often recover.

33.15 Checked vs Unchecked

One of Java's most important ideas.

- Checked Exceptions
- Unchecked Exceptions

33.16 Unchecked Exceptions

Derived from:

```
1 RuntimeException
```

Examples:

```
1 NullPointerException
2
3 ArithmeticException
4
5 IndexOutOfBoundsException
```

Compiler does not force handling.

33.17 Why?

Because these usually indicate:

- Programming mistakes.
- Not environmental failures.

33.18 Example

```
1 String s = null;
2
3 s.length();
```

Produces:

```
1 NullPointerException
```

Bug in program logic.

33.19 Checked Exceptions

Examples:

```
1 IOException
2
3 SQLException
4
5 ClassNotFoundException
```

Compiler forces handling.

33.20 Example

```
1 FileReader f =
2 new FileReader(
3 "data.txt"
4 );
```

Compiler says:
You must handle possibility that file does not exist.

33.21 Two Choices

Option 1:

```
1 try-catch
```

Option 2:

```
1 throws
```

33.22 throws

Observe:

```
1 public void read()
2 throws IOException
3 {
4     ...
5 }
```

Meaning:
I am not handling it.
My caller must decide.

33.23 Important Insight

throws
does NOT throw.
throws
declares possibility.
throw
actually creates and launches an exception.

33.24 throw

Observe:

```
1 throw new IllegalArgumentException(  
2 "Age cannot be negative"  
3 );
```

Programmer manually creates an exception.

33.25 finally

Observe:

```
1 try  
2 {  
3     ...  
4 }  
5 finally  
6 {  
7     ...  
8 }
```

finally executes almost always.
Whether exception occurs or not.

33.26 Why finally Exists

Suppose:

File opened.
Exception occurs.
File still must be closed.
finally guarantees cleanup.

33.27 Exception Propagation

Observe:

```
1 A()
2
3 calls
4
5 B()
6
7 calls
8
9 C()
```

C throws.
If B ignores,
goes to A.
If A ignores,
goes to main.
If nobody handles:
Program terminates.

33.28 Custom Exceptions

Observe:

```
1 class InvalidAgeException
2 extends Exception
3 {
4 }
```

Now domain language becomes part of the program.

33.29 OOP Interpretation

Exception is simply:

An object.
Polymorphism works.
Inheritance works.
Encapsulation works.
Everything learned earlier reappears.

33.30 Rosetta Rules

Rosetta Rule

Exception is an object describing failure.

Rosetta Rule

throw launches an exception.

Rosetta Rule

throws declares possibility.

Rosetta Rule

try contains risky code.

Rosetta Rule

catch handles failure.

Rosetta Rule

finally performs cleanup.

Rosetta Rule

Checked exceptions are enforced by compiler.

Rosetta Rule

Unchecked exceptions are usually programming mistakes.

33.31 Mental Compression

Exception

⇒

Failure Information
throw

⇒

Launch Failure
throws

⇒

Declare Failure
try

⇒

Risky Region
catch

⇒

Recovery
finally

⇒

Chapter 34

Functional Interfaces and Lambda Expressions

34.1 The Thirty Ninth Wrong Idea

Common Trap

Many beginners think:
Lambda
=
Short Function Syntax.
False.
Lambda is Java's mechanism for passing behavior.
The short syntax is merely a consequence.

34.2 The Historical Problem

Suppose we want:

Sort students by marks.

Before Java 8:

```
1 class MarksComparator
2 implements Comparator<Student>
3 {
4     @Override
5     public int compare(
6         Student a,
7         Student b)
8     {
9         return a.marks
10            - b.marks;
11     }
12 }
```

Then:

```
1 Collections.sort(
2     students,
3     new MarksComparator()
4 );
```

Question:

How much code actually contains business logic?

Only:

```
1 a.marks - b.marks
```

Everything else is ceremony.

34.3 The Deeper Question

What are we really passing?

An object?

Not exactly.

We are passing:

A comparison behavior.

34.4 Java's Original Limitation

Unlike C:

No function pointers.

Unlike modern languages:

No first-class functions.

Everything had to be wrapped inside objects.

34.5 The Key Observation

Consider:

```
1 interface Comparator<T>
2 {
3     int compare(
4         T a,
5         T b
6     );
7 }
```

Question:

How many abstract methods?

Exactly one.

Very important.

34.6 Functional Interface

Definition:

An interface containing exactly one abstract method.

Examples:

Comparator

Runnable

Callable

Consumer

Supplier

Function

34.7 Why One Method Matters

Suppose:

```
1 interface Flyable
2 {
3     void fly();
4 }
```

Question:

If someone writes:

```
1 () -> ...
```

Which method should Java connect it to?

Only one exists.

No ambiguity.

34.8 The Lambda Syntax

Observe:

```
1 (a,b)
2 -> a.marks - b.marks
```

Read as:

Given a and b,

return comparison result.

Go Back

34.9 What Java Actually Creates

Many beginners imagine:

Function created.

Not exactly.

Java internally creates an object implementing the functional interface.

34.10 Mental Translation

Java roughly transforms:

```
1 (a,b)
2 -> a.marks - b.marks
```

into:

```
1 new Comparator<Student>()
2 {
3     @Override
4     public int compare(
5         Student a,
6         Student b)
7     {
```

```

8         return
9         a.marks-b.marks;
10    }
11 }

```

Conceptually.

34.11 The Structure

General form:

```

1 (parameters)
2 -> expression

```

or

```

1 (parameters)
2 -> {
3     statements
4 }

```

34.12 Examples

No parameters:

```

1 () -> System.out.println(
2     "Hello"
3 )

```

One parameter:

```

1 x -> x*x

```

Two parameters:

```

1 (a,b) -> a+b

```

34.13 The Runnable Example

Old Java:

```

1 Runnable r =
2 new Runnable()
3 {
4     @Override
5     public void run()
6     {
7         System.out.println(
8             "Running"
9         );
10    }
11 };

```

Lambda:

```
1 Runnable r =  
2 () -> System.out.println(  
3 "Running"  
4 );
```

Same meaning.
Much smaller.

34.14 The @FunctionalInterface Annotation

Observe:

```
1 @FunctionalInterface  
2 interface Flyable  
3 {  
4     void fly();  
5 }
```

Compiler verifies:
Exactly one abstract method.

34.15 Why Lambdas Matter

Without Lambdas:

Behavior
must be wrapped inside classes.

With Lambdas:

Behavior becomes portable.
Can be stored.
Passed around.
Returned.

34.16 Collections Example

Observe:

```
1 students.sort(  
2 (a,b)  
3 -> a.marks - b.marks  
4 );
```

Comparator object created automatically.

34.17 Variable Capture

Observe:

```

1  int factor = 10;
2
3  Function<Integer,Integer>
4  f =
5  x -> x * factor;

```

Lambda can use surrounding variables.
This is called:
Closure

34.18 The Effective Final Rule

Observe:

```

1  int x = 10;

```

Lambda may capture x.
But x must not later change.
Effectively Final.

34.19 The Functional Package

Java provides:

```

1  java.util.function

```

Containing:
Function
Consumer
Supplier
Predicate
BiFunction
Many others.

34.20 Four Interfaces To Remember Forever

Predicate<T>

Question:
Returns boolean.

```

1  x -> x > 10

```

Function<T,R>
Transforms value.

```

1  x -> x*x

```

Consumer<T>
Consumes value.

```
1 x -> System.out.println(x)
```

Supplier<T>
Produces value.

```
1 () -> 42
```

34.21 The Deep OOP Shift

Old Java:

Object

Modern Java:

Object + Behavior

Behavior itself becomes something that can travel.

34.22 Rosetta Rules

Rosetta Rule

A functional interface has one abstract method.

Rosetta Rule

Lambdas implement functional interfaces.

Rosetta Rule

Lambdas represent behavior.

Rosetta Rule

Java creates interface implementations behind the scenes.

Rosetta Rule

@FunctionalInterface enables compiler checking.

Rosetta Rule

Predicate returns boolean.

Rosetta Rule

Function transforms data.

Rosetta Rule

Consumer consumes data.

Rosetta Rule

Supplier produces data.

34.23 Mental Compression

Lambda

⇒

Portable Behavior
Functional Interface

⇒

One Abstract Method
Predicate

⇒

Question
Function

⇒

Transformation
Consumer

⇒

Use Value
Supplier

⇒

Generate Value
Lambda

⇒

Behavior As Data

Chapter 35

Method References – When Lambdas Become Redundant

35.1 The Fortieth Wrong Idea

Common Trap

Many beginners think:

Method Reference

=

Different Feature.

False.

Method References are simply a shorter way of writing certain lambdas.

Every method reference can be expressed as a lambda.

Not every lambda can be expressed as a method reference.

35.2 The Motivation

Consider:

```
1 names.forEach(  
2 x -> System.out.println(x)  
3 );
```

Observe carefully.

Lambda receives:

x

Then immediately passes *x* to another method.

No extra logic.

No transformation.

No decision.

Just forwarding.

35.3 The Simplification

Java allows:

```
1 names.forEach(  
2 System.out::println  
3 );
```

This is called:
Method Reference

35.4 The Symbol

Observe:

```
1 ::
```

Read as:
Method Reference Operator
Sometimes called:
Double Colon Operator

35.5 Mental Model

Think:

```
1 System.out::println
```

as:
Use this method whenever the functional interface needs implementation.

35.6 The Hidden Translation

Java roughly converts:

```
1 System.out::println
```

into:

```
1 x -> System.out.println(x)
```

Conceptually.

35.7 The Important Requirement

Method References work only when:
Method signature matches the functional interface.

35.8 Example

Consumer<String>
requires:

```
1 void accept(String s)
```

```
    println(String)
    matches.
    Therefore:
```

```
1 System.out::println
```

```
    is valid.
```

35.9 Four Categories

Java provides four major forms of method references.
Very important.

35.10 Category 1

Static Method Reference
Syntax:

```
1 ClassName::method
```

Example:

```
1 Integer::parseInt
```

Equivalent to:

```
1 s -> Integer.parseInt(s)
```

35.11 Example

```
1 Function<String,Integer>
2 f =
3 Integer::parseInt;
```

Usage:

```
1 f.apply("123");
```

Returns:

123

35.12 Category 2

Instance Method Of Particular Object

Syntax:

```
1 object::method
```

Example:

```
1 System.out::println
```

Equivalent:

```
1 x ->  
2 System.out.println(x)
```

35.13 Category 3

Instance Method Of Arbitrary Object

Most confusing category.

Syntax:

```
1 ClassName::method
```

But method is not static.

Example:

```
1 String::length
```

Equivalent:

```
1 s -> s.length()
```

35.14 Example

```
1 Function<String,Integer>  
2 f =  
3 String::length;
```

Usage:

```
1 f.apply("Hello");
```

Returns:

35.15 Category 4

Constructor Reference

Syntax:

```
1 ClassName::new
```

Example:

```
1 Supplier<Student>  
2 s =  
3 Student::new;
```

Equivalent:

```
1 () -> new Student()
```

35.16 Mental Interpretation

Method Reference does not execute immediately.

It creates behavior.

Execution happens later.

Exactly like lambdas.

35.17 Collections Example

Observe:

```
1 list.forEach(  
2 System.out::println  
3 );
```

For each element:

println called automatically.

35.18 Sorting Example

Observe:

```
1 list.sort(  
2 String::compareToIgnoreCase  
3 );
```

Equivalent to:

```
1 (a,b)  
2 ->  
3 a.compareToIgnoreCase(b)
```

35.19 Method Reference vs Lambda

Question:

Should we always use method references?

No.

Use method reference only when:

Lambda merely forwards call.

35.20 Bad Example

```
1 x -> x*x
```

No existing method.

Cannot become method reference.

35.21 Good Example

```
1 x -> Integer.parseInt(x)
```

Becomes:

```
1 Integer::parseInt
```

35.22 The Deeper Insight

Lambdas and Method References are both implementations of:

Functional Interfaces

Never forget this.

Method references are not a separate execution model.

Only cleaner syntax.

35.23 Rosetta Rules

Rosetta Rule

Method References are shorthand for lambdas.

Rosetta Rule

They use the :: operator.

Rosetta Rule

Signature must match the functional interface.

Rosetta Rule

Static methods may be referenced.

Rosetta Rule

Instance methods may be referenced.

Rosetta Rule

Constructors may be referenced.

Rosetta Rule

Method References create behavior.

Rosetta Rule

Execution happens later.

35.24 Mental Compression

Lambda

⇒

Behavior
Method Reference

⇒

Existing Behavior
::

⇒

Method Reference Operator
Class::new

⇒

Constructor Reference
System.out::println

⇒

Consumer
String::length

⇒

Function

Chapter 36

Streams – The Functional View of Collections

36.1 The Fortieth First Wrong Idea

Common Trap

Many beginners think:
Stream
=
Collection.
False.
A Stream is not storage.
A Stream is a pipeline of operations performed on data.
Collections store.
Streams process.

36.2 The Fundamental Shift

Old Java thinking:

- Get data.
- Loop manually.
- Process manually.
- Store manually.

Modern Java thinking:
Describe transformations.
Let Stream execute them.

36.3 A Familiar Example

Suppose:

```
1 List<Integer> nums =  
2 Arrays.asList(  
3 1,2,3,4,5,6,7,8,9  
4 );
```

Goal:

Keep even numbers.
Square them.
Sum them.

36.4 Traditional Approach

```
1 int sum = 0;
2
3 for(Integer x : nums)
4 {
5     if(x % 2 == 0)
6     {
7         sum += x*x;
8     }
9 }
```

Works.
But Java sees:
Filtering
Transformation
Reduction
all mixed together.

36.5 Stream Approach

```
1 int sum =
2 nums.stream()
3     .filter(x -> x%2==0)
4     .map(x -> x*x)
5     .reduce(
6         0,
7         Integer::sum
8     );
```

Observe:
What
not
How

36.6 The Stream Pipeline

Every stream typically contains:

Source

|

Intermediate Operations

|

Terminal Operation

36.7 Visualization

Collection

```
|  
stream()  
|  
filter()  
|  
map()  
|  
sorted()  
|  
collect()
```

36.8 Source

Example:

```
1 nums.stream()
```

Creates stream from collection.
Data source.

36.9 Intermediate Operations

Examples:

```
filter()  
map()  
sorted()  
distinct()  
limit()  
skip()
```

These create new streams.
They do not produce final result.

36.10 Terminal Operations

Examples:

```
collect()
forEach()
count()
reduce()
findFirst()
anyMatch()
These finish the pipeline.
```

36.11 filter()

Question:

Should element survive?

Uses:

```
1 Predicate<T>
```

Example:

```
1 .filter(  
2 x -> x % 2 == 0  
3 )
```

Keeps only even numbers.

36.12 Visualization

```
1 2 3 4 5 6
```

```
|
```

```
filter even
```

```
|
```

```
2 4 6
```

36.13 map()

Question:

How should element transform?

Uses:

```
1 Function<T,R>
```

Example:

```
1 .map(x -> x*x)
```

36.14 Visualization

2 4 6

|

square

|

4 16 36

36.15 collect()

Most common terminal operation.

Example:

```
1 List<Integer> result =  
2 nums.stream()  
3   .filter(x -> x%2==0)  
4   .collect(  
5     Collectors.toList()  
6   );
```

Stream converted back into collection.

36.16 forEach()

Consumes elements.

Example:

```
1 nums.stream()  
2   .forEach(  
3     System.out::println  
4   );
```

36.17 sorted()

Example:

```
1 nums.stream()  
2   .sorted();
```

Uses natural ordering.

36.18 Custom Sorting

```
1 students.stream()
2   .sorted(
3     (a,b)
4     ->
5     a.marks-b.marks
6   );
```

Comparator again appears.
Everything connects.

36.19 distinct()

Removes duplicates.
Example:

```
1 nums.stream()
2   .distinct();
```

36.20 limit()

Take first N elements.

```
1 .limit(5)
```

36.21 skip()

Ignore first N elements.

```
1 .skip(5)
```

36.22 reduce()

The most important terminal operation.

Question:

How can many values become one?

36.23 Example

```
1 int sum =
2 nums.stream()
3   .reduce(
4     0,
5     Integer::sum
6   );
```

Visualization:

1 2 3 4

|

1+2+3+4

|

10

36.24 Lazy Evaluation

One of the deepest ideas.

Observe:

```
1 nums.stream()  
2   .filter(...)  
3   .map(...);
```

Nothing happens.

No terminal operation.

No execution.

36.25 Why?

Streams are lazy.

They build a plan first.

Execute later.

36.26 The Pipeline Mental Model

Think:

Pipeline Blueprint

not

Pipeline Execution

until terminal operation arrives.

36.27 The OOP Connection

Observe:

Collections

store objects.

Streams

transform objects.

Functional Interfaces

define operations.

Lambdas

provide behavior.

Everything learned earlier converges here.

36.28 Rosetta Rules

Rosetta Rule

Collections store data.

Rosetta Rule

Streams process data.

Rosetta Rule

A stream pipeline contains:

Source

Intermediate Operations

Terminal Operation

Rosetta Rule

`filter()` uses Predicate.

Rosetta Rule

`map()` uses Function.

Rosetta Rule

`forEach()` uses Consumer.

Rosetta Rule

`reduce()` combines many values into one.

Rosetta Rule

Streams are lazy.

Rosetta Rule

Terminal operations trigger execution.

36.29 Mental Compression

Collection

⇒

Storage

Stream

⇒

Processing Pipeline

filter

⇒

Selection
map

⇒

Transformation
reduce

⇒

Aggregation
collect

⇒

Build Result
Lazy

⇒

Plan First
Execute Later

Chapter 37

[

Advanced Streams — flatMap, Optional, Collectors and Parallel Streams]

37.1 The Forty Second Wrong Idea

Common Trap

Many beginners think:

Streams

=

Better Loops.

False.

Streams are a data-processing language embedded inside Java.

The deeper operations appear when processing becomes complex.

37.2 The Problem Of Nested Collections

Suppose:

```
1 List<List<Integer>>  
2 data;
```

Example:

```
[  
  [1,2],  
  [3,4],  
  [5,6]  
]
```

Question:

How can we obtain:

```
1 2 3 4 5 6
```

One sequence.

37.3 The First Attempt

```
1 data.stream()  
2   .map(x -> x.stream())
```

Result:

```
Stream<  
  Stream<Integer>  
>
```

Not what we wanted.
We created:
A stream of streams.

37.4 Visualization

```
[1,2]  
[3,4]  
[5,6]
```

```
|
```

```
map(stream)
```

```
|
```

```
S1  
S2  
S3
```

Still nested.

37.5 The Solution

Use:

```
1 flatMap()
```

37.6 Example

```
1 data.stream()  
2   .flatMap(  
3     list -> list.stream()  
4   );
```

Result:

1 2 3 4 5 6

Single stream.

37.7 Mental Model

map()

⇒

Transform.
flatMap()

⇒

Transform
+
Flatten.

37.8 Visualization

```
List<List<Integer>>
```

|

```
flatMap
```

|

```
List<Integer>
```

37.9 Where flatMap Appears

Nested collections.

JSON processing.

Database joins.

Optional pipelines.

File processing.

37.10 The Next Problem

Suppose:

```
1 Student s =  
2 findStudent();
```

Question:

What if student not found?

Traditionally:

```
1 return null;
```

37.11 The Null Problem

Observe:

```
1 Student s =  
2 findStudent();  
3  
4 s.getMarks();
```

Possible:

```
1 NullPointerException
```

One of the most common bugs in Java history.

37.12 The Optional Idea

Instead of returning:

```
1 Student
```

return:

```
1 Optional<Student>
```

Meaning:

Value may exist.

Value may not exist.

37.13 Mental Interpretation

Think:

Box

containing

0 or 1 object

37.14 Creating Optional

Observe:

```
1 Optional<Integer> x =  
2 Optional.of(10);
```

Contains:

10

37.15 Empty Optional

Observe:

```
1 Optional<Integer> x =  
2 Optional.empty();
```

Contains nothing.

37.16 Checking

```
1 if(x.isPresent())  
2 {  
3     ...  
4 }
```

Question:

Does value exist?

37.17 Retrieving

```
1 x.get();
```

Returns value.

Dangerous if empty.

37.18 Safer Retrieval

Observe:

```
1 x.orElse(0);
```

Meaning:

Give stored value.

Otherwise return 0.

37.19 Example

```
1 int value =  
2 x.orElse(100);
```

If empty:
returns 100.

37.20 Optional Pipeline

Observe:

```
1 student  
2 .map(  
3     Student::getMarks  
4 )  
5 .orElse(0);
```

No null checks.
Cleaner logic.

37.21 Collectors

Question:

After processing a stream,
where should results go?

Answer:
Collectors

37.22 Most Common Collector

```
1 Collectors.toList()
```

Example:

```
1 List<Integer> result =  
2 stream.collect(  
3     Collectors.toList()  
4 );
```

37.23 toSet()

```
1 Collectors.toSet()
```

Produces:
Set

37.24 joining()

Observe:

```
1 Collectors.joining(",")
```

Example:

A,B,C,D

Built from stream.

37.25 counting()

Observe:

```
1 Collectors.counting()
```

Counts elements.

37.26 groupingBy()

One of the most powerful.

Example:

```
1 students.stream()  
2 .collect(  
3 Collectors.groupingBy(  
4 Student::getBranch  
5 )  
6 );
```

Produces:

```
CSE -> [...]  
ECE -> [...]  
ME  -> [...]
```

Like automatic HashMap creation.

37.27 partitioningBy()

Special case of grouping.

Only two groups.

True

False

Example:

```
1 Collectors.partitioningBy(  
2 x -> x % 2 == 0  
3 )
```

Even numbers.

Odd numbers.

37.28 Parallel Streams

One of the most misunderstood features.
Observe:

```
1 nums.parallelStream()
```

Question:

What changes?

Execution strategy.

37.29 Visualization

Normal Stream:

```
1 2 3 4 5 6
```

```
|
```

```
single thread
```

```
|
```

```
result
```

Parallel Stream:

```
1 2 | 3 4 | 5 6
```

```
\      |      /
```

```
multiple threads
```

```
|
```

```
result
```

37.30 The Promise

Large datasets.
Multiple CPU cores.
Potential speedup.

37.31 The Trap

Parallel

≠

Automatically Faster.
Thread coordination costs.
Data splitting costs.
Merging costs.

37.32 When Parallel Helps

Huge datasets.
CPU-intensive calculations.
Independent operations.

37.33 When Parallel Hurts

Small datasets.
Heavy synchronization.
I/O-heavy workloads.

37.34 The Grand Connection

Observe how everything now joins:

```
Collections
  |
Generics
  |
Iterator
  |
Lambda
  |
Method Reference
  |
Streams
  |
Collectors
  |
Optional
```

Modern Java is built on this stack.

37.35 Rosetta Rules

Rosetta Rule

map transforms.

Rosetta Rule

flatMap transforms and flattens.

Rosetta Rule

Optional represents possible absence.

Rosetta Rule

Avoid null when Optional expresses intent.

Rosetta Rule

Collectors build final results.

Rosetta Rule

groupBy creates grouped maps.

Rosetta Rule

partitioningBy creates true/false groups.

Rosetta Rule

Parallel Streams change execution strategy.

Rosetta Rule

Parallel does not guarantee faster.

37.36 Mental Compression

flatMap

⇒

Flatten Nested Streams
Optional

⇒

Maybe Value
Collectors

⇒

Build Results
groupBy

⇒

Automatic HashMap
partitioningBy

⇒

True/False Split
parallelStream

⇒

Multi-Core Processing

Chapter 38

[

Concurrency and Multithreading — Multiple Flows of Execution]

38.1 The Forty Third Wrong Idea

Common Trap

Many beginners think:
Thread
=
Small Program.
False.
A thread is a flow of execution.
A program may contain many threads.
All sharing the same process.

38.2 The C Programmer's Mental Model

Until now:

```
main()
```

```
|
```

```
A
```

```
|
```

```
B
```

```
|
```

```
C
```

```
|
```

```
End
```

One execution path.
One instruction stream.

38.3 The New World

Imagine:

Thread 1

A
|
B
|
C

Thread 2

X
|
Y
|
Z

Both execute simultaneously.

38.4 Process vs Thread

One of the most important interview questions.

Process

=

Independent running program.

Thread

=

Execution path inside process.

38.5 Visualization

Chrome Process

|
+--- Thread
|
+--- Thread
|
+--- Thread
|
+--- Thread

38.6 Why Threads Exist

Suppose:

- Downloading file.
- Updating GUI.
- Listening to network.
- Playing audio.
- All at once.

38.7 Without Threads

Download

Then

GUI

Then

Audio

Everything blocks everything.

38.8 With Threads

Download Thread

GUI Thread

Audio Thread

Network Thread

Independent execution.

38.9 Creating Threads

Classical method:

```
1 class MyThread
2 extends Thread
3 {
4     @Override
5     public void run()
6     {
7         System.out.println(
8             "Running"
9         );
10    }
11 }
```

38.10 Starting Thread

Observe:

```
1  MyThread t =  
2  new MyThread();  
3  
4  t.start();
```

Question:

Why not:

```
1  t.run();
```

38.11 Critical Insight

```
1  run()
```

=

Normal method call.

No new thread.

```
1  start()
```

=

JVM creates new thread.

Then JVM invokes run().

38.12 The Preferred Method

Instead of extending Thread:

```
1  class Task  
2  implements Runnable  
3  {  
4      public void run()  
5      {  
6          ...  
7      }  
8  }
```

Then:

```
1  Thread t =  
2  new Thread(  
3  new Task()  
4  );  
5  
6  t.start();
```

38.13 Why Runnable Is Better

Inheritance already consumed?

No problem.

Runnable separates:

Task

from

Thread

Very important design idea.

38.14 Lambda Version

Because Runnable is a functional interface:

```
1 Thread t =  
2 new Thread(  
3 () ->  
4 {  
5     System.out.println(  
6     "Running"  
7     );  
8 }  
9 );
```

Modern Java style.

38.15 Thread States

Conceptually:

NEW

|

RUNNABLE

|

RUNNING

|

BLOCKED

|

TERMINATED

38.16 sleep()

Observe:

```
1 Thread.sleep(1000);
```

Meaning:

Pause approximately 1000 milliseconds.

38.17 The Scheduler

Question:

Which thread runs now?

Answer:

JVM + Operating System Scheduler decide.

Not programmer.

38.18 Interleaving

Suppose:

```
1 Thread A
```

prints:

A

Thread B prints:

B

Possible output:

AABB

ABAB

BBAA

BABA

All legal.

38.19 The Dangerous Part

Multiple threads.

Shared memory.

Now timing matters.

38.20 Race Condition

Observe:

```
1 counter++;
```

Looks harmless.

Actually:

Read

Modify

Write

Three operations.

38.21 The Problem

Suppose:

```
counter = 5
```

Thread A reads 5.

Thread B reads 5.

Thread A writes 6.

Thread B writes 6.

Expected:

7

Actual:

6

Lost update.

Race condition.

38.22 synchronized

Java provides:

```
1 synchronized
```

To protect critical sections.

Example:

```

1 public synchronized
2 void increment()
3 {
4     counter++;
5 }

```

Only one thread enters at a time.

38.23 Monitor Lock

Every Java object secretly contains a monitor.

Think:

Door lock.

Thread must obtain key.

38.24 Deadlock

The most famous concurrency bug.

Thread A:

Needs Lock 1

Then Lock 2

Thread B:

Needs Lock 2

Then Lock 1

Result:

Nobody moves.

38.25 Visualization

A holds L1

waiting for L2

B holds L2

waiting for L1

Permanent waiting.

38.26 join()

Observe:

```

1 t.join();

```

Meaning:

Wait until thread t finishes.

38.27 Executor Framework

Modern Java rarely creates threads manually.
Instead:

```
1 ExecutorService
```

manages thread pools.

38.28 Example

```
1 ExecutorService pool =  
2 Executors  
3 .newFixedThreadPool(4);
```

Four worker threads.
Reusable.
Efficient.

38.29 Future

Suppose task returns value.
Use:

```
1 Future<T>
```

Represents result available later.

38.30 Mental Model

Future

=

Promise of future value.

38.31 The Deeper Truth

Concurrency is not mainly about speed.
It is about managing multiple activities.
Parallelism
is often about speed.
Concurrency
is about structure.

38.32 Rosetta Rules

Rosetta Rule

Process contains threads.

Rosetta Rule

Thread is a flow of execution.

Rosetta Rule

`start()` creates a new thread.

Rosetta Rule

`run()` alone does not.

Rosetta Rule

`Runnable` represents work.

Rosetta Rule

Multiple threads create race conditions.

Rosetta Rule

`synchronized` protects critical sections.

Rosetta Rule

Deadlock is circular waiting.

Rosetta Rule

`ExecutorService` is preferred over manual thread management.

38.33 Mental Compression

Process

⇒

Program
Thread

⇒

Execution Path
Runnable

⇒

Work
Thread

⇒

Worker
synchronized

⇒

Lock
Race Condition

⇒

Timing Bug
Deadlock

⇒

Circular Waiting
ExecutorService

⇒

Thread Manager

Chapter 39

[

Java Memory Model, volatile, Atomic Classes and Concurrent Collections]

39.1 The Forty Fourth Wrong Idea

Common Trap

Many beginners think:
If Thread A changes a variable,
Thread B immediately sees it.
False.
Visibility is not guaranteed.
Modern CPUs have caches.
Each thread may observe different memory states.

39.2 The Invisible Enemy

Suppose:

```
1 boolean running = true;
```

Thread A:

```
1 while(running)
2 {
3 }
```

Thread B:

```
1 running = false;
```

Question:

Will Thread A stop?

Many beginners say:

Obviously yes.

Reality:

Not necessarily.

39.3 Why?

CPU may cache:

```
1 running
```

Thread A keeps reading cache.
Never observes new value.
Loop may continue forever.

39.4 The Java Memory Model

JMM defines:

What threads may see.
When writes become visible.
Which reorderings are legal.
Without JMM:
Multithreaded Java would be impossible.

39.5 The Core Problem

Two different questions:

Visibility
Atomicity
Most bugs come from confusing them.

39.6 Visibility

Question:

Can another thread see my write?

Example:

```
1 x = 100;
```

Visibility concerns whether other threads observe 100.

39.7 Atomicity

Question:

Can operation be interrupted?

Example:

```
1 counter++;
```

Actually:

Read

Modify

Write

Not atomic.

39.8 volatile

Java provides:

```
1 volatile
```

Example:

```
1 volatile boolean running;
```

Meaning:

Do not keep stale cached copies.

Always synchronize visibility.

39.9 What volatile Guarantees

Visibility.

Ordering.

NOT atomicity.

Very important.

39.10 The Famous Trap

Observe:

```
1 volatile int count;
```

Then:

```
1 count++;
```

Still unsafe.

Why?

`count++`

is three operations.

volatile cannot combine them.

39.11 The Happens-Before Relationship

Most important JMM idea.

Think:

Visibility guarantee chain.

If:

A

happens-before

B

Then effects of A must be visible to B.

39.12 Example

Writing volatile variable:

⇒

Reading same volatile variable.
Creates happens-before relationship.

39.13 synchronized Revisited

Observe:

```
1 synchronized(lock)
2 {
3     ...
4 }
```

Many beginners think:
Only locking.
Actually provides:
Mutual exclusion.
Visibility guarantees.
Memory synchronization.

39.14 The Next Problem

Suppose:

```
1 counter++;
```

Millions of times.
Using synchronized works.
But can be expensive.

39.15 Atomic Classes

Package:

```
1 java.util.concurrent.atomic
```

Provides lock-free operations.

39.16 AtomicInteger

Observe:

```
1 AtomicInteger count =  
2 new AtomicInteger(0);
```

Increment:

```
1 count.incrementAndGet();
```

Thread-safe.

Atomic.

39.17 Mental Interpretation

AtomicInteger behaves like:

Safe shared integer.

Special hardware-assisted updates.

39.18 Compare-And-Set

Fundamental atomic operation.

Observe:

```
1 compareAndSet(  
2 oldValue,  
3 newValue  
4 );
```

Meaning:

If current value equals oldValue

replace with newValue.

Otherwise fail.

39.19 Why CAS Matters

Modern lock-free algorithms are built on:

CAS

instead of locks.

39.20 The Collection Problem

Suppose:

```
1 HashMap<K,V>
```

shared by many threads.

Safe?

No.

39.21 Why HashMap Fails

Multiple threads may:

- Resize simultaneously.
- Insert simultaneously.
- Corrupt internal structure.

39.22 Synchronized Collection

Old approach:

```
1 Collections
2 .synchronizedMap(...)
```

One giant lock.
Correct.
But may reduce scalability.

39.23 ConcurrentHashMap

Modern solution.
Observe:

```
1 ConcurrentHashMap
2 <
3 String,
4 Integer
5 >
6 map;
```

Thread-safe.
Designed for concurrency.

39.24 Mental Model

HashMap:

⇒

Single-thread assumption.
ConcurrentHashMap:

⇒

Concurrent access expected.

39.25 Other Concurrent Collections

`ConcurrentLinkedQueue`

`CopyOnWriteArrayList`

`BlockingQueue`

`ConcurrentSkipListMap`

Each optimized for different concurrency patterns.

39.26 BlockingQueue

Extremely important.

Producer:

adds tasks.

Consumer:

removes tasks.

Thread-safe communication.

39.27 Producer Consumer Pattern

Producer

|

`BlockingQueue`

|

Consumer

Fundamental concurrent design.

39.28 The Deep Insight

Concurrency bugs often occur without crashes.

Program runs.

Results wrong.

Hardest category of bugs.

39.29 The Concurrency Ladder

Thread

|

Synchronization

|

Visibility

|
Atomicity

|
JMM

|
Lock-Free Structures

Understanding increases at each level.

39.30 Rosetta Rules

Rosetta Rule

Visibility and atomicity are different.

Rosetta Rule

volatile guarantees visibility.

Rosetta Rule

volatile does not guarantee atomicity.

Rosetta Rule

synchronized provides locking and visibility.

Rosetta Rule

AtomicInteger provides atomic updates.

Rosetta Rule

CAS powers many lock-free algorithms.

Rosetta Rule

HashMap is not thread-safe.

Rosetta Rule

ConcurrentHashMap is designed for concurrency.

Rosetta Rule

BlockingQueue supports producer-consumer systems.

39.31 Mental Compression

Visibility

⇒

Can Others See?

Atomicity

⇒

Can It Be Interrupted?

volatile

⇒

Visibility

synchronized

⇒

Lock + Visibility

AtomicInteger

⇒

Atomic Updates

CAS

⇒

Lock-Free Primitive

ConcurrentHashMap

⇒

Thread-Safe Map

Chapter 40

JVM Internals — Stack, Heap, Class Loading and Bytecode

40.1 The Forty Fifth Wrong Idea

Common Trap

Many beginners think:
Java
=
Language.
False.
Java is actually:
Language
+
Compiler
+
Bytecode
+
JVM
+
Runtime
All together form the system.

40.2 The Complete Journey

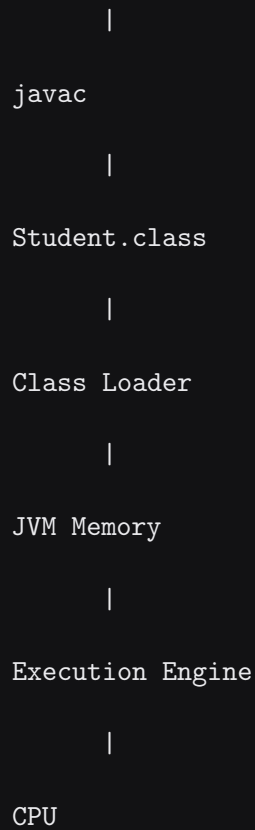
Consider:

```
1 Student s =  
2 new Student();
```

Many things happen.
Not one.

40.3 The Big Picture

Student.java



Everything begins here.

40.4 Source Code

You write:

```
1 public class Student
2 {
3     int marks;
4 }
```

This is not executed.
It is merely text.

40.5 Compilation

Java compiler:

```
1 javac Student.java
```

produces:

```
1 Student.class
```

40.6 Critical Observation

.class file
does NOT contain machine code.
It contains:
Bytecode

40.7 Mental Model

Think:
C

⇒

Machine Code
Java

⇒

Bytecode

40.8 Why Bytecode Exists

Question:
Why not compile directly?
Answer:
Portability.

40.9 Visualization

Windows JVM

|

Bytecode

|

Linux JVM

|

Bytecode

|

Mac JVM

Same class file.
Different machines.

40.10 The JVM

JVM means:

Java Virtual Machine

40.11 Mental Model

Think:

Tiny Computer

inside

your computer.

40.12 What JVM Provides

Memory Management

Garbage Collection

Class Loading

Thread Management

Security

Execution

40.13 The Runtime Memory Layout

Every Java program contains:

Method Area

Heap

Java Stack

PC Register

Native Stack

These are crucial.

40.14 Visualization

```
+-----+
|  Method Area  |
+-----+
```

```
+-----+
|      Heap      |
+-----+
```

```
+-----+
|      Stack      |
+-----+
```


We now study each.

40.15 The Stack

Most important connection to C and C++.

40.16 What Lives In Stack

Local variables
Method calls
References
Function state

40.17 Example

```
1 void f()  
2 {  
3     int x = 10;  
4 }
```

x lives in stack.

40.18 Method Invocation

```
1 main()  
2 {  
3     A();  
4 }
```

Creates:
Stack Frame
for A.

40.19 Visualization

A()

main()

Top frame executes.

40.20 Stack Frame

Contains:

- Local variables
- Parameters
- Return information
- Temporary values

40.21 Recursive Calls

Observe:

```
1 fact(5)
```

Creates:

fact(1)

fact(2)

fact(3)

fact(4)

fact(5)

Multiple frames.

40.22 Stack Overflow

Too many frames.

Result:

```
1 StackOverflowError
```

40.23 The Heap

Question:

Where do objects live?

Answer:

Heap.

40.24 Example

```
1 Student s =  
2 new Student();
```

Student object
lives in heap.

40.25 Visualization

Stack

s
|
|
v

Heap

Student Object

40.26 Critical Insight

Reference

is not object.

Reference points to object.

Exactly like a restricted pointer.

Mental Model

C++ Pointer

=

Address

Java Reference

=

Managed Handle

Both point somewhere.

Java simply hides arithmetic.

40.27 Method Area

Stores:

Class metadata

Method definitions

Static variables

Runtime constant pool

40.28 Example

```
1 class Student
2 {
3     static int count;
4 }
```

count belongs here.

Not in heap.

40.29 Class Loading

Question:

How does JVM discover Student?

Answer:

Class Loader.

40.30 Stages

Loading

Linking

Initialization

40.31 Loading

JVM finds:

```
1 Student.class
```

Reads bytecode.

Builds runtime representation.

40.32 Linking

Contains:

Verification

Preparation

Resolution

40.33 Verification

Checks:

Valid bytecode?

Illegal instructions?

Corruption?

40.34 Initialization

Executes:

```
1 static
2 {
3     ...
4 }
```

Static initialization occurs.

40.35 Bytecode

Question:

What is inside .class file?
Bytecode instructions.

40.36 View Bytecode

Command:

```
1 javap -c Student
```

Displays bytecode.

40.37 Example

Source:

```
1 int a = 5;  
2 int b = 7;  
3 int c = a + b;
```

Approximate Bytecode:

iconst_5

istore_1

bipush 7

istore_2

iload_1

iload_2

iadd

istore_3

Notice:

Stack-based execution.

40.38 JVM Stack Machine

Unlike many CPUs.

Bytecode often operates through operand stacks.

40.39 Visualization

Push 5

Push 7

Add

Store Result

40.40 Execution Engine

Question:

Who runs bytecode?

Execution Engine.

Historically:

Interpreter.

Later:

JIT Compiler.

Next chapter.

40.41 Rosetta Rules

Rosetta Rule

Java source is not executed directly.

Rosetta Rule

javac produces bytecode.

Rosetta Rule

Bytecode runs on JVM.

Rosetta Rule

Local variables live in stack frames.

Rosetta Rule

Objects live in heap.

Rosetta Rule

References are not objects.

Rosetta Rule

Static members live in Method Area.

Rosetta Rule

Class Loader brings classes into JVM.

Rosetta Rule

Bytecode is platform independent.

Rosetta Rule

JVM executes bytecode.

Mental Compression

Source Code

⇒

Bytecode

⇒

JVM

⇒

Machine
Stack

⇒

Method Execution
Heap

⇒

Objects
Method Area

⇒

Class Metadata
Reference

⇒

Handle To Object

Chapter 41

JIT Compiler, HotSpot and Runtime Optimizations

41.1 The Forty Seventh Wrong Idea

Common Trap

Many beginners think:
Java executes bytecode.
Bytecode is interpreted.
Therefore Java is always slow.
False.
Modern JVMs do much more than interpret bytecode.

41.2 The Historical JVM

Originally:

Source

|

javac

|

Bytecode

|

Interpreter

|

CPU

Interpreter executes one instruction at a time.

Simple.
Portable.
Not very fast.

41.3 The Problem

Suppose:

```
1  for(int i=0;i<100000000;i++)  
2  {  
3      sum += i;  
4  }
```

Interpreter repeatedly decodes same instructions.
Millions of times.
Wasteful.

41.4 The Observation

Programs are not uniformly important.
Some code runs once.
Some code runs millions of times.

41.5 Example

Startup code:

⇒

Rare.
Inner loops:

⇒

Extremely frequent.

41.6 The HotSpot Idea

Question:

Can JVM identify frequently executed code?

Answer:

Yes.

This became:

HotSpot.

41.7 Mental Model

Think:

Cold Code

Warm Code

Hot Code

HotSpot focuses on hot code.

41.8 The JIT Compiler

JIT means:

Just In Time Compiler

41.9 The Big Idea

Instead of:

Compile everything immediately.

JVM waits.

Observes execution.

Then compiles important parts.

41.10 Visualization

Bytecode

|

Interpreter

|

Frequently Used?

|

YES

|

JIT Compile

|

Native Machine Code

41.11 Critical Insight

JVM sees actual runtime behavior.

Traditional compiler does not.

This is powerful.

41.12 Method Invocation Counters

JVM tracks:

How often methods execute.

41.13 Example

```
1 calculate();
```

called:

10 times

100 times

1000 times

10000 times

Eventually:

Hot Method.

41.14 Hot Code

Frequently executed code.

Worth optimizing.

41.15 Cold Code

Rarely executed code.

Leave interpreted.

41.16 Why This Matters

Traditional compilation:

Optimize everything.

JVM:

Optimize only important parts.

Efficient strategy.

41.17 Inlining

One of the most important optimizations.

Suppose:

```

1 int square(int x)
2 {
3     return x*x;
4 }

```

Used:

```

1 sum += square(i);

```

Normally:
Function call.
Return.
Overhead.

41.18 Inlining

JIT may transform:

```

1 sum += square(i);

```

into:

```

1 sum += i*i;

```

No method call.
Faster.

Mental Model

Inlining
=
Copying small method body
directly into caller.

41.19 Dead Code Elimination

Observe:

```

1 int x = 5;
2 x = 10;

```

First assignment useless.
JIT may remove it.

41.20 Loop Optimization

JIT aggressively optimizes loops.
Because loops often dominate runtime.

41.21 Constant Folding

Observe:

```
1 int x = 5 + 7;
```

Can become:

```
1 int x = 12;
```

Compile-time computation.

41.22 Escape Analysis

One of the most fascinating optimizations.

Question:

Does object leave method?

41.23 Example

```
1 void f()  
2 {  
3     Point p =  
4     new Point();  
5 }
```

p never escapes.
Only used locally.

41.24 Traditional Thinking

Object

⇒

Heap.
Always.

41.25 HotSpot Thinking

Maybe not.

Maybe object never needs heap.

41.26 Stack Allocation

If object never escapes:

JIT may place it on stack.
Not heap.
Huge optimization.

Deep Insight

This is one reason modern Java can sometimes approach C++ speed.
The JVM learns from runtime behavior.

41.27 Allocation Elimination

Even more surprising.

Sometimes JVM removes object entirely.

If object has no observable effect.

41.28 Speculative Optimization

JVM makes educated guesses.

Example:

Method usually receives one type.

JIT optimizes for that type.

41.29 Deoptimization

What if guess becomes wrong?

JVM can rollback.

Return to interpreter.

Very unusual capability.

41.30 Tiered Compilation

Modern JVM uses multiple levels.

Interpretation.

Basic compilation.

Aggressive compilation.

41.31 Why Java Can Surprise People

Traditional compiler:

Compile once.

Done.

JVM:

Observe.

Measure.

Learn.

Optimize.

Adapt.

During execution.

41.32 The Cost

Nothing is free.

JIT requires:

Memory.

Profiling.

Compilation time.

Complex runtime.

41.33 Startup vs Long Runtime

Short program:

May not benefit much.

Long-running application:

Benefits enormously.

41.34 Rosetta Rules

Rosetta Rule

Modern JVM is not merely an interpreter.

Rosetta Rule

HotSpot identifies frequently executed code.

Rosetta Rule

JIT compiles hot code into native machine code.

Rosetta Rule

Inlining removes many method calls.

Rosetta Rule

Escape Analysis studies object lifetime and visibility.

Rosetta Rule

Some objects may avoid heap allocation entirely.

Rosetta Rule

JVM can optimize during execution.

Rosetta Rule

JVM can also deoptimize.

Mental Compression

Bytecode

⇒

Interpreter

⇒

Hot Code

⇒

JIT

⇒

Native Code

Inlining

⇒

Remove Function Calls

Escape Analysis

⇒

Does Object Leave Scope?

Deoptimization

⇒

Rollback Optimization

HotSpot

⇒

Observe + Learn + Optimize

Chapter 42

Reflection – Making Programs Examine Themselves

42.1 The Forty Eighth Wrong Idea

Common Trap

Many beginners think:
Classes are fixed.
Objects are fixed.
Code simply executes.
False.
Java programs can inspect their own structure at runtime.
This ability is called:
Reflection.

42.2 The Strange Question

Suppose:

```
1 class Student
2 {
3     private String name;
4     private int marks;
5
6     public void study()
7     {
8     }
9 }
```

Question:

Can a running program discover:

Class Name?

Fields?

Methods?

Constructors?

Inheritance?

Annotations?

Answer:

Yes.
Reflection provides this.

42.3 The Class Object

Most important concept.
Every loaded class has a corresponding object.
Type:

```
1 Class
```

42.4 Mental Model

Think:
Student
=
Blueprint For Objects
Class<Student>
=
Blueprint Information
Metadata about Student.

42.5 Obtaining Class Objects

Method 1:

```
1 Student.class
```

Method 2:

```
1 obj.getClass()
```

Method 3:

```
1 Class.forName(  
2 "Student"  
3 );
```

All produce a Class object.

42.6 The Most Famous Reflection Call

Observe:

```
1 Class<?> c =  
2 Student.class;
```

Question:
What is inside c?
Answer:
Metadata describing Student.

42.7 Discovering Class Name

```
1 System.out.println(  
2 c.getName()  
3 );
```

Possible output:

Student

42.8 Discovering Fields

```
1 Field[] fields =  
2 c.getDeclaredFields();
```

Returns all fields.
Even private fields.

42.9 Discovering Methods

```
1 Method[] methods =  
2 c.getDeclaredMethods();
```

Returns method metadata.

42.10 Discovering Constructors

```
1 Constructor<?>[] cons =  
2 c.getDeclaredConstructors();
```

Returns constructor information.

42.11 Visualizing Reflection

Student.class

|

Class Object

|

Fields

Methods

Constructors

42.12 Creating Objects Dynamically

Question:

Can we create an object without writing:

```
new Student()
```

?

Answer:

Yes.

```
1 Class<?> c =  
2 Student.class;  
3  
4 Object obj =  
5 c.getDeclaredConstructor()  
6 .newInstance();
```

Object created dynamically.

42.13 Why This Matters

Frameworks often do not know your classes beforehand.

They discover them at runtime.

42.14 Invoking Methods Dynamically

Suppose:

```
1 public void study()  
2 {  
3 }
```

Reflection:

```
1 Method m =  
2 c.getMethod("study");  
3  
4 m.invoke(obj);
```

Method called dynamically.

42.15 Mental Model

Normal Call:

```
1 obj.study();
```

Reflection Call:

```
1 method.invoke(obj);
```

One is compile-time.
One is runtime.

42.16 Accessing Private Fields

Observe:

```
1 private int marks;
```

Normally inaccessible.
Reflection can bypass this.

```
1 Field f =  
2 c.getDeclaredField(  
3 "marks"  
4 );  
5  
6 f.setAccessible(true);
```

Now field can be accessed.

42.17 The Danger

Reflection can break encapsulation.
Powerful.
Dangerous.

Common Trap

Reflection can access details the original class designer never intended to expose.

42.18 Reflection and Generics

Question:

Can Reflection always know:
List<String>
?
Not always.
Because of:
Type Erasure.
We studied this earlier.

42.19 Reflection Performance

Reflection is usually slower than:

```
1 obj.method();
```

because runtime lookup occurs.

42.20 Reflection in Real Frameworks

Spring:

- Discovers components.

- Creates objects.

- Injects dependencies.

Hibernate:

- Maps objects to database rows.

Jackson:

- Converts JSON to objects.

JUnit:

- Finds test methods.

42.21 The Deep Insight

Without Reflection:

- Frameworks would require massive manual configuration.

With Reflection:

- Programs can understand other programs.

Deep Insight

Reflection turns Java classes into data.

Once classes become data,
programs can manipulate them.

42.22 Rosetta Rules

Rosetta Rule

Every loaded class has a Class object.

Rosetta Rule

Reflection allows runtime inspection of classes.

Rosetta Rule

Fields, methods and constructors can be discovered dynamically.

Rosetta Rule

Reflection can create objects.

Rosetta Rule

Reflection can invoke methods.

Rosetta Rule

Reflection can bypass encapsulation.

Rosetta Rule

Reflection powers many modern Java frameworks.

Mental Compression

Class

⇒

Blueprint

Class Object

⇒

Metadata About Blueprint

Reflection

⇒

Inspect Program At Runtime

Method.invoke()

⇒

Dynamic Method Call

Field

⇒

Runtime Field Access

Reflection

⇒

Program Examines Itself

Part VI

Design and Architecture

Chapter 43

[

Abstract Classes — Incomplete Ideas and Partial Reality]

43.1 The Thirteenth Wrong Idea

Common Trap

Many beginners think:
Abstract Class
=
Class With Abstract Methods
Technically correct.
Conceptually weak.

43.2 The Real Question

Suppose:

Animal

exists.
Can an Animal speak?
Usually yes.
Can an Animal eat?
Usually yes.
Can we create a generic Animal?

```
1 new Animal()
```

What kind?
Dog?
Cat?
Horse?
Not clear.

Deep Insight

Some concepts are too general to exist directly.

43.3 The Geometry Example

Suppose:

Shape

Can we create:

```
1 new Shape()  
  
?  
What would that look like?  
Circle?  
Rectangle?  
Triangle?  
No answer exists.  
Yet:
```

Shape

is still useful.

43.4 The Key Observation

Some concepts exist only as abstractions.

Examples:

- Shape
- Animal
- Vehicle
- Employee

Useful for reasoning.

Not necessarily useful as concrete objects.

43.5 The Abstract Class

Java provides:

```
1 abstract class Animal  
2 {  
3 }
```

Meaning:

Animal is incomplete.

Animal cannot be instantiated.

43.6 Instantiation Attempt

```
1 Animal a =  
2 new Animal();
```

Compiler:

Illegal.

Because Animal is abstract.

43.7 The Mental Model

Mental Model

Blueprint.
Not building.
Recipe.
Not meal.
Map.
Not territory.

Abstract classes describe possibilities.

Concrete classes create reality.

43.8 Concrete Subclass

```
1 abstract class Animal  
2 {  
3 }
```

```
1 class Dog extends Animal  
2 {  
3 }
```

Now:

```
1 Dog d =  
2 new Dog();
```

Valid.

43.9 Abstract Methods

Suppose every animal speaks.

But:

Dog

and

Cat

speak differently.
Can Animal implement:

```
1 speak()
```

?
Not really.
So Animal says:

```
1 abstract void speak();
```

Meaning:

Every Animal must speak.

But:

I do not know how.

43.10 First Complete Example

```
1 abstract class Animal
2 {
3     abstract void speak();
4 }
```

```
1 class Dog extends Animal
2 {
3     @Override
4     void speak()
5     {
6         System.out.println("Woof");
7     }
8 }
```

```
1 class Cat extends Animal
2 {
3     @Override
4     void speak()
5     {
6         System.out.println("Meow");
7     }
8 }
```

Now every Animal provides a version.

43.11 Contract Without Implementation

Observe:

```
1 abstract void speak();
```

No body.

No implementation.

Only a promise.

This is a huge idea.

43.12 Abstract Class Can Have Code

Common beginner misconception.

Many think:

Abstract = NoImplementation

False.

Example:

```
1 abstract class Animal
2 {
3     void eat()
4     {
5         System.out.println("Eating");
6     }
7
8     abstract void speak();
9 }
```

Animal provides:

```
1 eat()
```

But leaves:

```
1 speak()
```

for subclasses.

43.13 Partial Reality

Deep Insight

Abstract classes can be partially complete.

Some behaviour implemented.

Some behaviour deferred.

This is why they are so useful.

43.14 The Template Idea

Suppose every employee:

- arrives
- works
- leaves

But:

Work differs.

Professor works differently than Engineer.

An abstract class can define:

Common Structure

while leaving:

Specific Steps

to subclasses.

This idea later becomes:

Template Method Pattern

43.15 Type Relationships

Observe:

```
1 Animal a =  
2 new Dog();
```

Still valid.

Even though:

Animal

cannot be instantiated.

Because Animal remains a type.

43.16 Abstract Classes As Contracts

Normal class:

Type + Implementation

Abstract class:

Type + Partial Implementation + Contract

43.17 When To Use Abstract Classes

Use when:

Classes share:

- common state
- common methods
- common logic

but some behaviour remains specialized.

43.18 Rosetta Rules

Rosetta Rule

Abstract classes represent incomplete concepts.

Rosetta Rule

Abstract classes cannot be instantiated.

Rosetta Rule

Abstract methods declare obligations.

Rosetta Rule

Subclasses must fulfill those obligations.

Rosetta Rule

Abstract classes may contain implementation.

Rosetta Rule

Abstract classes combine:

Contract

+

Partial Implementation

43.19 Mental Compression

Concrete Class:

Type + Implementation

Abstract Class:

Type + Partial Implementation + Contract

Abstract Method:

Chapter 44

Interfaces – Pure Contracts and Capabilities

44.1 The Fourteenth Wrong Idea

Common Trap

Many beginners think:
Interface
=
Abstract Class With No Implementation
Historically close.
Conceptually weak.

44.2 The Real Question

Suppose:

Dog

and

Robot

Question:

Can they belong to the same inheritance hierarchy?

No.

Dog is an Animal.

Robot is not.

Yet both may:

Move

Interesting.

Inheritance fails.

Capability remains.

44.3 The Hidden Limitation Of Inheritance

Inheritance answers:

What are you?

Example:

Dog IS – A Animal

But many real-world questions are:

What can you do?

These are different.

44.4 Capability Thinking

Consider:

- Bird
- Airplane
- Superman
- Drone

Different things.

Yet all can:

Fly

Inheritance cannot express this elegantly.

Interfaces can.

44.5 The First Interface

```
1 interface Flyable
2 {
3     void fly();
4 }
```

Observe:

No state.

No identity.

No implementation requirement.

Only capability.

44.6 Mental Model

Mental Model

Interface

=

Contract

=

Promise

=

Capability

Interface says:

If you claim to be Flyable, you must know how to fly.

It does not say how.

44.7 Implementing An Interface

```
1 class Bird implements Flyable
2 {
3     @Override
4     public void fly()
5     {
6         System.out.println("Flap");
7     }
8 }
```

```
1 class Airplane implements Flyable
2 {
3     @Override
4     public void fly()
5     {
6         System.out.println("Jet Engine");
7     }
8 }
```

Same capability.

Different implementations.

44.8 The Type View

Observe:

```
1 Flyable f;
```

Question:

What object may be stored?

Any object implementing:

Flyable

Examples:

- Bird
- Airplane
- Drone
- Superman

This is polymorphism again.

44.9 A Huge Difference

Abstract Class:

IS-A

Interface:

CAN-DO

This distinction changes everything.

44.10 The Animal Problem

Suppose:

Dog

Can:

Run

Can:

Swim

Can:

Bark

Inheritance would become messy.

Interfaces allow independent capabilities.


```

1 interface Runnable
2 {
3     void run();
4 }
5
6 interface Swimmable
7 {
8     void swim();
9 }

```

```

1 class Dog
2 implements Runnable,
3     Swimmable
4 {
5 }

```

Much cleaner.

44.11 The Multiple Inheritance Problem

C++ allows:

Multiple Inheritance

Example:

```

1 class A
2 {
3 };
4
5 class B
6 {
7 };
8
9 class C : public A,
10         public B
11 {
12 };

```

This creates difficult ambiguities.

Known as:

Diamond Problem

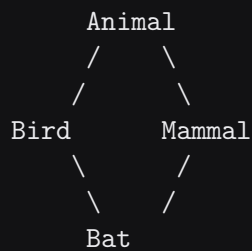
Java avoided this.

Java says:

- One superclass
- Many interfaces

44.12 The Diamond Problem

Imagine:



If both define:

```
1 move()
```

Which version should Bat inherit?

Ambiguity.

Java avoids this with single inheritance.

44.13 Interface Polymorphism

Observe:

```
1 Flyable f =  
2 new Bird();
```

Later:

```
1 f =  
2 new Airplane();
```

Same variable.

Different implementation.

Client code remains unchanged.

44.14 The Deep Design Insight

Clients should depend on:

Capabilities

not

ConcreteClasses

Bad:

```
1 Dog d =  
2 new Dog();
```

Better:

```
1 Runnable r =  
2 new Dog();
```

Now code depends on behaviour.
Not implementation.

44.15 Programming To Interfaces

One of the most important software principles.

Rosetta Rule

Program to abstractions.
Not implementations.

Example:

Bad:

```
1 ArrayList<String> list  
2 =  
3 new ArrayList<>();
```

Better:

```
1 List<String> list  
2 =  
3 new ArrayList<>();
```

Why?

Because client cares about:

List

not

ArrayList

Tomorrow:

```
1 LinkedList
```

can replace it.

44.16 Java 8 Surprise

Historically:

Interfaces contained only:

Abstract Methods

Java 8 introduced:

default

methods.

Example:

```
1 interface Flyable
2 {
3     default void land()
4     {
5         System.out.println("Landing");
6     }
7 }
```

Interfaces gained limited implementation.

Do not confuse this with abstract classes.

Purpose remains:

Capability

44.17 Abstract Class vs Interface

Feature	Abstract Class	Interface
State	Yes	No Instance State
Constructor	Yes	No
Inheritance	Single	Multiple
Capability Focus	Partial	Strong
IS-A	Yes	CAN-DO

44.18 Rosetta Rules

Rosetta Rule

Abstract Classes model identity.

Rosetta Rule

Interfaces model capability.

Rosetta Rule

Inheritance answers:
What are you?

Rosetta Rule

Interfaces answer:
What can you do?

Rosetta Rule

Program to interfaces.
Not implementations.

Rosetta Rule

Depend on contracts.
Not concrete classes.

44.19 Mental Compression

Class:

Reality

Abstract Class:

Partial Reality

Interface:

Pure Capability

Inheritance:

IS-A

Interface:

CAN-DO

Modern Java heavily favors interfaces because capabilities compose better than inheritance hierarchies.

Chapter 45

Composition – The HAS-A Relationship

45.1 The Fifteenth Wrong Idea

Common Trap

Many beginners think:
Inheritance is the primary way to reuse code.
Experienced engineers often think:
Inheritance is the last option.

45.2 The Real Question

Earlier:

Dog IS-A Animal

Inheritance worked.
Now consider:

Car

and

Engine

Question:

Car IS-A Engine?

No.

Question:

Engine IS-A Car?

No.

But:

Car HAS-A Engine

Different relationship.

45.3 The Two Fundamental Relationships

Relationship	Mechanism
IS-A	Inheritance
HAS-A	Composition

45.4 Composition

Deep Insight

Composition means:
Building larger objects from smaller objects.

Example:

Car

contains:

Engine

Wheel

Transmission

45.5 First Example

```
1 class Engine
2 {
3     void start()
4     {
5         System.out.println("Engine Started");
6     }
7 }
```

```
1 class Car
2 {
3     private Engine engine;
4
5     Car()
```



```

6      {
7          engine = new Engine();
8      }
9  }

```

Car is composed of Engine.
Not inherited from Engine.

45.6 Mental Model

Mental Model

A house is not a wall.
A house has walls.
A computer is not a CPU.
A computer has a CPU.
A university is not a professor.
A university has professors.

45.7 Inheritance Creates Dependency

Suppose:

```

1  class Dog extends Animal
2  {
3  }

```

Dog becomes tightly connected to Animal.
Changes in Animal affect Dog.
This can become dangerous.

45.8 The Fragile Base Class Problem

Suppose:

```

1  class Animal
2  {
3      void move()
4      {
5      }
6  }

```

Hundreds of subclasses depend on it.
Changing Animal may unexpectedly break many subclasses.
This is called:

Fragile Base Class

One reason inheritance is risky.

45.9 Composition Reduces Coupling

Observe:

```
1 class Car
2 {
3     private Engine engine;
4 }
```

Car uses Engine.

Car is not Engine.

Relationship is weaker.

This often makes systems easier to evolve.

45.10 Delegation

Suppose:

```
1 class Engine
2 {
3     void start()
4     {
5         System.out.println("Started");
6     }
7 }
```

```
1 class Car
2 {
3     private Engine engine;
4
5     void start()
6     {
7         engine.start();
8     }
9 }
```

Car delegates work.

$Car \rightarrow Engine$

This pattern is called:

Delegation

45.11 The Flying Dog Disaster

Imagine:

```
1 Animal
```

Then:

```

1 Bird
    Then:
1 FlyingBird
    Then:
1 MagicFlyingBird
    Then:
1 UltraMagicFlyingBird

Inheritance hierarchy grows endlessly.
This is a common design failure.

```

45.12 Behavior Composition

Instead of:

```

Animal
|
Bird
|
FlyingBird
|
MagicFlyingBird

```

Think:

Bird

plus

FlyBehaviour

Capability becomes a component.

45.13 The Strategy Insight

Suppose:

- Duck flies
- Eagle flies
- Ostrich does not

Bad approach:
Huge inheritance tree.
Better:

```
1 FlyBehaviour
```

as a separate object.
Objects gain behavior through composition.
This eventually becomes:

Strategy Pattern

45.14 A Deep Principle

Inheritance says:

You Are This

Composition says:

You Have This

The second is often more flexible.

45.15 Runtime Flexibility

Suppose:

```
1 class Car
2 {
3     Engine engine;
4 }
```

Today:

PetrolEngine

Tomorrow:

ElectricMotor

Composition allows replacement.
Inheritance usually cannot change identity at runtime.

45.16 The Famous Rule

Rosetta Rule

Favor Composition Over Inheritance.

Notice:

Not:

Never use inheritance.

Instead:

Prefer composition when possible.

45.17 When Inheritance Is Appropriate

Inheritance works well when:

- true IS-A exists
- substitutability exists
- hierarchy is stable
- abstraction is natural

Example:

Dog IS-A Animal

Reasonable.

45.18 When Composition Is Better

Composition works well when:

- behavior varies
- capabilities vary
- implementation changes
- flexibility is important

Example:

Car HAS-A Engine

Much better than inheritance.

45.19 Real Java Examples

1 ArrayList

uses composition internally.

1 HashMap

uses composition internally.

Most major frameworks rely heavily on:

Composition

not giant inheritance trees.

45.20 Rosetta Rules

Rosetta Rule

IS-A

→

Inheritance

Rosetta Rule

HAS-A

→

Composition

Rosetta Rule

Composition creates flexibility.

Rosetta Rule

Inheritance creates stronger coupling.

Rosetta Rule

Delegation is composition in action.

Rosetta Rule

Favor Composition Over Inheritance.

45.21 Mental Compression

Inheritance:

Identity

Composition:

Assembly

Chapter 46

Reference Types, Object Types and Casting

46.1 The Sixteenth Wrong Idea

Common Trap

Many beginners think:

```
1 Animal a = new Dog();
```

means:

“a is a Dog.”

Not exactly.

46.2 The Hidden Truth

Observe:

```
1 Animal a = new Dog();
```

There are actually two types involved.

Reference Type:

```
1 Animal
```

Object Type:

```
1 Dog
```

Rosetta Rule

Every reference has a type.

Every object has a type.

They need not be identical.

46.3 The Memory Picture

Stack

a
|
|
v

Heap

+-----+
| Dog Object |
+-----+

Reference:

Animal

Actual Object:

Dog

Two different concepts.

46.4 Why This Exists

Suppose:

```
1 Animal a = new Dog();
```

Compiler only knows:

Animal

Runtime knows:

Dog

This separation enables polymorphism.

46.5 What Methods Are Visible?

Suppose:

```
1 class Animal
2 {
3     void eat() {}
4 }
```

```

1 class Dog extends Animal
2 {
3     void bark(){ }
4 }

```

Now:

```

1 Animal a = new Dog();

```

Allowed:

```

1 a.eat();

```

Not Allowed:

```

1 a.bark();

```

Compiler Error.

46.6 Why?

Compiler checks using:

Reference Type

not

Object Type

Reference type defines:

Visible Interface

Object type defines:

Runtime Behaviour

46.7 The Two Rules

Rule 1:

Compiler uses reference type.

Rule 2:

Runtime uses object type.

Memorize forever.

46.8 Method Dispatch Example

```

1 class Animal
2 {
3     void speak()
4     {
5         System.out.println("Animal");
6     }
7 }

```

```

1 class Dog extends Animal
2 {
3     @Override
4     void speak()
5     {
6         System.out.println("Woof");
7     }
8 }

```

```

1 Animal a = new Dog();
2
3 a.speak();

```

Output:

```

1 Woof

```

Compiler:

Animal

Runtime:

Dog

Runtime wins.

46.9 Upcasting

Observe:

```

1 Dog d = new Dog();
2
3 Animal a = d;

```

This conversion is called:

Upcasting

Specific

→

General
Dog

→

Animal
Always safe.
Java performs it automatically.

46.10 Mental Model

Mental Model

Dog contains everything required to behave as Animal.
Therefore:
Dog can always become Animal.

46.11 Downcasting

Now reverse direction.

```
1 Animal a = new Dog();  
2  
3 Dog d = (Dog)a;
```

This is:

Downcasting

General

→

Specific
Animal

→

Dog
Potentially dangerous.

46.12 Why Dangerous?

Suppose:

```
1 Animal a = new Cat();  
2  
3 Dog d = (Dog)a;
```

Compiler accepts.
Runtime throws:

```
1 ClassCastException
```

Because Cat is not Dog.

46.13 The instanceof Operator

Before casting:

```
1 if(a instanceof Dog)
2 {
3     Dog d = (Dog)a;
4 }
```

Safe.

Deep Insight

instanceof asks:
“Is this object really of that type?”

46.14 Modern Java

Java supports:

```
1 if(a instanceof Dog d)
2 {
3     d.bark();
4 }
```

Pattern matching.
Cleaner syntax.

46.15 The Object Class

Now a huge fact.

Every Java class ultimately extends:

```
1 Object
```

Even if you never write it.
Example:

```
1 class Student
2 {
3 }
```

Compiler sees:

```
1 class Student extends Object
2 {
3 }
```

46.16 The Root Of Everything

```
Object
|
+--- Animal
|
+--- Student
|
+--- String
|
+--- Integer
```

Everything eventually derives from Object.

46.17 Why This Matters

Because Object provides methods available everywhere.

Most important:

```
1 toString()
2 equals()
3 hashCode()
4 getClass()
```

46.18 toString()

Default:

```
1 Student@4abf23
```

Usually overridden.

Example:

```
1 @Override
2 public String toString()
3 {
4     return name;
5 }
```

Now:

```
1 System.out.println(student);
```

prints meaningful information.

46.19 The equals() Trap

Suppose:

```

1 String a =
2 new String("ABC");
3
4 String b =
5 new String("ABC");

```

Question:

```

1 a == b

```

?

False.

Why?

Different objects.

46.20 Reference Equality

```

1 ==

```

checks:

Same Object?

Not:

Same Value?

46.21 Logical Equality

```

1 equals()

```

checks:

Equivalent Meaning?

Example:

```

1 a.equals(b)

```

Returns:

```

1 true

```

Because contents match.

46.22 The hashCode () Contract

If:

```
1 a.equals(b)
```

is true

Then:

```
1 a.hashCode()  
2 ==  
3 b.hashCode()
```

must also be true.

This rule powers:

```
1 HashMap  
2 HashSet  
3 Hashtable
```

Extremely important later.

46.23 Rosetta Rules

Rosetta Rule

Reference type determines visible methods.

Rosetta Rule

Object type determines runtime behaviour.

Rosetta Rule

Upcasting is safe.

Rosetta Rule

Downcasting may fail.

Rosetta Rule

instanceof checks actual object type.

Rosetta Rule

Every class ultimately extends Object.

Rosetta Rule

== checks identity.

Rosetta Rule

equals () checks logical equality.

Rosetta Rule

`hashCode ()` must agree with `equals ()`.

Chapter 47

Annotations — Metadata for Programs

47.1 The Forty Ninth Wrong Idea

Common Trap

Many beginners think:
Annotations execute code.
False.
Annotations usually do not execute anything themselves.
They describe information.
Other tools read that information.

47.2 The Strange Symbols

You have seen:

```
1 @Override
```

or

```
1 @Deprecated
```

or

```
1 @Test
```

Question:

What are these?

Answer:

Annotations.

47.3 The Big Idea

Annotation = Metadata

Metadata means: Data About Data

Mental Model

Student Object = Data Annotation = Data About Code

47.4 Visualization

Class + Annotation --- Metadata

47.5 First Example

```
1  @Override
2  public String toString(){
3
4      return "Student";
5
6  }
```

Meaning:
Compiler,
I intend to override a method.
Compiler can verify this.

47.6 Why Useful?

Suppose:

```
1  @Override
2  public String toString(){
3
4  }
```

Notice:
toString
instead of:
toString
Compiler reports error.
Annotation prevented bug.

47.7 Built-In Annotations

Common examples:

- @Override
- @Deprecated
- @SuppressWarnings
- @FunctionalInterface

47.8 @Deprecated

Example:

```
1  @Deprecated
2  public void oldMethod(){
3
4  }
```

Meaning:
Avoid using this.
May disappear later.

47.9 @SuppressWarnings

Observe:

```
1 @SuppressWarnings("unchecked")
```

Tells compiler:
I understand this warning.
Do not show it.

47.10 @FunctionalInterface

Observe:

```
1 @FunctionalInterface
2 interface Calculator{
3     int add(int a,int b);
4 }
```

Compiler verifies:
Exactly one abstract method.

47.11 Where Can Annotations Be Applied?

Classes

Methods

Fields

Constructors

Parameters

Packages

47.12 Custom Annotations

Question:

Can we create our own?

Yes.

Example:

```
1 @interface Author{
2
3 }
```

Creates annotation type.

47.13 Using Custom Annotation

```
1 @Author
2 class Student{
3
4 }
```

47.14 Annotations With Values

Observe:

```
1 @interface Author{
2     String name();
3 }
```

Usage:

```
1 @Author(name = "PS")
2 class Student{
3
4 }
5
```

47.15 Default Values

```
1 @interface Version{
2
3     int value() default 1;
4
5 }
```

Usage:

```
1 @Version
2 class Student{
3
4 }
```

Value automatically becomes:

```
1 1
```

47.16 Multiple Elements

```
1 @interface Info{
2
3     String author();
4     int version();
5
6 }
```

Usage:

```
1 @Info(  
2   author="PS",  
3   version=1  
4 )
```

47.17 Annotation Retention

Important concept.

Question:

How long should annotation live?

47.18 Retention Policies

Three major policies: SOURCE CLASS RUNTIME

47.19 SOURCE

Exists only in source code.

Removed after compilation.

47.20 CLASS

Stored in class file.

May not be available at runtime.

47.21 RUNTIME

Available during execution.

Reflection can read it.

Most framework annotations use this.

47.22 Example

```
1 @Retention(  
2   RetentionPolicy.RUNTIME  
3 )  
4 @interface Author{  
5  
6 }
```

47.23 Target

Question:

Where may annotation be used?

Answer:

@Target

47.24 Example

```
1 @Target(  
2 ElementType.METHOD  
3 )
```

Only methods allowed.

47.25 Reflection + Annotation

The real power begins here.

Suppose:

```
1 @Author(name="PS")  
2 class Student{  
3  
4 }
```

Reflection can inspect:

Annotation exists?

What values?

47.26 Framework Magic

Consider:

```
1 @Entity  
2 class Student{  
3  
4 }
```

Hibernate reads annotation.

Then decides:

Store in database table.

47.27 Another Example

```
1 @Test  
2 public void addTest(){  
3  
4 }
```

JUnit discovers method.

Runs test automatically.

47.28 The Deep Insight

Annotations do not usually contain behavior.

Annotations contain metadata.

Frameworks provide behavior.

Deep Insight

Reflection + Annotations = Framework Foundation
This combination powers much of modern Java.

47.29 Rosetta Rules

Rosetta Rule

Annotation = Metadata.

Rosetta Rule

Metadata = Data About Code.

Rosetta Rule

Annotations do not normally execute code.

Rosetta Rule

Reflection can read annotations.

Rosetta Rule

@Retention controls lifetime.

Rosetta Rule

@Target controls location.

Rosetta Rule

Custom annotations can be created.

Rosetta Rule

Modern frameworks heavily rely on annotations.

Mental Compression

Annotation $\Rightarrow$ Metadata

Reflection $\Rightarrow$ Read Metadata

@Target $\Rightarrow$ Where Annotation Can Appear

@Retention $\Rightarrow$ How Long Annotation Lives

Reflection + Annotations $\Rightarrow$ Framework Magic

Chapter 48

Design Patterns – The Evolution of OOP

48.1 The Fifty First Wrong Idea

Common Trap

Many beginners think:
Design Patterns
=
Advanced Java Features.
False.
Patterns are not features.
Patterns are not syntax.
Patterns are not keywords.
You can write every design pattern in:
Java
C++
Python
Rust
C#
Go
because patterns are ideas.
Not language constructs.

48.2 The Bigger Question

Before learning patterns we must ask:

Why were they invented?

If OOP already gives us:

Classes

Objects

Inheritance

Polymorphism

Interfaces

then why wasn't that enough?

Why did experienced engineers still feel something was missing?

48.3 The History Nobody Teaches

During the 1980s and early 1990s large object oriented systems were being built everywhere.

Banks.

Operating systems.

Telecommunication software.

Military software.

GUI applications.

At first everything looked wonderful.

OOP appeared to solve complexity.

Then systems became larger.

Much larger.

Millions of lines.

Thousands of classes.

And new problems appeared.

48.4 The First Disease

```
1 class StudentSystem
2 {
3     void saveStudent() {}
4     void sendEmail() {}
5     void generateReport() {}
6     void printCertificate() {}
7     void calculateGrades() {}
8     void processPayment() {}
9 }
```

Initially this feels convenient.

Everything is in one place.

Then requirements change.

A new report format arrives.

Email logic changes.

Payment gateway changes.

Now one class keeps changing.

Soon it becomes dangerous.

48.5 The Second Disease

Suppose we write:

```
1 if(type.equals("DOG"))
2 {
3 }
4 else if(type.equals("CAT"))
5 {
6 }
7 else if(type.equals("BIRD"))
8 {
9 }
10 else if(type.equals("FISH"))
11 {
12 }
```

Seems harmless.
Six months later:
20 animals.
50 animals.
100 animals.
Now every modification touches existing code.
Bugs begin appearing.

48.6 The Third Disease

Suppose:

```
1 Car
2 |
3 Engine
4 |
5 FuelPump
6 |
7 Sensor
8 |
9 Battery
```

Every component knows about every other component.
This is called:
Tight Coupling
Changing one part causes changes everywhere.

48.7 The Real Problem

All these diseases are symptoms.
The root cause is:
Managing Change.
Software constantly changes.
Design Patterns emerged as survival mechanisms.

Mental Model

Algorithms help manage computational complexity.
Design Patterns help manage structural complexity.

48.8 What Exactly Is A Pattern?

A pattern is:
A reusable solution
to
a recurring design problem.
Notice carefully:
Reusable Solution
Recurring Problem
Patterns exist because the same problems keep appearing in software.

48.9 Pattern Discovery

Nobody sat down and invented all patterns at once.

Instead:

Engineers repeatedly solved the same design problems.

Certain solutions worked.

Those solutions became patterns.

48.10 The Compression Effect

Imagine saying:

"This object should notify many listeners whenever its state changes."

or simply:

Observer Pattern.

One phrase compresses pages of explanation.

Deep Insight

Design Patterns are a vocabulary.

They allow engineers to communicate large designs using small words.

48.11 The Three Kingdoms

Almost all classic patterns belong to one of three groups.

48.12 The First Kingdom – Creational Patterns

Common Trap

Many beginners think:

Object creation is trivial.

Just write:

```
1 new Student();
```

and move on.

This is true for small programs.

Large systems discover a hidden problem.

48.13 The Question Nobody Asks

What exactly happens when we write:

```
1 Student s = new Student();
```

The object is created.

Simple.

But now imagine:

A bank.

An operating system.

An e-commerce website.
A game engine.
Thousands of objects are created.
Some must be shared.
Some must be configured.
Some must be chosen dynamically.
Some are expensive to create.
Now creation becomes a design problem.

48.14 The Hidden Cost Of new

Observe:

```
1 Dog d = new Dog();
```

This line contains more information than most beginners realize.
It says:
Create Object
Use Dog Specifically
Use Dog Constructor
Depend On Dog Class
The caller now knows everything.

Mental Model

The keyword:

```
1 new
```

creates coupling.
The caller becomes connected to a specific implementation.

48.15 A Small Example

Suppose:

```
1 class Dog implements Animal
2 {
3 }
```

and:

```
1 Animal a = new Dog();
```

Tomorrow management says:
Replace Dog
with
RobotDog.
Every location using:

```
1 new Dog()
```

must be found.
and changed.
This is the beginning of creation complexity.

48.16 The Goal

Creational patterns attempt to answer:

Who should create objects?
When should objects be created?
How many should exist?
How should they be configured?

48.17 The Family Tree

---Creational Patterns

Singleton

Factory Method

Abstract Factory

Builder

Prototype

48.18 The Simplest Pattern

The first pattern ever encountered by most programmers is:

Singleton
But before seeing the solution, we need the disease.

48.19 The Problem

Suppose we are building a logging system.

Every class wants access to the logger.

Naive approach:

```
1 Logger a = new Logger();  
2  
3 Logger b = new Logger();  
4  
5 Logger c = new Logger();
```

Three logger objects.
Do we actually need three?
Usually no.
One shared logger is enough.

48.20 The Core Idea

Question:

Can we guarantee that only one object ever exists?
This question gave birth to the Singleton Pattern.

Deep Insight

Most patterns begin this way.
Not with code.
With a design pressure.
A repeated problem creates a repeated solution.
Repeated solutions become patterns.

48.21 Rosetta Rule

Rosetta Rule

Patterns are not invented first.
Problems come first.
Patterns emerge afterward.

48.22 Pattern #1 – Singleton

The Singleton pattern is often the first design pattern learned.
Unfortunately it is also one of the most misunderstood.
Most explanations begin with:
Only One Object.
That is true.
But it does not explain why.

48.23 The Disease

Suppose we are building:

- A Logger
- A Configuration Manager
- A Database Connection Pool
- A System Registry

These components represent shared system resources.
Creating many copies may lead to:
Wasted memory
Inconsistent state
Conflicting behavior

48.24 The Naive Design

```
1  Logger log1 = new Logger();  
2  
3  Logger log2 = new Logger();
```

```
4
5 Logger log3 = new Logger();
```

Question:

Are these the same logger?

No.

Three completely different objects.

48.25 The Hidden Problem

Suppose:

```
1 log1.setLevel("DEBUG");
```

and:

```
1 log2.setLevel("ERROR");
```

Now the system contains multiple conflicting configurations.

Which one is correct?

Nobody knows.

48.26 The Desired Property

We want:

Exactly One Instance.

Not:

One Per Class

Not:

One Per Method

Not:

One Per User

Exactly One.

48.27 First Attempt

Suppose we write:

```
1 class Logger
2 {
3 }
```

Nothing prevents:

```
1 new Logger();
2 new Logger();
3 new Logger();
```

We need control.

48.28 The Key Observation

Who creates objects?

Constructors.

Therefore:

If we control the constructor,
we control object creation.

48.29 The First Barrier

```
1 class Logger
2 {
3     private Logger()
4     {
5     }
6 }
```

Now:

```
1 new Logger();
```

fails.

Nobody outside the class can call the constructor.

48.30 The New Problem

Excellent.

Now nobody can create objects.

Including us.

We solved one problem and created another.

48.31 The Solution

Let the class create its own instance.

```
1 public class Logger
2 {
3     private static final
4     Logger INSTANCE =
5         new Logger();
6
7     private Logger()
8     {
9     }
10 }
```

Notice carefully.

The class creates itself.

48.32 How Do Users Access It?

We need an access point.

```
1 public static Logger getInstance()  
2 {  
3     return INSTANCE;  
4 }
```

Usage:

```
1 Logger log =  
2 Logger.getInstance();
```

48.33 Visualization

User

|

getInstance()

|

Logger

|

INSTANCE

Everyone receives the same object.

48.34 Proof

```
1 Logger a =  
2 Logger.getInstance();  
3  
4 Logger b =  
5 Logger.getInstance();
```

Question:

Are a and b different?

No.

Both references point to:

INSTANCE

48.35 Mental Model

Mental Model

Normal Class

↓

Many Constructors Calls

↓

Many Objects

Singleton

↓

One Internal Object

↓

Shared Everywhere

48.36 Why It Became Popular

Singleton appears useful for:

- Logging
- Configuration
- Caches
- Registries
- System Managers

48.37 The Trap Nobody Mentions

Common Trap

Many developers started using Singleton everywhere.
Soon entire applications became collections of global objects.
This created new problems.
Hidden dependencies.
Difficult testing.
Tight coupling.

48.38 A Deeper Perspective

Singleton solves:

- Instance Control.

It does NOT solve:

- Architecture.

Many beginners confuse these.

48.39 The Singleton Question

Before using Singleton ask:

- Does the system truly require exactly one instance?

- Or is it merely convenient?

This question prevents misuse.

48.40 Rosetta Rules

Rosetta Rule

Singleton exists to control instance count.

Rosetta Rule

Private constructor prevents external creation.

Rosetta Rule

A static instance stores the single shared object.

Rosetta Rule

`getInstance()` provides access.

Rosetta Rule

Singleton solves a design problem, not an architectural one.

Rosetta Rule

Not every shared object should become a Singleton.

Mental Compression

Problem

⇒

Too Many Objects
Need

⇒

Exactly One Instance
Method

⇒

Private Constructor
+
Static Instance
+
getInstance ()
Result

⇒

Controlled Creation

48.41 Pattern #2 – Factory Method

Most beginners believe:

Factory

=

A fancy wrapper around

```
1 new
```

This is not the real idea.

Factory exists because object creation creates coupling.

48.42 The Disease

Consider:

```
1 Animal animal =  
2 new Dog();
```

Looks innocent.

But this line reveals something.

The caller knows:
The object is an Animal.
The concrete class is Dog.
The Dog constructor exists.
How Dog is created.
The caller knows too much.

48.43 The Hidden Dependency

Suppose management says:
Replace Dog
with
RobotDog.
Now every location containing:

```
1 new Dog()
```

must change.
Large systems may contain thousands of such locations.

48.44 The Real Question

Can users request objects without knowing how those objects are created?
This question creates the Factory Pattern.

48.45 The First Attempt

Suppose we centralize creation.

```
1 public class AnimalFactory
2 {
3     public Animal create()
4     {
5         return new Dog();
6     }
7 }
```

Usage:

```
1 AnimalFactory factory =
2 new AnimalFactory();
3
4 Animal animal =
5 factory.create();
```

Notice what disappeared.
The caller no longer knows about Dog.

48.46 Visualization

Client

|

Factory

|

Dog

Client and Dog are no longer directly connected.

48.47 The Core Insight

Factory does not eliminate object creation.

It relocates object creation.

Mental Model

Factory

=

Creation Specialist

Client

=

Object User

48.48 The Bigger Problem

Suppose we support:

Dog

Cat

Bird

Fish

How should the factory decide?

48.49 Parameterized Factory

```
1 public class AnimalFactory
2 {
3     public Animal create(
4         String type)
5     {
6         if(type.equals("DOG"))
7         {
8             return new Dog();
9         }
10
11         if(type.equals("CAT"))
```

```

12         {
13             return new Cat();
14         }
15
16         return null;
17     }
18 }

```

Usage:

```

1 Animal animal =
2 factory.create("DOG");

```

48.50 The Improvement

Client knows:

Animal

Factory

Client does NOT know:

Dog Constructor

Cat Constructor

Bird Constructor

Creation logic.

48.51 A Familiar Example

Think about:

```

1 DriverManager
2 .getConnection(...)

```

Question:

Do you manually create database connection objects?

Usually no.

Factory logic creates them.

48.52 Factory And OCP

Factory begins helping the:

Open Closed Principle.

Because object creation becomes centralized.

Changes occur in one place.

Not everywhere.

48.53 The Factory Method Pattern

The previous example was a simple factory.

Factory Method goes further.

Instead of one factory deciding everything,

subclasses decide creation.

48.54 The Idea

Abstract Creator:

```
1 abstract class AnimalCreator
2 {
3     abstract Animal create();
4 }
```

Concrete Creator:

```
1 class DogCreator
2 extends AnimalCreator
3 {
4     Animal create()
5     {
6         return new Dog();
7     }
8 }
```

Another Creator:

```
1 class CatCreator
2 extends AnimalCreator
3 {
4     Animal create()
5     {
6         return new Cat();
7     }
8 }
```

48.55 Why Useful?

Now creation logic becomes extensible.

New animal?

Create new creator.

Existing code remains untouched.

48.56 Mental Model

Mental Model

Simple Factory

↓

One Central Creator

Factory Method

↓

Many Specialized Creators

48.57 The Deep Insight

The keyword:

1 `new`

creates dependency.

Factory patterns attempt to move that dependency away from business logic.

This is the beginning of:

Loose Coupling

which becomes one of the central ideas in software architecture.

Deep Insight

Most patterns are really attempts to control coupling.

Factory is one of the first.

48.58 When NOT To Use Factory

Common Trap

Not every object needs a factory.

If creation is trivial:

```
1 Student s =  
2 new Student();
```

A factory may add unnecessary complexity.

Patterns should remove complexity.

Not create it.

48.59 Rosetta Rules

Rosetta Rule

Factory centralizes object creation.

Rosetta Rule

Factory reduces knowledge of concrete implementations.

Rosetta Rule

Factory relocates creation logic.

Rosetta Rule

Factory helps reduce coupling.

Rosetta Rule

Factory Method delegates creation to subclasses.

Rosetta Rule

Patterns should simplify design, not complicate it.

Mental Compression

Problem

⇒

Object Creation Creates Coupling
Need

⇒

Hide Creation Details
Solution

⇒

Factory
Advanced Version

⇒

Factory Method
Result

⇒

Loose Coupling

48.60 Pattern #3 – Abstract Factory

After learning Factory Method, many students believe:

Problem solved.

We can create objects without directly using:

1 new

Unfortunately large systems introduce a new problem.

48.61 The New Disease

Suppose we are building a GUI.

The application must support:

Windows

macOS

Linux

Each platform requires:

Buttons
Checkboxes
Menus
Text Fields
Notice carefully.
We are not creating one object.
We are creating a family of related objects.

48.62 The Family Problem

Windows requires:

```
1 WindowsButton
2 WindowsCheckbox
3 WindowsMenu
```

Mac requires:

```
1 MacButton
2 MacCheckbox
3 MacMenu
```

Question:

How do we guarantee that a Windows button never appears with a Mac checkbox?

48.63 The Naive Solution

```
1 Button b =
2   new WindowsButton();
3
4 Checkbox c =
5   new MacCheckbox();
```

The UI now looks inconsistent.
Parts belong to different families.

48.64 The Real Requirement

We do not want:

One Factory

We want:

One Factory Family.

48.65 Mental Model

Mental Model

Factory

↓

Creates One Product

Abstract Factory

↓

Creates Related Products

A Complete Family

48.66 The Interfaces

First define product interfaces.

```
1 interface Button
2 {
3     void paint();
4 }
```

```
1 interface Checkbox
2 {
3     void check();
4 }
```

48.67 Concrete Products

Windows family:

```
1 class WindowsButton
2 implements Button
3 {
4 }
```

```
1 class WindowsCheckbox
2 implements Checkbox
3 {
4 }
```

Mac family:

```
1 class MacButton
2 implements Button
3 {
4 }
```

```
1 class MacCheckbox
2 implements Checkbox
3 {
4 }
```

48.68 The Abstract Factory

Now define a factory interface.

```
1 interface GUIFactory
2 {
3     Button createButton();
4
5     Checkbox createCheckbox();
6 }
```

Notice what happened.
The factory creates an entire family.
Not just one object.

48.69 Windows Factory

```
1 class WindowsFactory
2 implements GUIFactory
3 {
4     public Button createButton()
5     {
6         return new WindowsButton();
7     }
8
9     public Checkbox createCheckbox()
10    {
11        return new WindowsCheckbox();
12    }
13 }
```

48.70 Mac Factory

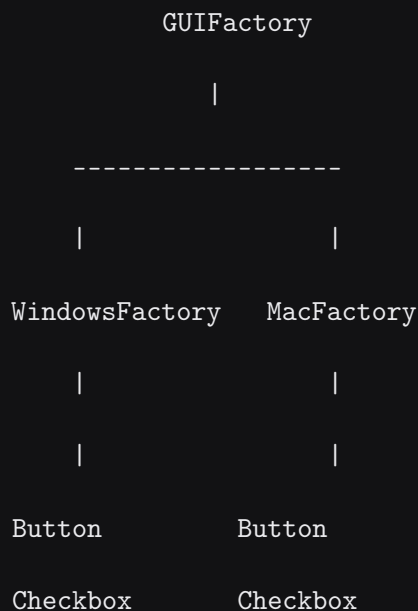
```
1 class MacFactory
2 implements GUIFactory
3 {
4     public Button createButton()
5     {
6         return new MacButton();
7     }
8
9     public Checkbox createCheckbox()
10    {
11        return new MacCheckbox();
12    }
13 }
```


48.71 Client Code

```
1 GUIFactory factory =  
2 new WindowsFactory();  
3  
4 Button b =  
5 factory.createButton();  
6  
7 Checkbox c =  
8 factory.createCheckbox();
```

Observe carefully.
Client never mentions:
WindowsButton
WindowsCheckbox

48.72 Visualization



Each factory produces an entire consistent ecosystem.

48.73 Why Frameworks Love It

Frameworks often support:

- Different Databases
- Different Operating Systems
- Different UI Themes
- Different Hardware Vendors

Abstract Factory allows the framework to switch families without changing client code.

48.74 Connection To Real Life

Imagine ordering furniture.

Modern Style Package
Classic Style Package
Luxury Style Package
Each package contains:
Chair
Table
Sofa
You choose a package.
Everything matches.
That package is an Abstract Factory.

48.75 The Deep Insight

Factory hides:
Which Object
Abstract Factory hides:
Which Family Of Objects
This is a much stronger form of abstraction.

48.76 When NOT To Use

Common Trap

If your system creates only one type of object,
Abstract Factory may be over-engineering.
It becomes useful only when multiple related products must evolve together.

48.77 Rosetta Rules

Rosetta Rule

Factory creates products.

Rosetta Rule

Abstract Factory creates families of products.

Rosetta Rule

Related objects are kept consistent.

Rosetta Rule

Clients depend on interfaces, not concrete families.

Rosetta Rule

Changing families should not require changing client code.

48.78 Mental Compression

Mental Compression

Factory

⇒

Creates One Product

Abstract Factory

⇒

Creates Product Families

Need

⇒

Consistency Across Related Objects

Result

⇒

Whole Systems Become Interchangeable

48.79 Pattern #4 – Builder

After Factory and Abstract Factory, many developers believe:
Object creation is solved.
Unfortunately another problem appears in large systems.
Some objects are difficult to build.

48.80 The Disease

Suppose we are creating a Student.
Simple:

```
1 Student s =  
2 new Student();
```

No problem.
Now imagine an Enterprise Server.
It requires:
Host
Port
Username
Password

Database
Cache
Security Settings
Timeout Settings
Logging Settings
Monitoring Settings
Suddenly construction becomes a problem.

48.81 Constructor Explosion

Naive approach:

```
1 public Server(  
2     String host,  
3     int port,  
4     String user,  
5     String password,  
6     int timeout,  
7     boolean logging,  
8     boolean monitoring  
9 )  
10 {  
11 }
```

Looks ugly.
Now add more parameters.
Even worse.

48.82 The Bigger Problem

Consider:

```
1 new Server(  
2     "localhost",  
3     5432,  
4     "admin",  
5     "secret",  
6     5000,  
7     true,  
8     false  
9 );
```

Question:
What does:

```
1 true
```

mean?
Nobody knows.
What does:

```
1 false
```

mean?
Nobody knows.
Readability disappears.

48.83 The Real Requirement

We want:
Flexible Construction
Readable Construction
Step-by-Step Construction
Without giant constructors.
This creates the Builder Pattern.

48.84 Mental Model

Mental Model

Factory

↓

Answers:

"What object?"

Builder

↓

Answers:

"How should it be assembled?"

48.85 The Core Idea

Instead of:
Building the object in one step
We build it gradually.
Like constructing a house.
Foundation.
Walls.
Roof.
Interior.
One step at a time.

48.86 Builder Class

```
1 public class ServerBuilder
2 {
3     private String host;
4     private int port;
5     private boolean logging;
6
7     public ServerBuilder host(
8         String host)
9     {
```

```

10         this.host = host;
11         return this;
12     }
13
14     public ServerBuilder port(
15         int port)
16     {
17         this.port = port;
18         return this;
19     }
20
21     public ServerBuilder logging(
22         boolean logging)
23     {
24         this.logging = logging;
25         return this;
26     }
27 }

```

Each method configures one piece of the object.

48.87 The Build Step

Eventually:

```

1 public Server build()
2 {
3     return new Server(
4         host,
5         port,
6         logging
7     );
8 }

```

Now construction becomes explicit.

48.88 Usage

```

1 Server server =
2 new ServerBuilder()
3
4 .host("localhost")
5
6 .port(5432)
7
8 .logging(true)
9
10 .build();

```

Observe how readable this becomes.
Each configuration step is visible.

48.89 Visualization

ServerBuilder

|

host()

|

port()

|

logging()

|

build()

|

Server

48.90 Why This Feels Natural

Humans often build things incrementally.

We don't describe an entire house in one sentence.

We describe it piece by piece.

Builder mirrors that process.

48.91 Immutable Objects

Builder became extremely popular with immutable classes.

Why?

Because immutable objects must be fully initialized before creation.

Builder makes that easy.

48.92 A Famous Example

Modern Java frameworks use:

Builder

everywhere.

Examples:

StringBuilder

HttpRequest Builder

Lombok Builder

Spring Builders

Configuration Builders

48.93 The Deep Insight

Factory separates:

- Creation

- from

- Usage

Builder separates:

- Construction

- from

- Representation

This is a deeper idea.

The object and the process used to build it become independent.

48.94 The Director (Classic Builder)

The original GoF Builder contains another participant.

Director.

Director

|

Builder Interface

|

Concrete Builder

|

Product

The Director controls construction order.

Modern Java often skips this extra layer.

48.95 When NOT To Use Builder

Common Trap

If an object contains:

- Two fields

- Three fields

- Simple initialization

Builder may be unnecessary.

Patterns should reduce complexity.

Not create it.

48.96 Builder vs Factory

Mental Model

Factory

↓

Which object?

Builder

↓

How should the object be assembled?

48.97 Rosetta Rules

Rosetta Rule

Builder solves construction complexity.

Rosetta Rule

Builder replaces constructor explosion.

Rosetta Rule

Builder creates objects step by step.

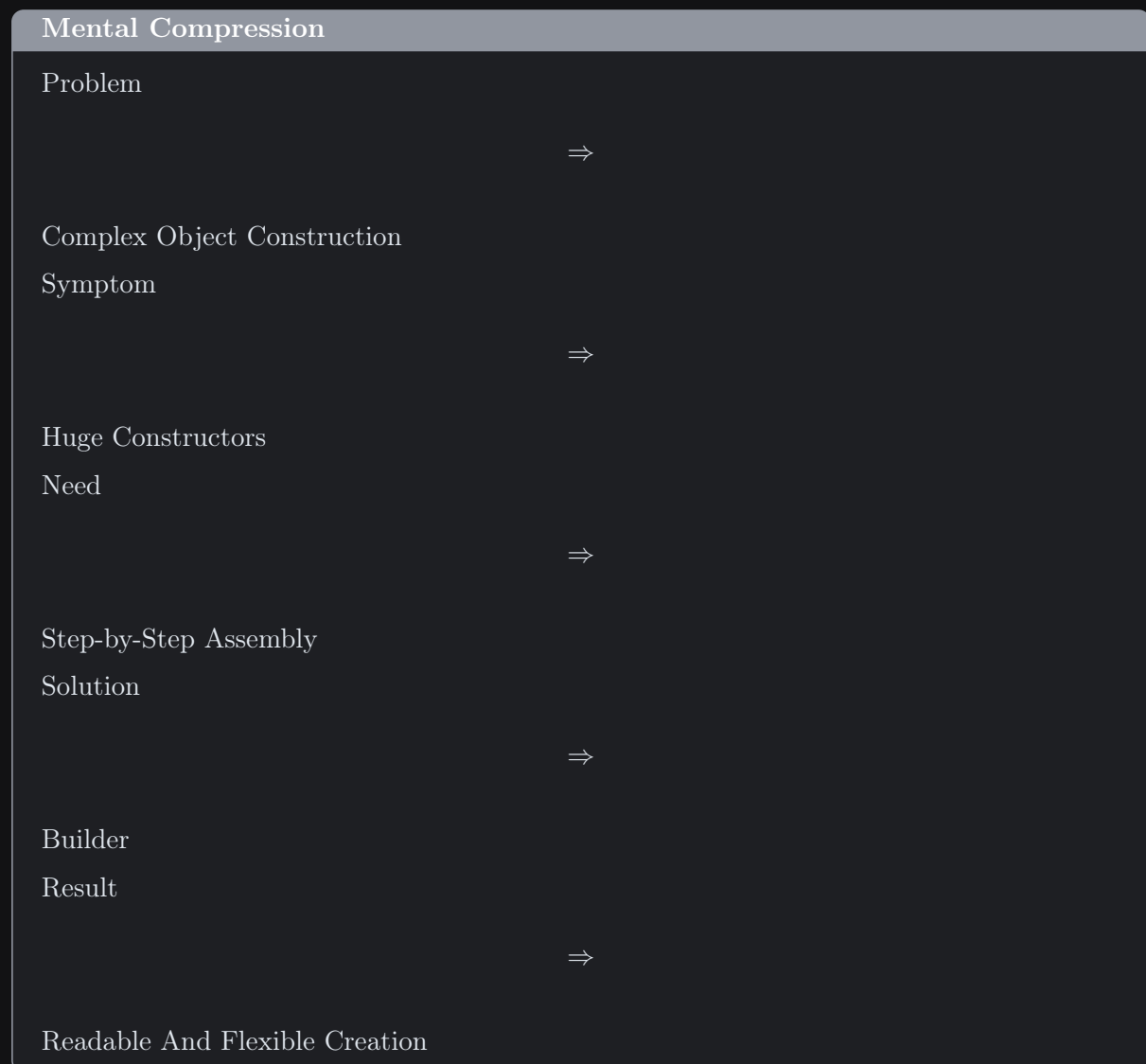
Rosetta Rule

Builder improves readability.

Rosetta Rule

Builder works especially well with immutable objects.

48.98 Mental Compression



48.99 Pattern #5 – Prototype

After Builder, a new question appears.
What if creating an object is expensive?
Or difficult?
Or requires huge configuration?
Must we build it again from scratch?

48.100 The Disease

Imagine creating:
A Complex Game Character
A Large Configuration Object
A Huge Document Template
A CAD Model
A GUI Layout

Construction may require:
Hundreds of fields
Thousands of calculations
Complex initialization
Building repeatedly becomes expensive.

48.101 The Real Question

Suppose we already have a perfectly configured object.
Why rebuild it?
Why not copy it?
This idea creates the Prototype Pattern.

48.102 Mental Model

Mental Model

Factory
↓
Create New
Builder
↓
Assemble New
Prototype
↓
Copy Existing

48.103 Real Life Example

Suppose an architect designs:
Apartment Layout A
The next apartment is almost identical.
Would the architect start from an empty page?
Usually no.
They duplicate the blueprint.
Then make small modifications.
That duplication is the essence of Prototype.

48.104 The Traditional Way

Without Prototype:

```
1 Character hero =  
2 new Character();  
3  
4 hero.setName("Warrior");  
5  
6 hero.setHealth(100);  
7  
8 hero.setStrength(50);
```

```

9
10 hero.setArmor(30);
11
12 hero.setWeapon("Sword");

```

Creating every character requires repeating setup.

48.105 The Prototype Idea

Create one fully configured object.

Then duplicate it.

```

1 Character warrior =
2   new Character();
3
4 warrior.setHealth(100);
5
6 warrior.setStrength(50);
7
8 warrior.setArmor(30);

```

Later:

```

1 Character anotherWarrior =
2   warrior.clone();

```

Instant copy.

48.106 Visualization

Original Object

|

Clone

|

New Object

Notice carefully.

Construction disappears.

Copying replaces construction.

48.107 The Prototype Interface

A simple version:

```

1 interface Prototype
2 {
3     Prototype clone();
4 }

```

Every object knows how to duplicate itself.

48.108 Concrete Prototype

```
1 class Character
2 implements Prototype
3 {
4     private int health;
5
6     public Character clone()
7     {
8         Character copy =
9             new Character();
10
11         copy.health =
12             this.health;
13
14         return copy;
15     }
16 }
```

The object performs its own duplication.

48.109 The Hidden Difficulty

Copying sounds easy.

Unfortunately objects often contain references.

Example:

```
1 class Student
2 {
3     Address address;
4 }
```

Now cloning becomes tricky.

48.110 Shallow Copy

Suppose:

```
1 Student s1 =
2 new Student();
3
4 Student s2 =
5 s1.clone();
```

Question:

Does s2 receive a new Address?

Not necessarily.

Often both objects reference the same Address.

48.111 Visualization

Student A -----> Address

Student B -----^

Two students.
One shared address.
This is:
Shallow Copy.

48.112 The Problem

Suppose:

```
1 s2.address.city =  
2 "Delhi";
```

Now:
s1.address.city
also changes.
Unexpected behavior.

48.113 Deep Copy

Instead:

```
1 copy.address =  
2 new Address(  
3     original.address  
4 );
```

Now each object receives its own Address object.

48.114 Visualization

Student A ---> Address A

Student B ---> Address B

Completely independent copies.

48.115 Java's Cloneable

Java historically introduced:

```
1 Cloneable
```

and:

```
1 Object.clone()
```

However many experienced Java developers avoid it today.
Why?
Because clone() has numerous design problems.
It is easy to misuse.
Deep copying becomes difficult.

48.116 Modern Java Practice

Instead of:

```
1 clone()
```

many systems prefer:
Copy Constructors
Factory Methods
Serialization-Based Copying
Custom Copy Logic

48.117 The Deep Insight

Factory creates objects.
Builder assembles objects.
Prototype duplicates objects.
The source of creation changes.

Deep Insight

Prototype treats an existing object as a template.
Future objects are derived from that template.

48.118 Where Prototype Appears

Game Engines
Graphic Editors
Object Templates
Document Systems
Configuration Systems
Caching Frameworks

48.119 When NOT To Use

Common Trap

If object creation is cheap,
Prototype may add unnecessary complexity.
Copying logic itself can become more complicated than creation.

48.120 Rosetta Rules

Rosetta Rule

Prototype creates objects through copying.

Rosetta Rule

Prototype reduces expensive reconstruction.

Rosetta Rule

Shallow copy shares references.

Rosetta Rule

Deep copy duplicates references.

Rosetta Rule

Modern Java often avoids direct use of Cloneable.

48.121 Mental Compression

Mental Compression	
Problem	
	⇒
Object Creation Is Expensive	
Need	
	⇒
Reuse Existing Configuration	
Method	
	⇒
Clone Existing Object	
Challenge	
	⇒
Shallow vs Deep Copy	
Result	
	⇒
Fast Object Duplication	

48.122 Pattern #5 – Prototype

After Builder, a new question appears.
What if creating an object is expensive?
Or difficult?
Or requires huge configuration?
Must we build it again from scratch?

48.123 The Disease

Imagine creating:
A Complex Game Character
A Large Configuration Object
A Huge Document Template
A CAD Model
A GUI Layout

Construction may require:
Hundreds of fields
Thousands of calculations
Complex initialization
Building repeatedly becomes expensive.

48.124 The Real Question

Suppose we already have a perfectly configured object.
Why rebuild it?
Why not copy it?
This idea creates the Prototype Pattern.

48.125 Mental Model

Mental Model

Factory
↓
Create New
Builder
↓
Assemble New
Prototype
↓
Copy Existing

48.126 Real Life Example

Suppose an architect designs:
Apartment Layout A
The next apartment is almost identical.
Would the architect start from an empty page?
Usually no.
They duplicate the blueprint.
Then make small modifications.
That duplication is the essence of Prototype.

48.127 The Traditional Way

Without Prototype:

```
1 Character hero =  
2 new Character();  
3  
4 hero.setName("Warrior");  
5  
6 hero.setHealth(100);  
7  
8 hero.setStrength(50);
```

```

9
10 hero.setArmor(30);
11
12 hero.setWeapon("Sword");

```

Creating every character requires repeating setup.

48.128 The Prototype Idea

Create one fully configured object.

Then duplicate it.

```

1 Character warrior =
2   new Character();
3
4   warrior.setHealth(100);
5
6   warrior.setStrength(50);
7
8   warrior.setArmor(30);

```

Later:

```

1 Character anotherWarrior =
2   warrior.clone();

```

Instant copy.

48.129 Visualization

Original Object

|

Clone

|

New Object

Notice carefully.

Construction disappears.

Copying replaces construction.

48.130 The Prototype Interface

A simple version:

```

1 interface Prototype
2 {
3     Prototype clone();
4 }

```

Every object knows how to duplicate itself.

48.131 Concrete Prototype

```
1 class Character
2 implements Prototype
3 {
4     private int health;
5
6     public Character clone()
7     {
8         Character copy =
9             new Character();
10
11         copy.health =
12             this.health;
13
14         return copy;
15     }
16 }
```

The object performs its own duplication.

48.132 The Hidden Difficulty

Copying sounds easy.

Unfortunately objects often contain references.

Example:

```
1 class Student
2 {
3     Address address;
4 }
```

Now cloning becomes tricky.

48.133 Shallow Copy

Suppose:

```
1 Student s1 =
2 new Student();
3
4 Student s2 =
5 s1.clone();
```

Question:

Does s2 receive a new Address?

Not necessarily.

Often both objects reference the same Address.

48.134 Visualization

Student A -----> Address

Student B -----^

Two students.
One shared address.
This is:
Shallow Copy.

48.135 The Problem

Suppose:

```
1 s2.address.city =  
2 "Delhi";
```

Now:
s1.address.city
also changes.
Unexpected behavior.

48.136 Deep Copy

Instead:

```
1 copy.address =  
2 new Address(  
3     original.address  
4 );
```

Now each object receives its own Address object.

48.137 Visualization

Student A ---> Address A

Student B ---> Address B

Completely independent copies.

48.138 Java's Cloneable

Java historically introduced:

```
1 Cloneable
```

and:

```
1 Object.clone()
```

However many experienced Java developers avoid it today.
Why?
Because clone() has numerous design problems.
It is easy to misuse.
Deep copying becomes difficult.

48.139 Modern Java Practice

Instead of:

```
1 clone()
```

many systems prefer:
Copy Constructors
Factory Methods
Serialization-Based Copying
Custom Copy Logic

48.140 The Deep Insight

Factory creates objects.
Builder assembles objects.
Prototype duplicates objects.
The source of creation changes.

Deep Insight

Prototype treats an existing object as a template.
Future objects are derived from that template.

48.141 Where Prototype Appears

Game Engines
Graphic Editors
Object Templates
Document Systems
Configuration Systems
Caching Frameworks

48.142 When NOT To Use

Common Trap

If object creation is cheap,
Prototype may add unnecessary complexity.
Copying logic itself can become more complicated than creation.

48.143 Rosetta Rules

Rosetta Rule

Prototype creates objects through copying.

Rosetta Rule

Prototype reduces expensive reconstruction.

Rosetta Rule

Shallow copy shares references.

Rosetta Rule

Deep copy duplicates references.

Rosetta Rule

Modern Java often avoids direct use of Cloneable.

48.144 Mental Compression

Mental Compression	
Problem	
	⇒
Object Creation Is Expensive	
Need	
	⇒
Reuse Existing Configuration	
Method	
	⇒
Clone Existing Object	
Challenge	
	⇒
Shallow vs Deep Copy	
Result	
	⇒
Fast Object Duplication	

48.145 Creational Patterns – The Big Picture

At this point we have learned:

- Singleton

- Factory Method

- Abstract Factory

- Builder

- Prototype

Most students now face a new problem.

They understand each pattern.

But they do not know when to use them.

48.146 The Common Mistake

Common Trap

Many developers memorize patterns like exam questions.

Singleton

↓

One Object

Builder

↓

Many Parameters

Factory

↓

Create Object

This approach fails in real systems.

Real software starts with problems.

Patterns are selected afterward.

48.147 The Five Questions

Whenever object creation becomes a problem, ask:

Question 1

Do I need exactly one object?

Question 2

Do I want to hide which object gets created?

Question 3

Do I need families of related objects?

Question 4

Is construction becoming complex?

Question 5

Is object creation expensive?

The answers guide pattern selection.

48.148 Pattern Selection Table

Problem	Pattern	Main Goal
One Instance	Singleton	Instance Control
Unknown Type	Factory Method	Hide Creation
Related Families	Abstract Factory	Consistency
Complex Construction	Builder	Step-by-Step Creation
Expensive Creation	Prototype	Duplication

48.149 Mental Model

Mental Model

Singleton

↓

How Many?

Factory

↓

Which One?

Abstract Factory

↓

Which Family?

Builder

↓

How Built?

Prototype

↓

Copy Or Rebuild?

48.150 A Real System Example

Imagine building an online game.

Game Configuration

↓

Singleton

Character Creation

↓

Factory

Platform-Specific UI

↓

Abstract Factory

Game Server Setup

↓

Builder

Monster Templates

↓

Prototype

Notice carefully.

Patterns rarely compete.

They usually cooperate.

48.151 The Deeper Insight

All Creational Patterns are attempting to solve the same fundamental problem.

Object Creation Is More Complex Than It First Appears.

The keyword:

1 new

looks simple.
Large software systems prove otherwise.

Deep Insight

The entire Creational Pattern family exists because object creation eventually becomes a design problem.
Not merely a syntax problem.

48.152 Rosetta Compression

Mental Compression

Creational Patterns
↓
Manage Creation
Singleton
↓
Control Quantity
Factory
↓
Control Type
Abstract Factory
↓
Control Families
Builder
↓
Control Construction
Prototype
↓
Control Duplication

48.153 Pattern #1 – Adapter

Adapter is often the first Structural Pattern that developers encounter in real projects.
Not because it is important.
Because incompatibility is everywhere.

48.154 The Disease

Suppose your application expects:

1 `processPayment()` ;

A third-party payment library provides:

```
1 executeTransaction();
```

Both perform the same job.
The interfaces are different.
The software cannot connect them.

48.155 The Real Problem

The problem is not behavior.
The problem is communication.
One side speaks:
Language A
The other speaks:
Language B
Someone must translate.

48.156 Real Life Analogy

Consider:
Indian Plug
European Socket
Electricity exists.
Power exists.
Devices exist.
The interfaces differ.
We insert:
An Adapter
between them.
Nothing else changes.

48.157 Mental Model

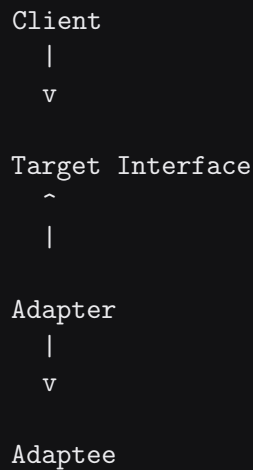
Mental Model

System A
↓
Adapter
↓
System B
Adapter translates.
Adapter does not create new functionality.

48.158 The Participants

Adapter contains:
Client
Target Interface
Adaptee
Adapter

48.159 Visualization



The Client only understands the Target Interface.
The Adapter speaks both languages.

48.160 The Target Interface

The application expects:

```
1 interface PaymentProcessor
2 {
3     void processPayment();
4 }
```

Every client already depends on this interface.

48.161 The Adaptee

The third-party library provides:

```
1 class PaymentGateway
2 {
3     public void
4     executeTransaction()
5     {
6         System.out.println(
7             "Payment Done"
8         );
9     }
10 }
```

Notice carefully.
The functionality exists.
The interface differs.

48.162 The Adapter

```
1 class PaymentAdapter
2 implements PaymentProcessor
3 {
4     private PaymentGateway
5         gateway;
6
7     public PaymentAdapter(
8         PaymentGateway gateway)
9     {
10         this.gateway =
11             gateway;
12     }
13
14     @Override
15     public void processPayment()
16     {
17         gateway
18             .executeTransaction();
19     }
20 }
```

The adapter translates one request into another.

48.163 Client Code

```
1 PaymentGateway gateway =
2 new PaymentGateway();
3
4 PaymentProcessor processor =
5 new PaymentAdapter(gateway);
6
7 processor.processPayment();
```

The client believes it is using:

```
1 PaymentProcessor
```

The gateway remains unchanged.

48.164 The Hidden Power

Suppose tomorrow we switch to:

- Stripe
- PayPal
- Razorpay
- Square

Only adapters change.
The application remains stable.

48.165 Object Adapter

The example above uses:

- Composition

- This is called:

- Object Adapter

- Because the adapter stores an object reference.

48.166 Class Adapter

Historically another version exists.

- Inheritance-based Adapter.

```
1 class Adapter
2 extends Adaptee
3 implements Target
4 {
5 }
```

- Java rarely uses this approach.

- Composition is usually superior.

48.167 Where Adapter Appears

Legacy Systems

- Payment Gateways

- Database Drivers

- REST APIs

- Third-Party Libraries

- Operating System Wrappers

- Cloud SDKs

- Almost every large project contains adapters.

48.168 The Deep Insight

Adapter does not solve a business problem.

- Adapter solves an integration problem.

Deep Insight

Software engineering often means connecting systems that were never designed to work together.

Adapter is the translator.

48.169 Adapter vs Factory

Mental Model

Factory

↓

Creates Objects

Adapter

↓

Connects Objects

One creates.

One integrates.

48.170 When NOT To Use

Common Trap

Do not create adapters simply to rename methods.

If both systems already share a compatible interface,
an adapter adds unnecessary complexity.

Use it only when genuine interface incompatibility exists.

48.171 Rosetta Rules

Rosetta Rule

Adapter connects incompatible interfaces.

Rosetta Rule

Adapter translates requests.

Rosetta Rule

Adapter preserves existing code.

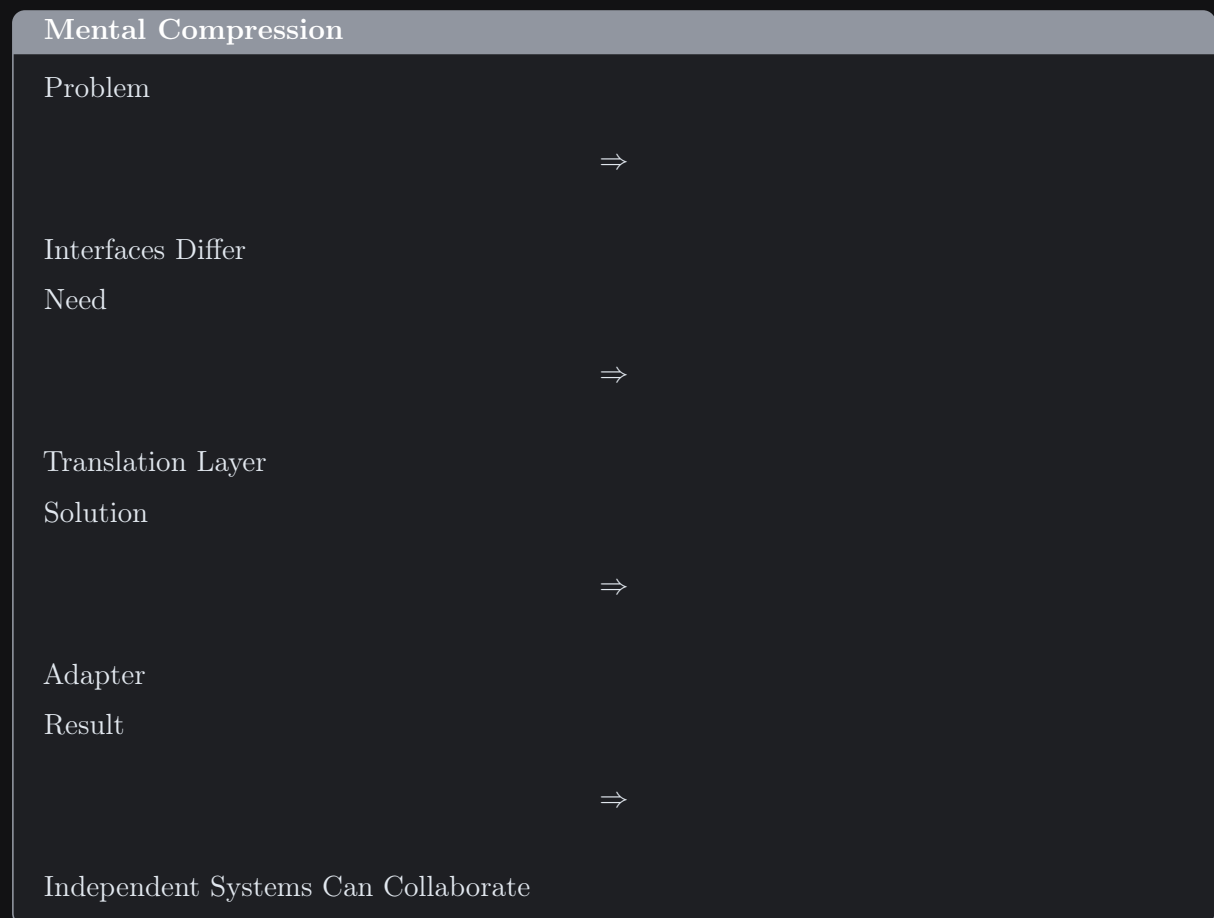
Rosetta Rule

Adapter is primarily an integration pattern.

Rosetta Rule

Composition-based adapters are usually preferred in Java.

48.172 Mental Compression



48.173 Pattern #2 – Facade

After Adapter solves:

"Incompatible Interfaces"

another problem appears.

The interfaces are compatible.

There are simply too many of them.

48.174 The Disease

Imagine starting a car.

Suppose the process required:

Activate Fuel System

Initialize Engine

Check Battery

Verify Sensors

Start Ignition

Enable Cooling

Enable Dashboard

Only then the car starts.

That would be ridiculous.

Instead we press:

Start Button
One operation.
Many internal actions.

48.175 The Real Problem

Large software systems often contain subsystems.

Database Layer
Authentication Layer
Logging Layer
Caching Layer
Networking Layer
The client should not need to understand all of them.

48.176 Mental Model

Mental Model

Subsystem Complexity
↓
Facade
↓
Simple Interface
Facade hides complexity.
Facade does not remove complexity.

48.177 A Simple Example

Suppose a Home Theater contains:

Projector
Amplifier
DVD Player
Lights
Screen

48.178 Without Facade

```
1 projector.on();  
2  
3 amplifier.on();  
4  
5 dvdPlayer.on();  
6  
7 screen.down();  
8  
9 lights.dim();
```

The client must know every single subsystem.

48.179 The Facade

```
1 class HomeTheaterFacade
2 {
3     private Projector projector;
4
5     private Amplifier amplifier;
6
7     private DVDPlayer dvd;
8
9     private Screen screen;
10
11    private Lights lights;
12
13    public void watchMovie()
14    {
15        projector.on();
16
17        amplifier.on();
18
19        dvd.on();
20
21        screen.down();
22
23        lights.dim();
24    }
25 }
```

48.180 Client Code

```
1 HomeTheaterFacade theater =
2 new HomeTheaterFacade();
3
4 theater.watchMovie();
```

One command.
Many operations.

48.181 Visualization

Client

|
v

Facade

|
+-----+
| |
v v

Projector

Amplifier

|
v

DVD Player

|
v

Lights

|
v

Screen

The client sees one door.
The subsystem remains hidden.

48.182 A Software Example

Suppose we want:

User Login

Internally it requires:

Database Query

Password Verification

Session Creation

Logging

Security Checks

Without Facade:

The client must call all components manually.

With Facade:

```
1 authFacade.login(  
2     username,  
3     password  
4 );
```

Much simpler.

48.183 The Hidden Benefit

Subsystems evolve.

Implementation changes.

Internal APIs change.

The facade remains stable.

Clients stay untouched.

48.184 Facade vs Adapter

This is a common interview question.

Mental Model

Adapter

↓

Makes Existing Interfaces Compatible

Facade

↓

Provides Simpler Interface

Adapter changes shape.

Facade hides complexity.

48.185 Real World Examples

Spring Framework

 Hibernate

 JDBC Utilities

 Cloud SDKs

 Operating Systems

Modern frameworks are filled with facades.

48.186 The Deep Insight

Complexity is unavoidable.

Exposure is optional.

Facade accepts complexity.

Then hides it behind a cleaner interface.

Deep Insight

A good facade creates the illusion that a complicated system is simple.

That illusion is valuable.

48.187 When NOT To Use

Common Trap

Do not place all application logic inside a giant facade.

A facade should simplify access.

It should not become a God Object.

48.188 Rosetta Rules

Rosetta Rule

Facade provides a simplified interface to a subsystem.

Rosetta Rule

Facade reduces client complexity.

Rosetta Rule

Facade hides subsystem details.

Rosetta Rule

Facade promotes loose coupling.

Rosetta Rule

Facade simplifies usage, not implementation.

48.189 Mental Compression

Mental Compression

Adapter solves:

Incompatible Interfaces

Facade solves:

Excessive Complexity

Adapter changes the shape.

Facade hides the shape.

Rosetta Rule

Adapter says:
"Let me translate."

Rosetta Rule

Facade says:
"Let me simplify."

48.190 The Second Kingdom

Deep Insight

The first kingdom answered:

How are objects created?

The second kingdom answers:

How are objects connected?

Mental Model

Suppose a city already exists.
The problem is no longer creating citizens.
The problem becomes:
Who talks to whom?
Who depends on whom?
Who should know about whom?

Rosetta Rule

Structural patterns organize relationships.

48.191 The Real Disease

Deep Insight

As software grows, relationships become more dangerous than objects.

Example:

```
1 A -> B
2 B -> C
3 C -> D
4 D -> E
```

Now changing E may affect:

$$D \rightarrow C \rightarrow B \rightarrow A$$

This is dependency explosion.

48.192 Why Structural Patterns Exist

Deep Insight

Structural patterns attempt to answer:

How can objects collaborate without becoming tangled?

Mental Model

Roads connect cities.
Too few roads:
Nothing communicates.
Too many roads:
Traffic chaos.
Good architecture requires controlled connections.

48.193 The Structural Pattern Family

The Gang of Four identified seven major structural patterns:

- Adapter
- Facade
- Decorator
- Proxy
- Composite
- Bridge
- Flyweight

Each attacks a different relationship problem.

48.194 A Bigger Perspective

Deep Insight

Most beginners think:

Programming = Writing Classes

Experts think:

Programming = Managing Dependencies

The class itself is rarely the problem.
The connections are.

48.195 Rosetta Summary

Rosetta Rule

Creational Patterns answer:
How do we create objects?

Rosetta Rule

Structural Patterns answer:
How do we connect objects?

Rosetta Rule

Bad architecture creates dependency webs.

Rosetta Rule

Good architecture creates controlled relationships.

Chapter 49

Creational Patterns

49.1 The First Kingdom

Deep Insight

The first category of design patterns deals with a surprisingly difficult question:
How should objects be created?
Most beginners assume object creation is trivial.

```
1 Car car = new Car();
```

Looks simple.

Yet many large software systems spend enormous effort controlling how objects are created.

Rosetta Rule

Creation is a responsibility.
Not merely a language feature.

49.2 The Hidden Cost Of new

Deep Insight

Every use of:

```
1 new Something()
```

creates a dependency.

The current class now knows:

- the concrete class
- the constructor
- the initialization process

This knowledge creates coupling.

49.3 A Small Thought Experiment

Suppose we write:

```
1 Payment payment =  
2   new CreditCardPayment();
```

Today:

```
1 CreditCardPayment
```

Tomorrow:

```
1 UPIPayment
```

Next year:

```
1 CryptoPayment
```

Every place containing:

```
1 new CreditCardPayment()
```

must now be modified.

49.4 The Real Goal

Deep Insight

The goal of creational patterns is not object creation.
The goal is controlled object creation.

The system should depend on:

```
1 Payment
```

instead of:

```
1 CreditCardPayment
```

49.5 The Family Tree

Creational patterns answer different questions.

Singleton	One object only
Factory Method	Which object?
Abstract Factory	Which family?
Builder	How to construct?
Prototype	How to clone?

49.6 A Mental Model

Mental Model

Imagine a car company.
Different problems require different solutions.
One CEO only.
Singleton.
Choosing which car model to manufacture.
Factory.
Choosing an entire product line.
Abstract Factory.
Building a custom car step by step.
Builder.
Copying an existing car design.
Prototype.

49.7 Why These Patterns Matter

Deep Insight

The history of OOP shows a gradual realization.
The difficult part of software is rarely storing data.
The difficult part is managing relationships.
Object creation is one of those relationships.

Rosetta Rule

Whenever **new** appears repeatedly, ask whether object creation itself has become a design problem.

49.8 Rosetta Summary

Rosetta Rule

Creational patterns solve object creation problems.

Rosetta Rule

The goal is not creating objects.
The goal is reducing coupling.

Rosetta Rule

Different creation problems lead to different patterns.

Rosetta Rule

Before learning a pattern, first identify the disease it cures.

Chapter 50

Singleton Pattern

50.1 The First Disease

Deep Insight

Sometimes a system must guarantee that exactly one object exists.
Not zero.
Not two.
Exactly one.

Examples:

- Configuration Manager
- Logger
- Application Settings
- Cache Manager
- Database Connection Pool

50.2 The Beginner Solution

Common Trap

A beginner writes:

```
1  Logger logger1 = new Logger();  
2  
3  Logger logger2 = new Logger();  
4  
5  Logger logger3 = new Logger();
```

Nothing prevents multiple objects.

50.3 The Hidden Problem

Imagine:

```

1 Config config1 = new Config();
2
3 Config config2 = new Config();

```

Now:

```

1 config1.setTheme("Dark");
2
3 config2.setTheme("Light");

```

Question:

Which configuration is correct?

The application now contains conflicting realities.

50.4 The Real Requirement

Deep Insight

The requirement is not:

Create a configuration object.

The requirement is:

Create the only configuration object.

This is a very different problem.

50.5 A Mental Model

Mental Model

Think about a country.

There may be millions of citizens.

But there is only one constitution.

If multiple constitutions existed simultaneously, chaos would emerge.

Some software objects behave exactly like that.

50.6 The Key Observation

Deep Insight

The disease exists because constructors are public.

Anyone can write:

```

1 new Config();

```

Therefore anyone can create more objects.

50.7 The First Barrier

The constructor must be hidden.

```

1 public class Config
2 {
3     private Config()
4     {
5
6     }
7 }

```

Now outside code cannot do:

```

1 new Config();

```

50.8 The New Problem

Common Trap

If nobody can call the constructor, how will the object ever be created?
We solved one problem.
Created another.

50.9 The Core Idea

The class must create its own object.

```

1 public class Config
2 {
3     private static Config instance =
4         new Config();
5
6     private Config()
7     {
8
9     }
10 }

```

Notice:

The object lives inside the class itself.

50.10 Accessing The Instance

Users need a controlled access point.

```

1 public static Config getInstance()
2 {
3     return instance;
4 }

```

Usage:

```

1 Config config =
2     Config.getInstance();

```

50.11 Visualization

Mental Model

Normal Class:

User

↓

new

↓

Object A

User

↓

new

↓

Object B

Singleton:

User

↓

getInstance()

↓

Same Object

50.12 Why This Works

Deep Insight

Only one object is ever created.

Every request returns the same reference.

Therefore:

$$\text{Number of Objects} = 1$$

50.13 The Famous Logger Example

Suppose:

```
1 Logger logger =  
2   Logger.getInstance();
```

Every part of the application writes to the same logger.

This creates a single source of truth.

50.14 Why Singleton Became Popular

Early software systems often contained global data.

Singleton appeared to provide:

- centralized access

- controlled creation
- shared state

For many years it became one of the most widely used patterns.

50.15 The Trap Nobody Mentions

Common Trap

Singleton often behaves like a global variable.
Global variables create hidden dependencies.
Hidden dependencies create maintenance problems.

Deep Insight

Many beginners learn Singleton and start using it everywhere.
This is usually a mistake.

50.16 A Deeper Perspective

The real lesson is not:

How do I create one object?

The real lesson is:

How do I control object creation?

Singleton is merely one answer.

50.17 When Singleton Is Appropriate

Singleton is useful when:

- the object represents a unique system resource
- shared state is required
- multiple copies would be incorrect
- centralized access is beneficial

Examples:

```
1 Configuration
2 Logger
3 CacheManager
```

50.18 When Singleton Is A Mistake

Avoid Singleton when:

- multiple instances may eventually be needed
- testing becomes difficult
- hidden global state appears
- dependency injection is available

50.19 Rosetta Summary

Rosetta Rule

Singleton solves:

Exactly one object must exist.

Rosetta Rule

The constructor becomes private.

Rosetta Rule

The class creates and stores its own instance.

Rosetta Rule

Users access the object through:

```
getInstance()
```

Rosetta Rule

Singleton controls quantity.

50.20 Mental Compression

Mental Compression

Disease:

Multiple copies create inconsistency.

Solution:

Allow only one object.

Mechanism:

Private Constructor

+

Static Instance

+

getInstance()

Question Singleton Answers:

How do we guarantee exactly one object?

Chapter 51

Factory Method Pattern

51.1 The Second Disease

Deep Insight

Singleton answered:

How many objects should exist?

Factory Method answers a different question:

Which object should be created?

At first this seems like a small question.

In large systems it becomes one of the biggest sources of coupling.

51.2 The Hidden Dependency

Suppose we write:

```
1 Payment payment =  
2     new CreditCardPayment();
```

Looks harmless.

But the current class now knows:

- the concrete class
- its constructor
- its implementation details

A dependency has been created.

51.3 Why This Becomes Dangerous

Imagine six months later.

The company adds:

```
1 UIPayment
2 PayPalPayment
3 CryptoPayment
```

Every place containing:

```
1 new CreditCardPayment()
```

must be modified.
The code becomes brittle.

51.4 The Real Question

Deep Insight

The problem is not object creation.
The problem is choosing the correct object.

Different situations require different concrete classes.
Yet client code should not know which class is chosen.

51.5 The Naive Solution

A common beginner approach:

```
1 if(type.equals("CARD"))
2 {
3     return new CreditCardPayment();
4 }
5 else if(type.equals("UPI"))
6 {
7     return new UIPayment();
8 }
9 else if(type.equals("PAYPAL"))
10 {
11     return new PayPalPayment();
12 }
```

Initially this feels reasonable.

51.6 The Growing Problem

Common Trap

Every new payment type requires modifying existing code.
The system becomes:

$$\textit{Modification} \rightarrow \textit{Risk} \rightarrow \textit{Bugs}$$

This violates one of the most important principles:

Open For Extension
Closed For Modification

51.7 A Mental Model

Mental Model

Imagine a restaurant.

Customers order food.

Customers should not decide:

- which farmer grows vegetables
- which truck delivers ingredients
- which chef cooks the meal

Customers simply request food.

The restaurant decides the details.

Factory Method works exactly the same way.

51.8 The Core Insight

Deep Insight

Object creation itself can be delegated.

Instead of asking:

Who creates objects?

Ask:

Who should be responsible for creation?

Responsibility moves away from client code.

51.9 The Product Interface

Client code should depend on abstractions.

```
1 public interface Payment
2 {
3     void pay();
4 }
```

Notice:

The client knows only:

```
1 Payment
```

Not:

```
1 CreditCardPayment
2 UPIPayment
3 PayPalPayment
```

51.10 Concrete Products

Different implementations exist.

```
1 public class CreditCardPayment
2     implements Payment
3 {
4     public void pay()
5     {
6         System.out.println("Card");
7     }
8 }
```

```
1 public class UPIPayment
2     implements Payment
3 {
4     public void pay()
5     {
6         System.out.println("UPI");
7     }
8 }
```

51.11 The Factory

Now we create a dedicated creator.

```
1 public abstract class PaymentFactory
2 {
3     public abstract Payment createPayment();
4 }
```

This class does not know the exact product.
It only defines the creation contract.

51.12 Concrete Factories

Each factory specializes creation.

```
1 public class CardFactory
2     extends PaymentFactory
3 {
4     public Payment createPayment()
5     {
6         return new CreditCardPayment();
7     }
8 }
```

```
1 public class UPIFactory
2     extends PaymentFactory
3 {
4     public Payment createPayment()
5     {
```



```

6         return new UPIPayment();
7     }
8 }

```

51.13 The Important Shift

Notice what changed.

Before:

```

1 new CreditCardPayment()

```

appeared inside client code.

Now:

```

1 factory.createPayment()

```

appears instead.

The client no longer depends on concrete classes.

51.14 Client Code

```

1 PaymentFactory factory =
2     new CardFactory();
3
4 Payment payment =
5     factory.createPayment();
6
7 payment.pay();

```

The client interacts only with abstractions.

51.15 Visualization

Mental Model

Without Factory Method:

Client

↓

CreditCardPayment

With Factory Method:

Client

↓

Factory

↓

CreditCardPayment

The dependency is redirected.

51.16 The Deeper Benefit

Deep Insight

Factory Method is not really about creation.
It is about isolation.

The client becomes isolated from implementation details.
Implementation details can change.
Client code remains stable.

51.17 Connection To Polymorphism

Factory Method becomes powerful because of polymorphism.
The client sees:

```
1 Payment
```

The factory decides whether that object is:

```
1 CreditCardPayment
2 UPIPayment
3 PayPalPayment
```

This is runtime flexibility.

51.18 Factory Method And OCP

Deep Insight

One of the deepest goals of OOP is:

Open For Extension

Closed For Modification

Adding a new payment type should require:

- new class
- new factory

Existing client code remains unchanged.

51.19 Where You Already Use It

Many frameworks use Factory Method.
Examples:

- JDBC Drivers

- Spring Beans
- Collection Framework Internals
- GUI Toolkits
- Logging Frameworks

Often developers use factories without realizing it.

51.20 When Factory Method Is Overkill

Common Trap

If the application only creates one simple class,
Factory Method adds unnecessary complexity.

Patterns solve problems.
Patterns are not goals.

51.21 The Deep Insight

Deep Insight

Factory Method separates:

	Using an object
from	
	Choosing an object

These are different responsibilities.

This separation dramatically reduces coupling.

51.22 Rosetta Summary

Rosetta Rule

Factory Method solves:

Which object should be created?

Rosetta Rule

Client code depends on abstractions.
Not concrete classes.

Rosetta Rule

Creation responsibility moves into factories.

Rosetta Rule

Factory Method reduces coupling.

Rosetta Rule

Factory Method supports the Open-Closed Principle.

51.23 Mental Compression

Mental Compression

Disease:

Client knows concrete classes.

Problem:

High coupling.

Solution:

Move creation into specialized factories.

Client sees:

1 **Payment**

Factory chooses:

1 **CreditCardPayment**

2 **UPIPayment**

3 **PayPalPayment**

Question Factory Method Answers:

Which object should be created?

Chapter 52

Abstract Factory Pattern

52.1 The Third Disease

Deep Insight

Factory Method solved:

Which object should be created?

A new problem appears when objects come in families.

Sometimes choosing one object is not enough.
We must choose an entire compatible collection.

52.2 The Family Problem

Suppose we are building a GUI framework.
We need:

- Button
- Checkbox
- TextField

For Windows:

- WindowsButton
- WindowsCheckbox
- WindowsTextField

For Mac:

- MacButton
- MacCheckbox
- MacTextField

Notice something.
These objects belong to families.

52.3 The Dangerous Situation

Common Trap

Imagine:

```
1  WindowsButton
2  MacCheckbox
3  WindowsTextField
```

appearing together.
The UI becomes inconsistent.

The system must guarantee compatibility.

52.4 The Real Requirement

Deep Insight

The requirement is not:
Choose one object.
The requirement is:
Choose an entire family of related objects.

This is fundamentally different from Factory Method.

52.5 A Mental Model

Mental Model

Imagine furnishing a house.
You can buy:
Victorian furniture
or
Modern furniture
or
Japanese furniture
Each style represents a family.
Buying random pieces from every family creates visual chaos.
Abstract Factory ensures the entire set comes from the same family.

52.6 The Product Interfaces

The client should depend only on abstractions.

```
1  public interface Button
2  {
3      void paint();
4  }
```

```
1 public interface Checkbox
2 {
3     void paint();
4 }
```

52.7 Concrete Products

Windows family:

```
1 public class WindowsButton
2     implements Button
3 {
4 }
```

```
1 public class WindowsCheckbox
2     implements Checkbox
3 {
4 }
```

Mac family:

```
1 public class MacButton
2     implements Button
3 {
4 }
```

```
1 public class MacCheckbox
2     implements Checkbox
3 {
4 }
```

52.8 The Abstract Factory

Now we define the family creator.

```
1 public interface GUIFactory
2 {
3     Button createButton();
4
5     Checkbox createCheckbox();
6 }
```

Notice:

The factory creates multiple related products.

52.9 Windows Factory

```

1 public class WindowsFactory
2     implements GUIFactory
3 {
4     public Button createButton()
5     {
6         return new WindowsButton();
7     }
8
9     public Checkbox createCheckbox()
10    {
11        return new WindowsCheckbox();
12    }
13 }

```

52.10 Mac Factory

```

1 public class MacFactory
2     implements GUIFactory
3 {
4     public Button createButton()
5     {
6         return new MacButton();
7     }
8
9     public Checkbox createCheckbox()
10    {
11        return new MacCheckbox();
12    }
13 }

```

52.11 Client Code

```

1 GUIFactory factory =
2     new WindowsFactory();
3
4 Button b =
5     factory.createButton();
6
7 Checkbox c =
8     factory.createCheckbox();

```

The client never knows the concrete classes.

52.12 The Deep Insight

Deep Insight

Factory Method creates one product.
Abstract Factory creates a product ecosystem.

The focus shifts from:

Object Consistency

to

Family Consistency

52.13 Factory Method vs Abstract Factory

Factory Method	Abstract Factory
Creates one object	Creates families
Solves type selection	Solves family selection
Single product focus	Product ecosystem focus

52.14 Where It Appears

Common examples:

- Cross-platform GUI frameworks
- Database driver families
- Theme engines
- Cloud provider SDKs
- Enterprise frameworks

52.15 When NOT To Use It

Common Trap

If there is only one product type,
Abstract Factory is unnecessary.

The pattern becomes useful only when multiple related products must remain compatible.

52.16 Rosetta Summary

Rosetta Rule

Factory Method chooses a product.

Rosetta Rule

Abstract Factory chooses a family.

Rosetta Rule

The goal is compatibility between products.

Rosetta Rule

Clients depend on abstractions, not concrete families.

52.17 Mental Compression

Mental Compression

Singleton:

How many?

Factory Method:

Which one?

Abstract Factory:

Which family?

Factory Method creates:

One product.

Abstract Factory creates:

An ecosystem.

Chapter 53

Builder Pattern

53.1 The Fourth Disease

Deep Insight

Singleton solved:

How many objects?

Factory Method solved:

Which object?

Abstract Factory solved:

Which family?

Builder solves a completely different problem:

How should a complex object be constructed?

53.2 Constructor Explosion

Consider a class:

```
1 public class Student
2 {
3     private String name;
4     private int age;
5     private String branch;
6     private double cgpa;
7 }
```

Initially:

```
1 Student(String name)
```

Later:

```
1 Student(String name,  
2     int age)
```

Then:

```
1 Student(String name,  
2     int age,  
3     String branch)
```

Then:

```
1 Student(String name,  
2     int age,  
3     String branch,  
4     double cgpa)
```

Soon the constructor list becomes ridiculous.

53.3 The Bigger Problem

Suppose only name is mandatory.

Everything else is optional.

How should we create:

```
1 Student
```

with:

- name only
- name + age
- name + cgpa
- name + branch
- name + age + branch

The number of combinations grows rapidly.

53.4 The Traditional Solution

Many beginners create dozens of constructors.

```
1 Student(String name)  
2  
3 Student(String name,  
4     int age)  
5  
6 Student(String name,  
7     double cgpa)  
8  
9 Student(String name,  
10    int age,  
11    String branch)
```

This quickly becomes difficult to maintain.

53.5 The Real Requirement

Deep Insight

The user should be able to construct an object step by step.
Not all information is known at once.

53.6 A Mental Model

Mental Model

Imagine ordering a custom laptop.
You choose:

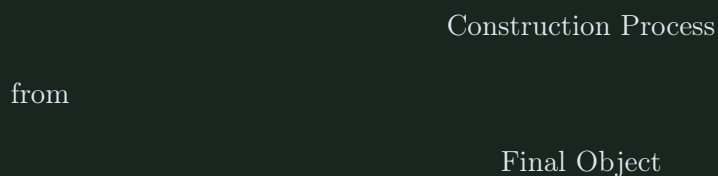
- RAM
- Processor
- Storage
- Graphics Card
- Display

The laptop is assembled gradually.
Nobody expects you to specify every possible configuration in a giant constructor.

53.7 The Core Idea

Deep Insight

Separate:



The Builder becomes responsible for construction.

53.8 The Product

```
1 public class Student
2 {
3     private String name;
4     private int age;
5     private String branch;
6     private double cgpa;
7
8     private Student()
9     {
```

```

10     }
11 }
12 }

```

Notice:
Users cannot directly create objects.

53.9 The Builder Class

```

1 public static class Builder
2 {
3     private String name;
4     private int age;
5     private String branch;
6     private double cgpa;
7 }

```

The builder stores construction information.

53.10 Builder Methods

```

1 public Builder name(String name)
2 {
3     this.name = name;
4     return this;
5 }

```

```

1 public Builder age(int age)
2 {
3     this.age = age;
4     return this;
5 }

```

Notice:
Each method returns:

```

1 this

```

This enables chaining.

53.11 Method Chaining

```

1 Student s =
2     new Student.Builder()
3         .name("PS")
4         .age(19)
5         .branch("MNC")
6         .build();

```

This reads almost like English.

53.12 The Build Step

Eventually construction finishes.

```
1 public Student build()
2 {
3     Student s = new Student();
4
5     s.name = name;
6     s.age = age;
7     s.branch = branch;
8     s.cgpa = cgpa;
9
10    return s;
11 }
```

The Builder creates the final object.

53.13 Why This Feels Natural

Deep Insight

Builder mirrors how humans construct things.
We rarely provide every detail simultaneously.
We usually build incrementally.

53.14 The Hidden Benefit

Suppose age must always be positive.

Without Builder:

Validation appears everywhere.

With Builder:

```
1 public Builder age(int age)
2 {
3     if(age < 0)
4         throw new IllegalArgumentException();
5
6     this.age = age;
7
8     return this;
9 }
```

Validation becomes centralized.

53.15 Immutable Objects

One major use of Builder is creating immutable objects.

Deep Insight

Immutable means:
Once created,
the object cannot change.

Example:

```
1 String
2 LocalDate
3 BigInteger
```

Builders make immutable objects practical.

53.16 A Famous Example

Java itself uses Builder-like APIs.

Examples:

```
1 StringBuilder
2
3 HttpRequest.Builder
4
5 Stream.Builder
```

The pattern appears frequently in modern Java.

53.17 The Director

Classic Builder literature introduces:

```
1 Director
```

The Director controls the construction sequence.

Example:

```
1 director.buildGamingPC();
```

The Director knows the steps.

The Builder performs the steps.

53.18 Builder Versus Constructor

Constructor	Builder
All information at once	Incremental creation
Poor readability	Highly readable
Constructor explosion	Flexible combinations
Limited validation	Centralized validation

53.19 Builder Versus Factory

Deep Insight

Factory focuses on:

Which object?

Builder focuses on:

How to construct?

These are completely different questions.

53.20 When NOT To Use Builder

Common Trap

If an object contains:

- two fields
- one constructor
- no optional data

Builder adds unnecessary complexity.

53.21 The Deep Insight

Deep Insight

Builder exists because construction itself can become a complex responsibility.

The pattern separates:

Object Representation

from

Object Construction

This separation scales beautifully.

53.22 Rosetta Summary

Rosetta Rule

Builder solves:

How should a complex object be constructed?

Rosetta Rule

Builder prevents constructor explosion.

Rosetta Rule

Builder supports optional parameters.

Rosetta Rule

Builder enables readable object creation.

Rosetta Rule

Builder is heavily used with immutable objects.

53.23 Mental Compression

Mental Compression

Disease:

Complex object construction.

Problem:

Too many constructors.

Solution:

Construct step by step.

Factory asks:

Which object?

Builder asks:

How should the object be assembled?

Mental Image:

Factory = Manufacturing Plant

Builder = Assembly Line

Chapter 54

Prototype Pattern

54.1 The Fifth Disease

Deep Insight

Until now every pattern assumed:

Create a new object.

Prototype starts with a different question:

What if creating the object is expensive?

Sometimes construction is more costly than copying.

54.2 A Strange Observation

Imagine:

```
1 GameCharacter hero =  
2   new GameCharacter();
```

The constructor loads:

- textures
- sounds
- animations
- abilities
- equipment

Creation may take significant time.

Now suppose we need:

```
1 1000
```

similar characters.

Should we repeat the entire construction process?

54.3 The Real Question

Deep Insight

Instead of asking:

How do we create an object?

Prototype asks:

How do we duplicate an existing object?

54.4 A Mental Model

Mental Model

Imagine a photocopy machine.

Creating a new textbook from scratch:

- write every chapter
- design every diagram
- print every page

Extremely expensive.

Creating another copy:

Place the book in a copier.

Press a button.

Done.

Prototype follows the same philosophy.

54.5 The Traditional Approach

Suppose we have:

```
1 Enemy e1 =  
2     new Enemy("Orc",  
3         100,  
4         50);
```

To create another enemy:

```
1 Enemy e2 =  
2     new Enemy("Orc",  
3         100,  
4         50);
```

Again:

```
1 Enemy e3 =  
2     new Enemy("Orc",
```

```
3         100,  
4         50);
```

Repeated construction.
Repeated initialization.
Repeated cost.

54.6 The Prototype Idea

Deep Insight

Create one object.
Copy it whenever needed.

This object becomes the prototype.

54.7 Visualization

Mental Model

Traditional:
Constructor
↓
Object A
Constructor
↓
Object B
Constructor
↓
Object C
Prototype:
Prototype Object
↓
Copy
↓
Object A
↓
Copy
↓
Object B
↓
Copy
↓
Object C

54.8 The Prototype Interface

A common design:

```

1 public interface Prototype
2 {
3     Prototype clone();
4 }

```

Every prototype knows how to copy itself.

54.9 Concrete Prototype

```

1 public class Enemy
2     implements Prototype
3 {
4     private String type;
5     private int health;
6
7     public Enemy(String type,
8                 int health)
9     {
10         this.type = type;
11         this.health = health;
12     }
13
14     public Enemy clone()
15     {
16         return new Enemy(type,
17                           health);
18     }
19 }

```

54.10 Using The Prototype

```

1 Enemy original =
2     new Enemy("Orc",100);
3
4 Enemy copy =
5     original.clone();

```

The copy starts with the same state.

54.11 The Hidden Difficulty

Common Trap

Copying is not always simple.
Objects often contain references to other objects.

Example:

```

1 Student
2     |
3     ---> Address

```

Should the copy receive:

- the same Address object?
- a new Address object?

This creates one of the biggest challenges in the pattern.

54.12 Shallow Copy

A shallow copy duplicates only the outer object.

```
1 Student s2 = s1.clone();
```

Result:

```
1 Student A
2   |
3   +--> Address X
4
5 Student B
6   |
7   +--> Address X
```

Both students reference the same address.

54.13 Why Shallow Copy Can Be Dangerous

Suppose:

```
1 s2.getAddress()
2 .setCity("Delhi");
```

Unexpectedly:

```
1 s1
```

also sees the change.

The copies are not truly independent.

54.14 Deep Copy

Deep Insight

A deep copy duplicates everything.

Result:

```
1 Student A
2   |
3   +--> Address X
4
5 Student B
6   |
7   +--> Address Y
```

Now changes remain isolated.

54.15 Mental Model For Deep Copy

Mental Model

Shallow Copy:

Photocopy a house ownership document.

Both owners point to the same house.

Deep Copy:

Build an entirely new house.

Now ownership is independent.

54.16 Java And Cloneable

Java historically introduced:

```
1 Cloneable
```

and

```
1 clone()
```

However many experienced Java developers avoid it.

Why?

Because:

- confusing semantics
- shallow copy problems
- difficult inheritance behavior

54.17 Modern Java Practice

Modern Java often prefers:

```
1 copy constructors
```

Example:

```
1 public Student(Student other)
2 {
3     this.name = other.name;
4     this.age = other.age;
5 }
```

or

```
1 static factory methods
```

These approaches are usually clearer.

54.18 Where Prototype Appears

Common examples:

- Game development
- Graphics editors
- CAD software
- Object templates
- Serialization frameworks

54.19 Prototype Registry

A useful extension is maintaining a collection of prototypes.

```
1 Map<String, Prototype>
```

Example:

```
1 orc
2 dragon
3 wizard
4 archer
```

Instead of constructing new objects, the system clones the appropriate prototype.

54.20 When NOT To Use Prototype

Common Trap

If object creation is cheap,
Prototype adds complexity without benefit.

Simple objects rarely need cloning.

54.21 The Deep Insight

Deep Insight

All previous creational patterns focused on:

Construction

Prototype focuses on:

Replication

This is a completely different way of thinking.

54.22 Rosetta Summary

Rosetta Rule

Prototype solves:

How can we create new objects by copying existing ones?

Rosetta Rule

The prototype acts as a template.

Rosetta Rule

Cloning may be cheaper than construction.

Rosetta Rule

Shallow copy duplicates references.

Rosetta Rule

Deep copy duplicates entire object graphs.

54.23 Mental Compression

Mental Compression

Disease:

Object construction is expensive.

Question:

Must we build every object from scratch?

Solution:

Copy an existing object.

Key Concepts:

Prototype

Clone

Shallow Copy

Deep Copy

Mental Image:

Factory builds.

Prototype photocopies.

Chapter 55

Creational Patterns — The Big Picture

55.1 Why We Need A Summary

Deep Insight

Many students finish learning all five creational patterns and immediately forget when to use them.

Why?

Because they memorize implementations.

Not problems.

A pattern is not defined by its code.

A pattern is defined by the disease it cures.

55.2 The Common Mistake

Common Trap

Beginners ask:

Which pattern is better?

This question is meaningless.

Patterns solve different problems.

Asking:

Singleton or Builder?

is like asking:

Hammer or Screwdriver?

55.3 The Five Questions

Every creational pattern exists because of a question.

Pattern	Question
Singleton	How many objects should exist?
Factory Method	Which object should be created?
Abstract Factory	Which family should be created?
Builder	How should the object be assembled?
Prototype	Can we copy instead of construct?

55.4 The Evolution

Notice something interesting.

The patterns form a ladder.

Mental Model

Singleton

↓

Factory Method

↓

Abstract Factory

↓

Builder

↓

Prototype

Each pattern answers a deeper creation problem.

55.5 Singleton In One Sentence

Deep Insight

Singleton controls quantity.

Question:

Should more than one object exist?

Examples:

- 1 Configuration
- 2 Logger
- 3 Cache

55.6 Factory Method In One Sentence

Deep Insight

Factory Method controls type selection.

Question:

Which implementation should be created?

Examples:

- 1 Payment
- 2 Notification
- 3 Database Driver

55.7 Abstract Factory In One Sentence

Deep Insight

Abstract Factory controls family selection.

Question:

Which compatible group of objects should exist?

Examples:

- 1 Windows UI
- 2 Mac UI
- 3 Linux UI

55.8 Builder In One Sentence

Deep Insight

Builder controls construction complexity.

Question:

How should a complex object be assembled?

Examples:

- 1 HttpRequest
- 2 Student
- 3 Computer
- 4 Pizza

55.9 Prototype In One Sentence

Deep Insight

Prototype controls replication.

Question:

Can copying be cheaper than construction?

Examples:

```
1 Game Characters
2 Document Templates
3 Graphic Objects
```

55.10 The Selection Table

Suppose somebody asks:

Which pattern should I use?

Use this table.

Problem	Pattern
Only one instance allowed	Singleton
Choose implementation dynamically	Factory Method
Choose entire compatible family	Abstract Factory
Too many constructor parameters	Builder
Construction expensive	Prototype

55.11 A Real System Example

Imagine building an IDE.

We may have:

```
1 Theme
2 Editor
3 Compiler
4 Project
5 Settings
```

Different patterns solve different problems.

- Settings may use Singleton.
- Compiler may use Factory Method.
- Theme system may use Abstract Factory.
- Project configuration may use Builder.
- Templates may use Prototype.

Patterns frequently coexist.

55.12 Connection To SOLID

Deep Insight

Patterns did not appear randomly.
Most patterns are applications of SOLID principles.

Open Closed Principle

Factory Method supports:

Open $\rightarrow$ *Extension*

Closed $\rightarrow$ *Modification*

New implementations can be added without changing client code.

Dependency Inversion Principle

Factories allow code to depend on:

```
1 Payment
```

instead of:

```
1 CreditCardPayment
```

This is DIP in action.

Single Responsibility Principle

Builder separates:

Construction

from

Representation.

Two responsibilities become independent.

55.13 The Deep Insight

Deep Insight

All creational patterns share one philosophy.

Creation itself is a responsibility.

Most beginners place creation everywhere:

```
1 new
2 new
3 new
4 new
```

Patterns move creation into dedicated locations.

This reduces coupling.

This improves flexibility.

This improves maintainability.

55.14 Why Patterns Feel Difficult Initially

Common Trap

Students often think:

Why not just use new?

For small programs:

Using

1 `new`

everywhere is acceptable.

Patterns become valuable only when systems grow.

Patterns are scaling tools.

55.15 The Grand Mental Model

Mental Model

Imagine building a city.

Singleton:

How many city halls?

Factory Method:

Which building type?

Abstract Factory:

Which architectural style?

Builder:

How is the building assembled?

Prototype:

Can existing blueprints be copied?

55.16 Rosetta Compression

Mental Compression

Creational Patterns Control Creation.

Singleton:

How many?

Factory Method:

Which one?

Abstract Factory:

Which family?

Builder:

How assembled?

Prototype:

Copy or construct?

All five patterns exist to remove creation-related coupling.

55.17 Rosetta Rules

Rosetta Rule

Do not memorize pattern code.
Memorize pattern problems.

Rosetta Rule

A pattern is a reusable solution to a recurring design problem.

Rosetta Rule

Creation is a responsibility.
Creational patterns manage that responsibility.

Rosetta Rule

Patterns are not goals.
Patterns are tools.

Chapter 56

Abstract Factory Pattern

56.1 The Third Disease

Deep Insight

Factory Method solved one problem:
How do we create objects without tightly coupling client code to concrete classes?
However, large systems introduce a new problem.
Sometimes objects do not exist alone.
They belong to families.

Mental Model

Buying a keyboard is easy.
Buying an entire operating system theme is different.
Now the button, scrollbar, menu and window must all match each other.
The problem is no longer creating one object.
The problem is creating a compatible family.

56.2 The Family Consistency Problem

Suppose we are building a GUI framework.

We need:

```
1 Button
2 Checkbox
3 Menu
4 Window
```

For Windows:

```
1 WindowsButton
2 WindowsCheckbox
3 WindowsMenu
4 WindowsWindow
```

For macOS:

```
1 MacButton
2 MacCheckbox
```

```
3 MacMenu
4 MacWindow
```

Common Trap

Mixing families creates inconsistent systems.
A Mac button beside a Windows menu usually indicates poor design.

56.3 The Naive Solution

Many beginners write:

```
1 if(os.equals("windows"))
2 {
3     button = new WindowsButton();
4     menu = new WindowsMenu();
5 }
6 else
7 {
8     button = new MacButton();
9     menu = new MacMenu();
10 }
```

This works initially.
But every new platform expands the conditional logic.
Soon the entire system depends upon platform checks.

Common Trap

Large chains of conditionals are often symptoms of a missing abstraction.

56.4 The Real Requirement

Deep Insight

The client should not choose individual products.
The client should choose a family.

Instead of saying:

```
1 Give me a WindowsButton.
2 Give me a WindowsMenu.
3 Give me a WindowsCheckbox.
```

the client should say:

```
1 Give me the Windows family.
```

and let the factory produce everything consistently.

56.5 A Mental Model

Mental Model

Imagine ordering furniture.
You do not individually choose:

- chair style
- table style
- sofa style
- cupboard style

Instead:



The chosen family guarantees compatibility.
Abstract Factory works similarly.

56.6 The Product Interfaces

First define abstractions.

```
1 public interface Button
2 {
3     void render();
4 }
```

```
1 public interface Menu
2 {
3     void display();
4 }
```

Client code depends only on these interfaces.

56.7 Concrete Products

Windows implementation:

```
1 public class WindowsButton
2 implements Button
3 {
4     public void render()
5     {
6         System.out.println("Windows Button");
7     }
8 }
```

```

1 public class WindowsMenu
2 implements Menu
3 {
4     public void display()
5     {
6         System.out.println("Windows Menu");
7     }
8 }

```

56.8 The Abstract Factory

Now define the family creator.

```

1 public interface GUIFactory
2 {
3     Button createButton();
4
5     Menu createMenu();
6 }

```

Notice:

The factory creates an entire family.

Not a single product.

Rosetta Rule

Factory Method creates one product.

Abstract Factory creates related products.

56.9 Windows Factory

```

1 public class WindowsFactory
2 implements GUIFactory
3 {
4     public Button createButton()
5     {
6         return new WindowsButton();
7     }
8
9     public Menu createMenu()
10    {
11        return new WindowsMenu();
12    }
13 }

```

56.10 Mac Factory

```

1 public class MacFactory
2 implements GUIFactory
3 {

```

```

4     public Button createButton()
5     {
6         return new MacButton();
7     }
8
9     public Menu createMenu()
10    {
11        return new MacMenu();
12    }
13 }

```

56.11 Client Code

```

1 GUIFactory factory =
2     new WindowsFactory();
3
4 Button button =
5     factory.createButton();
6
7 Menu menu =
8     factory.createMenu();

```

The client never references:

```

1 WindowsButton
2 WindowsMenu

```

directly.

This reduces coupling dramatically.

56.12 The Important Shift

Deep Insight

Factory Method asks:

Which object should be created?

Abstract Factory asks:

Which family should be created?

56.13 Visualization

Mental Model

Think of the system as two layers.

Product Layer:

```
1 Button
2 Menu
3 Checkbox
```

Factory Layer:

```
1 WindowsFactory
2 MacFactory
3 LinuxFactory
```

Each factory knows how to build an entire family.

```
1           GUIFactory
2           |
3  -----
4  |         |         |
5  Windows   Mac       Linux
6  Factory   Factory   Factory
7  |         |         |
8  |         |         |
9  Family    Family    Family
10
11 (Button, Menu, Checkbox)
```

The client chooses one factory.

The factory guarantees consistency.

56.14 Why Frameworks Love Abstract Factory

Deep Insight

Frameworks must support multiple environments.

Examples:

- Windows
- macOS
- Linux
- Android
- Cloud Providers
- Databases

The framework cannot hardcode implementations.

It must remain flexible.

Examples:

- 1 JDBC Drivers
- 2 GUI Toolkits
- 3 Logging Frameworks
- 4 Dependency Injection Containers

All frequently use Abstract Factory ideas.

56.15 Connection To Dependency Injection

Deep Insight

Dependency Injection answers:

Who supplies dependencies?

Abstract Factory answers:

Who creates related dependencies?

These ideas often appear together.

A Dependency Injection container frequently behaves like a gigantic Abstract Factory.

56.16 Abstract Factory vs Factory Method

Factory Method	Creates one product
Abstract Factory	Creates a family of products

Example:

Factory Method:

```
1 createButton()
```

Abstract Factory:

```
1 createButton()  
2 createMenu()  
3 createCheckbox()
```

Rosetta Rule

Factory Method solves:

Which object?

Abstract Factory solves:

Which family?

56.17 The Cost Of Abstract Factory

Common Trap

Every design pattern introduces complexity.
Abstract Factory is no exception.

To support one product family we may create:

- Product Interfaces
- Concrete Products
- Factory Interface
- Concrete Factories

The number of classes grows rapidly.
For small applications this is unnecessary.

56.18 When NOT To Use Abstract Factory

Do not use Abstract Factory when:

- only one implementation exists
- product families do not exist
- future variation is unlikely
- object creation is trivial

Common Trap

Many developers learn a pattern and then force it everywhere.
Patterns are tools.
Not goals.

56.19 The Deep Insight

Deep Insight

Abstract Factory is really about consistency.

The problem is not object creation.
The problem is preventing incompatible combinations.
The pattern guarantees:

Products created together belong together.

This is the real value.

56.20 Relationship To SOLID

Deep Insight

Abstract Factory naturally supports several SOLID principles.

OCP

New families can be added without changing client code.

DIP

Clients depend on abstractions.

Not concrete classes.

SRP

Creation logic is separated from business logic.

Rosetta Rule

Many design patterns are practical applications of SOLID principles.

SOLID explains why patterns work.

Patterns show how to apply SOLID.

56.21 Rosetta Summary

Rosetta Rule

Factory Method creates one product.

Abstract Factory creates related products.

Rosetta Rule

The client chooses a factory.

The factory chooses the concrete products.

Rosetta Rule

Abstract Factory protects family consistency.

Rosetta Rule

The real problem is not creation.

The real problem is compatibility.

56.22 Mental Compression

Mental Compression

Factory Method:

One Factory $\rightarrow$ One Product

Abstract Factory:

One Factory $\rightarrow$ One Family

Singleton:

Exactly One Instance

Factory Method:

Hide Concrete Creation

Abstract Factory:

Hide Family Creation

Chapter 57

Builder Pattern

57.1 The Fourth Disease

Deep Insight

Factory Method and Abstract Factory solved creation flexibility.
Builder solves a completely different problem.
Construction complexity.

Consider a class with many fields.

```
1 public class Student
2 {
3     private String name;
4     private int age;
5     private String branch;
6     private double cgpa;
7     private String email;
8     private String phone;
9     private String address;
10 }
```

Creating such objects becomes painful.

57.2 Constructor Explosion

The beginner approach is:

```
1 public Student(
2     String name,
3     int age
4 )
5 {
6     ...
7 }
```

Then requirements grow.

```
1 public Student(
2     String name,
3     int age,
4     String branch
```

```

5  )
6  {
7      ...
8  }

```

Then:

```

1  public Student(
2      String name,
3      int age,
4      String branch,
5      double cgpa
6  )
7  {
8      ...
9  }

```

Soon:

```

1  public Student(
2      String name,
3      int age,
4      String branch,
5      double cgpa,
6      String email,
7      String phone,
8      String address
9  )
10 {
11     ...
12 }

```

Common Trap

Large constructors become unreadable.
Nobody remembers parameter order.

57.3 The Hidden Bug

Imagine:

```

1  Student s =
2  new Student(
3      "PS",
4      18,
5      "CSE",
6      9.7,
7      "999999999999",
8      "ps@gmail.com",
9      "Hyderabad"
10 );

```

Looks fine.
But notice:

```
1 phone
2 email
```

were accidentally swapped.
Compiler says nothing.
Program runs.
Bug survives.

Rosetta Rule

Some of the most dangerous bugs are syntactically correct.

57.4 The Real Requirement

Deep Insight

We want:
Readable creation.
Flexible creation.
Safe creation.

Ideally:

```
1 Student student =
2 ...
```

should explain itself.
The code should read like English.

57.5 A Mental Model

Mental Model

Imagine ordering a pizza.
You do not say:

```
1 Pizza(
2   large,
3   thin,
4   extra_cheese,
5   olives,
6   mushrooms,
7   paneer
8 )
```

Instead you build it step by step.
Choose size.
Choose crust.
Choose toppings.
Finally place the order.
Builder follows the same philosophy.

57.6 The Core Idea

Deep Insight

Separate construction from representation.

Instead of creating the object directly:

```
1 new Student(...)
```

Create a helper responsible for construction.
That helper is the Builder.

57.7 The Builder Class

```
1 public class StudentBuilder
2 {
3     private String name;
4     private int age;
5     private String branch;
6
7     public StudentBuilder name(String name)
8     {
9         this.name = name;
10        return this;
11    }
12
13    public StudentBuilder age(int age)
14    {
15        this.age = age;
16        return this;
17    }
18
19    public StudentBuilder branch(String branch)
20    {
21        this.branch = branch;
22        return this;
23    }
24 }
```

Notice:
Each method returns the builder itself.
This enables chaining.

57.8 Method Chaining

```
1 StudentBuilder builder =
2 new StudentBuilder();
3
4 builder
5     .name("PS")
6     .age(18)
7     .branch("CSE");
```


Every method returns:

```
1  this
```

allowing the next method call.

Mental Model

Each call adds one more piece.
Like assembling LEGO blocks.